IMPERIAL

Gaussian Processes & Bayesian Optimization

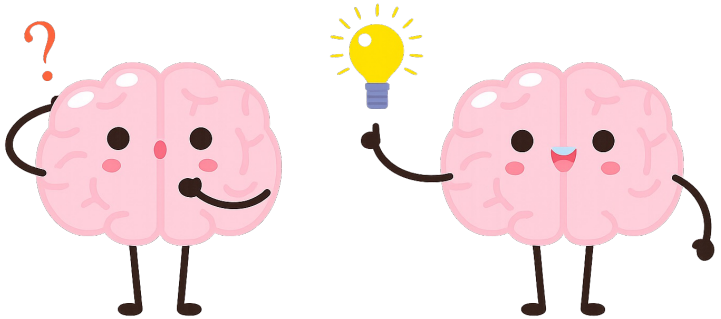


Sam Stricker

Agenda

1. Why Bayesian Optimization?
2. Gaussian Processes
3. Bayesian Optimization
4. What else is there...

Why Bayesian Optimization?

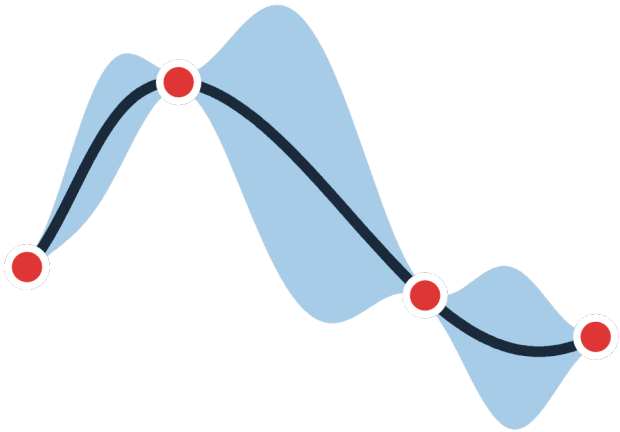


Expensive Functions

- **Neural Networks** are great if we have lots of data.
- **Gradients** are often not available
- In science often we can't do lots of **experiments**

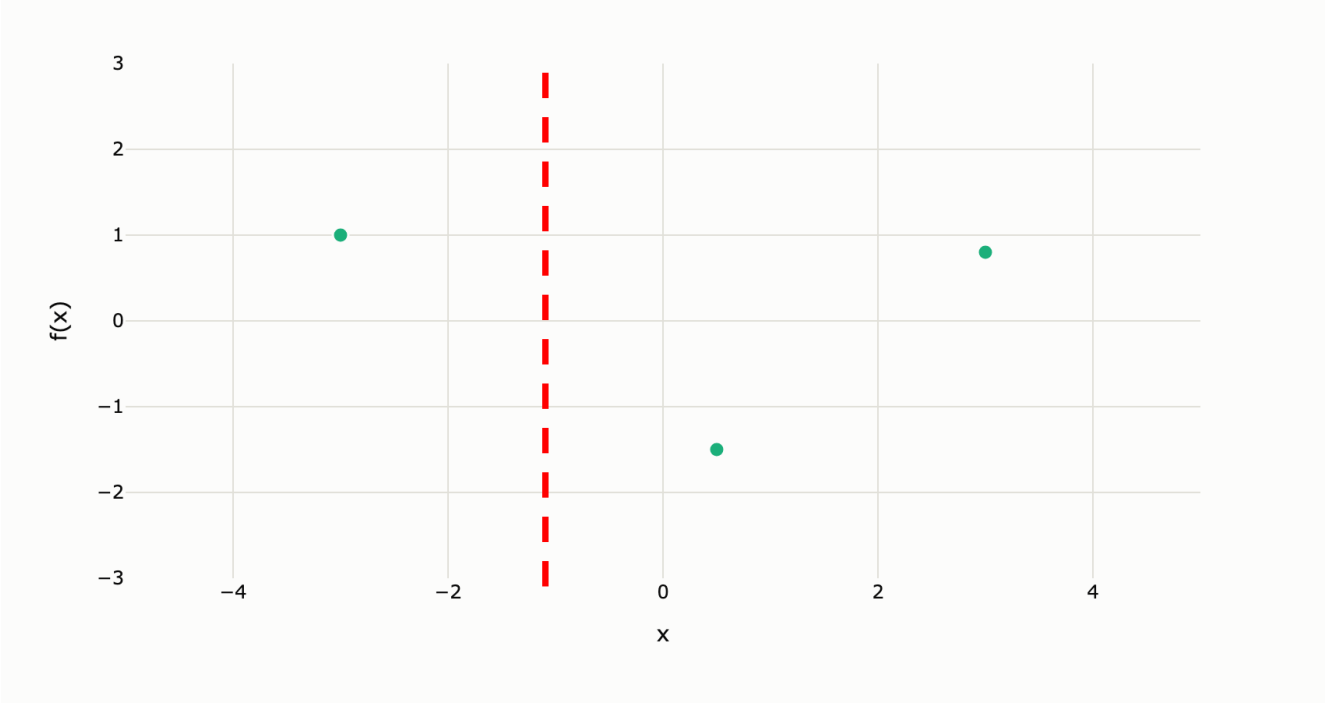


Gaussian Processes

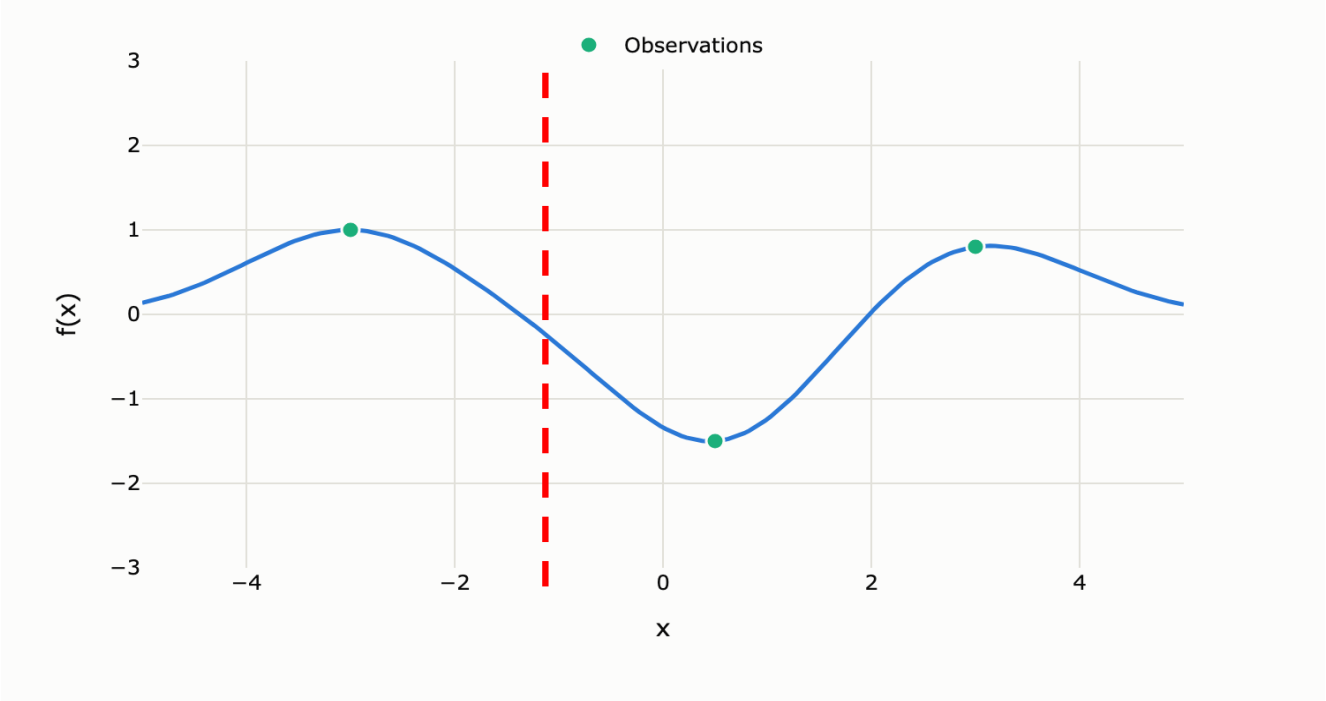


Expensive Process - Little Initial Data

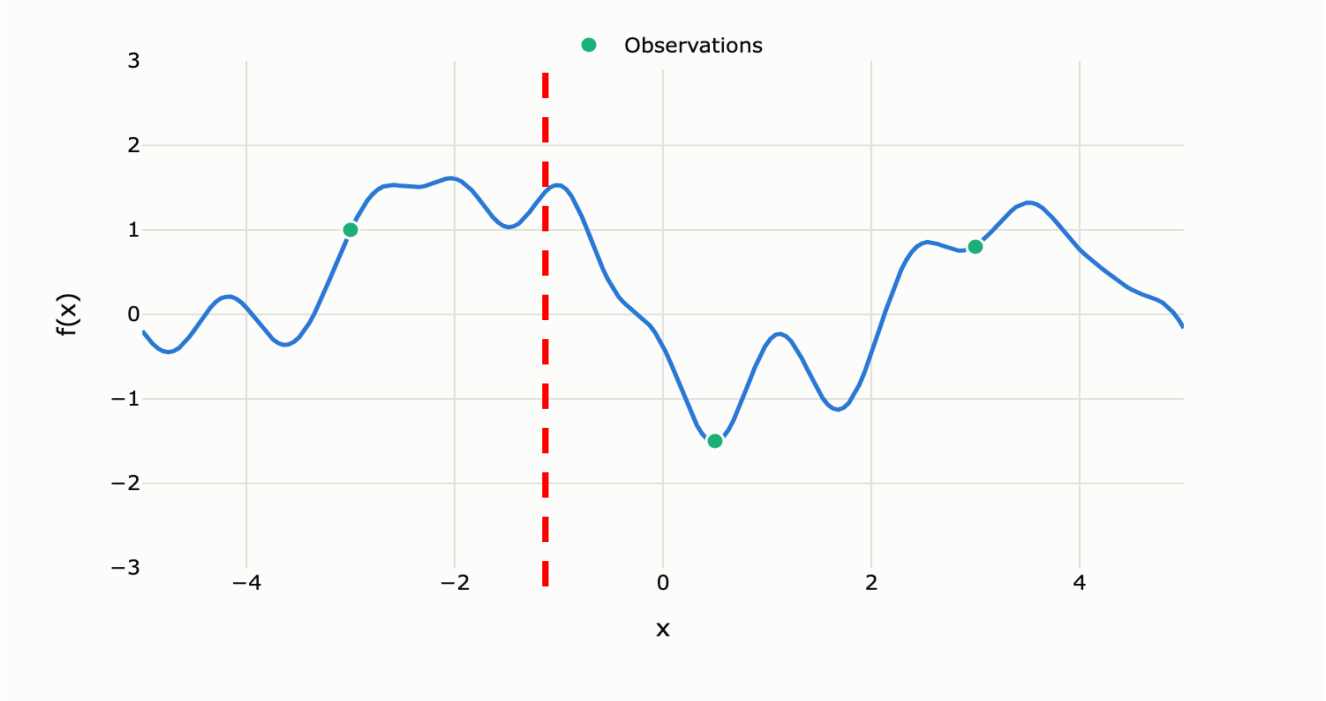




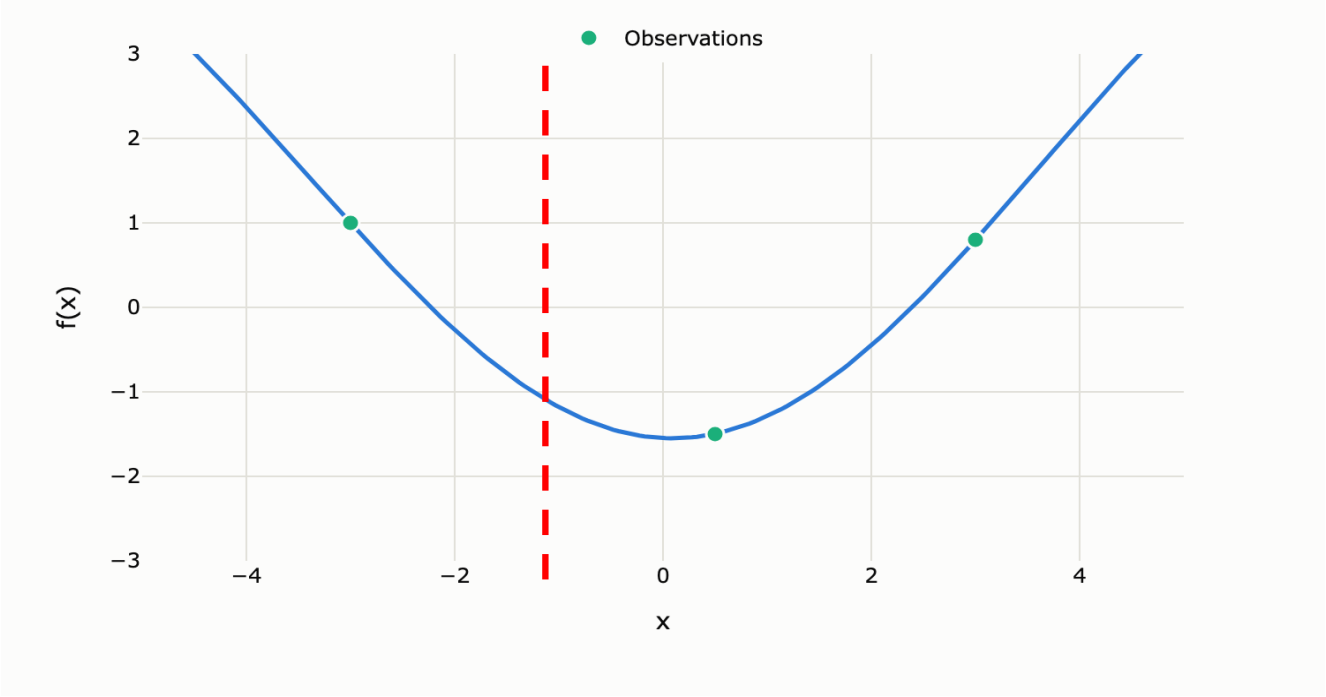
Expensive Process - Little Initial Data



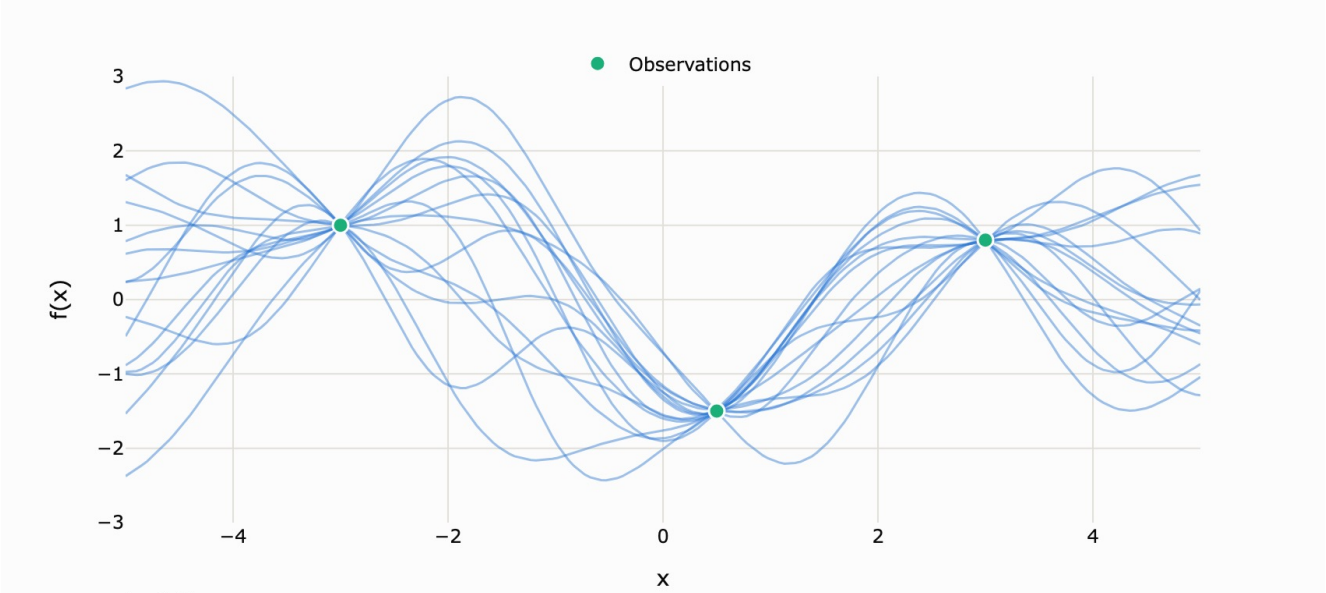
Expensive Process - Little Initial Data



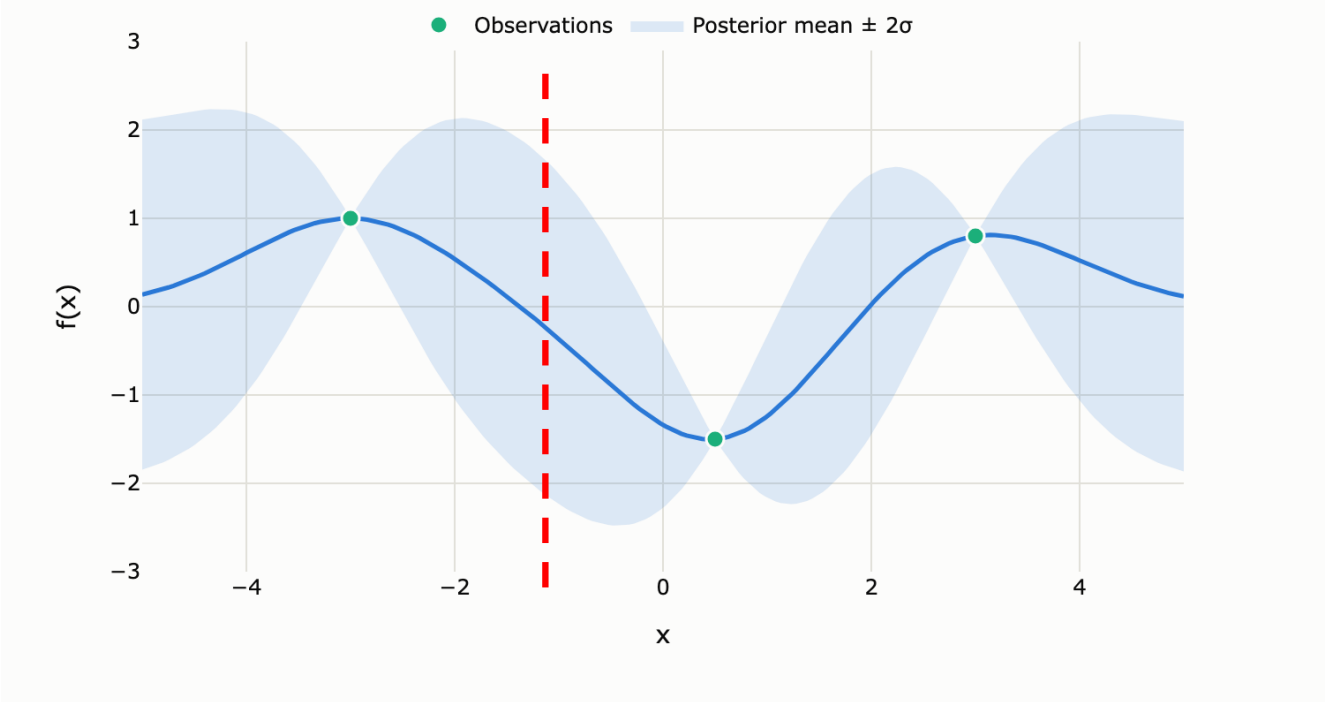
Expensive Process - Little Initial Data



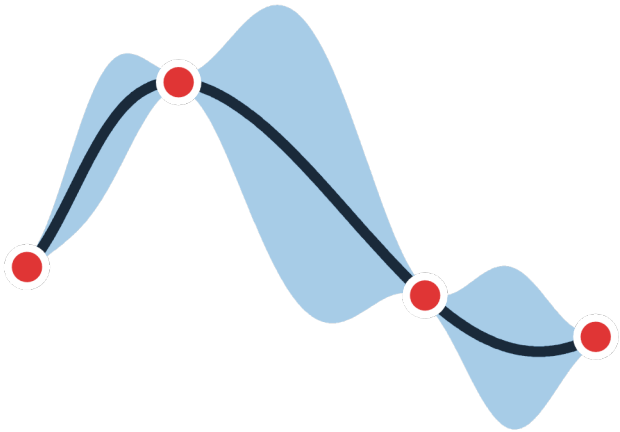
Expensive Process - Little Initial Data



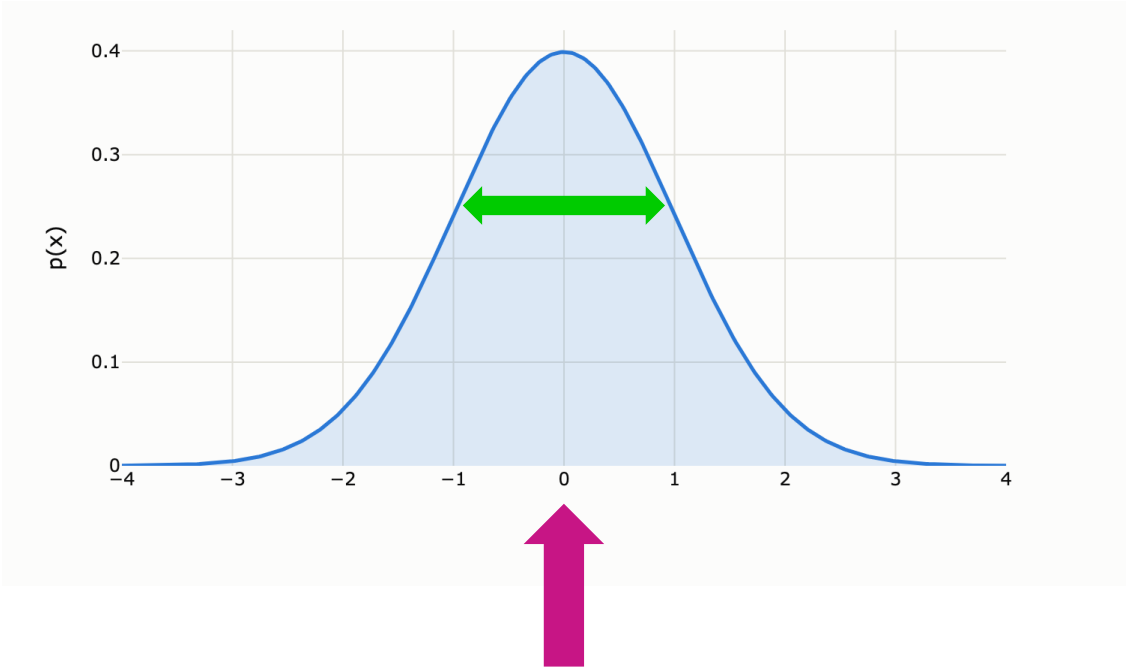
Expensive Process - Little Initial Data



How are we doing this?

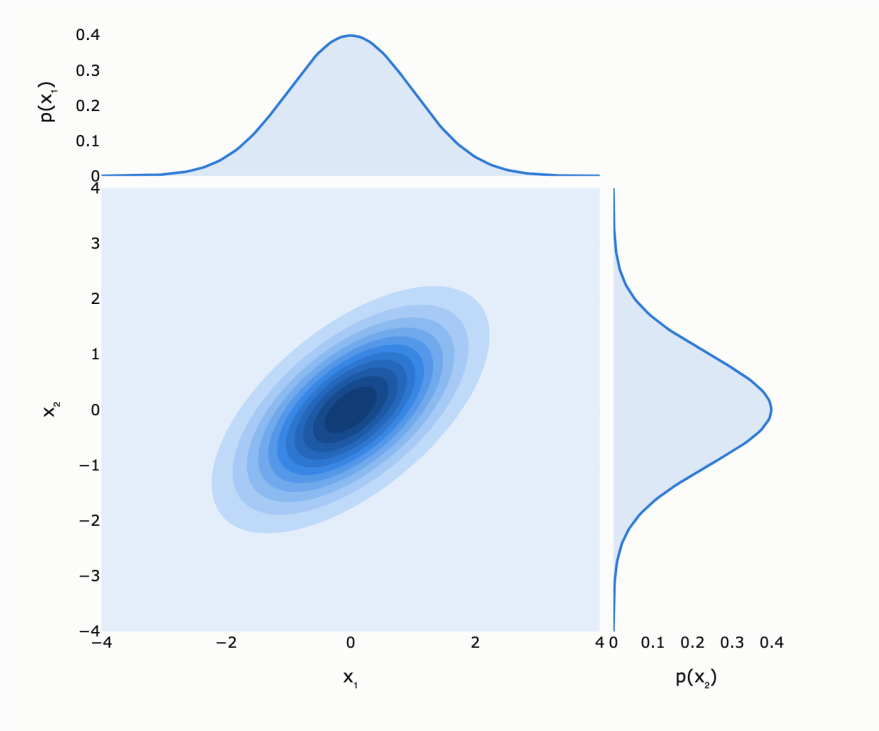


A Gaussian



$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Multivariate Gaussian

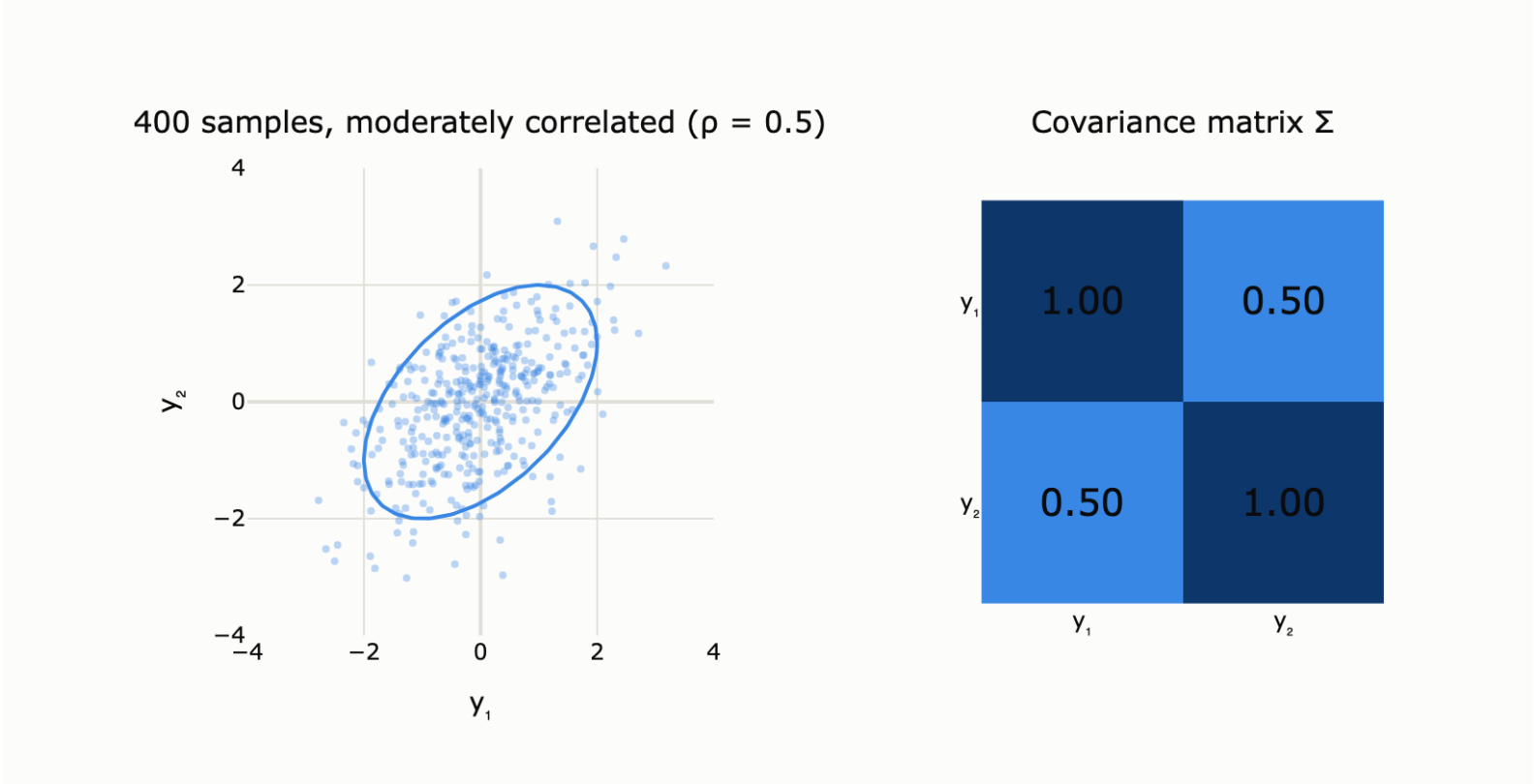


$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}(\det[\mathbf{\Sigma}])^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^T\mathbf{\Sigma}^{-1}(\mathbf{x}-\mu)\right]$$

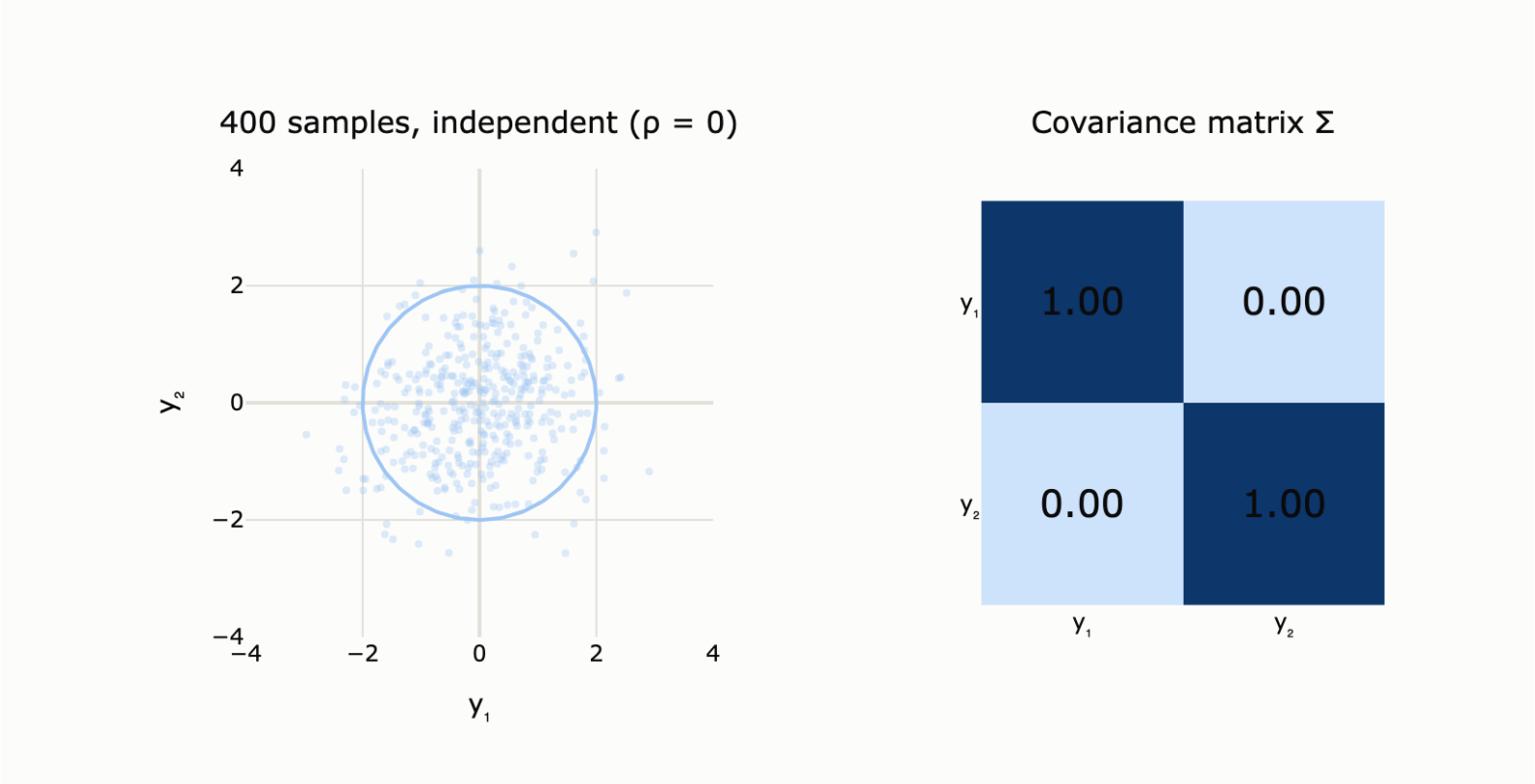
$$\boldsymbol{\mu} = \begin{bmatrix}\mu_x \\ \mu_y\end{bmatrix}$$

$$\Sigma = \begin{bmatrix}\sigma_x^2 & \sigma_{xy}^2 \\ \sigma_{xy}^2 & \sigma_y^2\end{bmatrix}$$

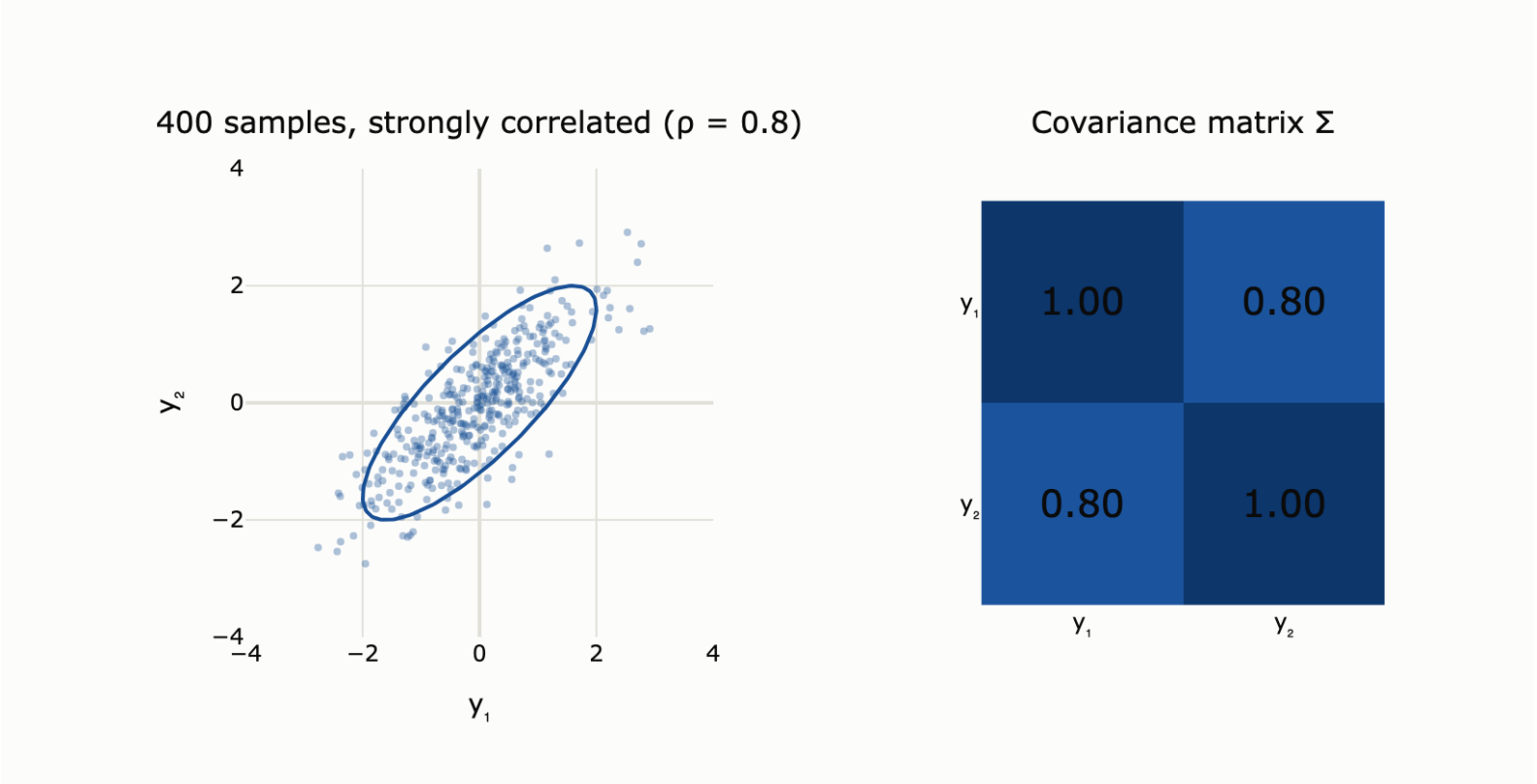
Multivariate Gaussian



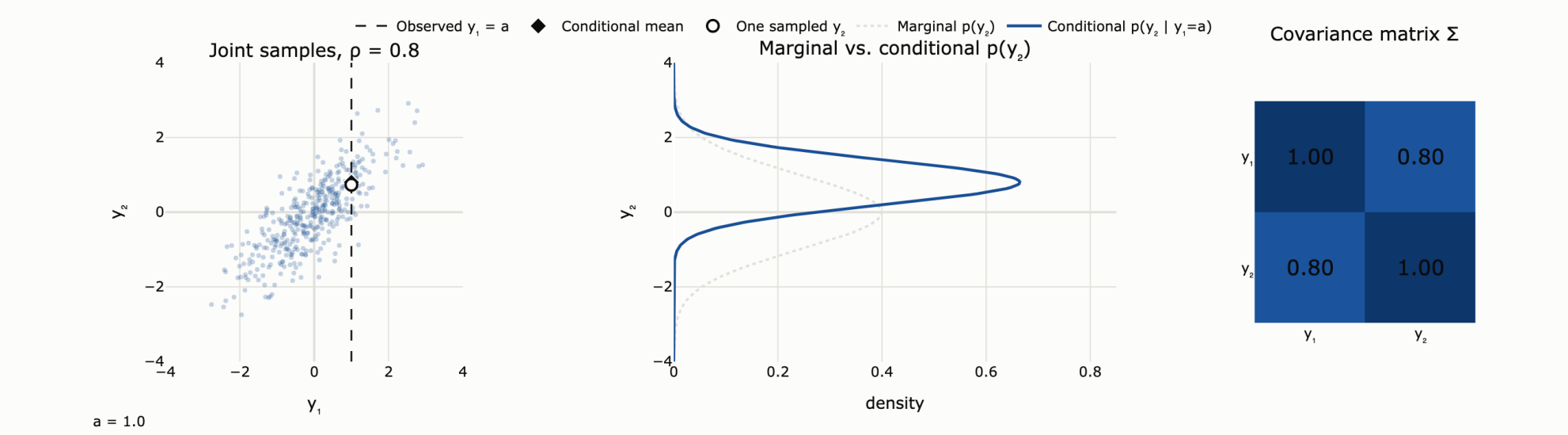
Multivariate Gaussian



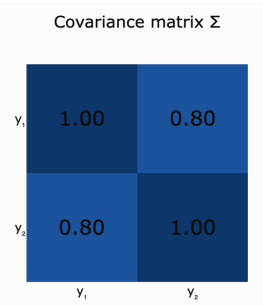
Multivariate Gaussian

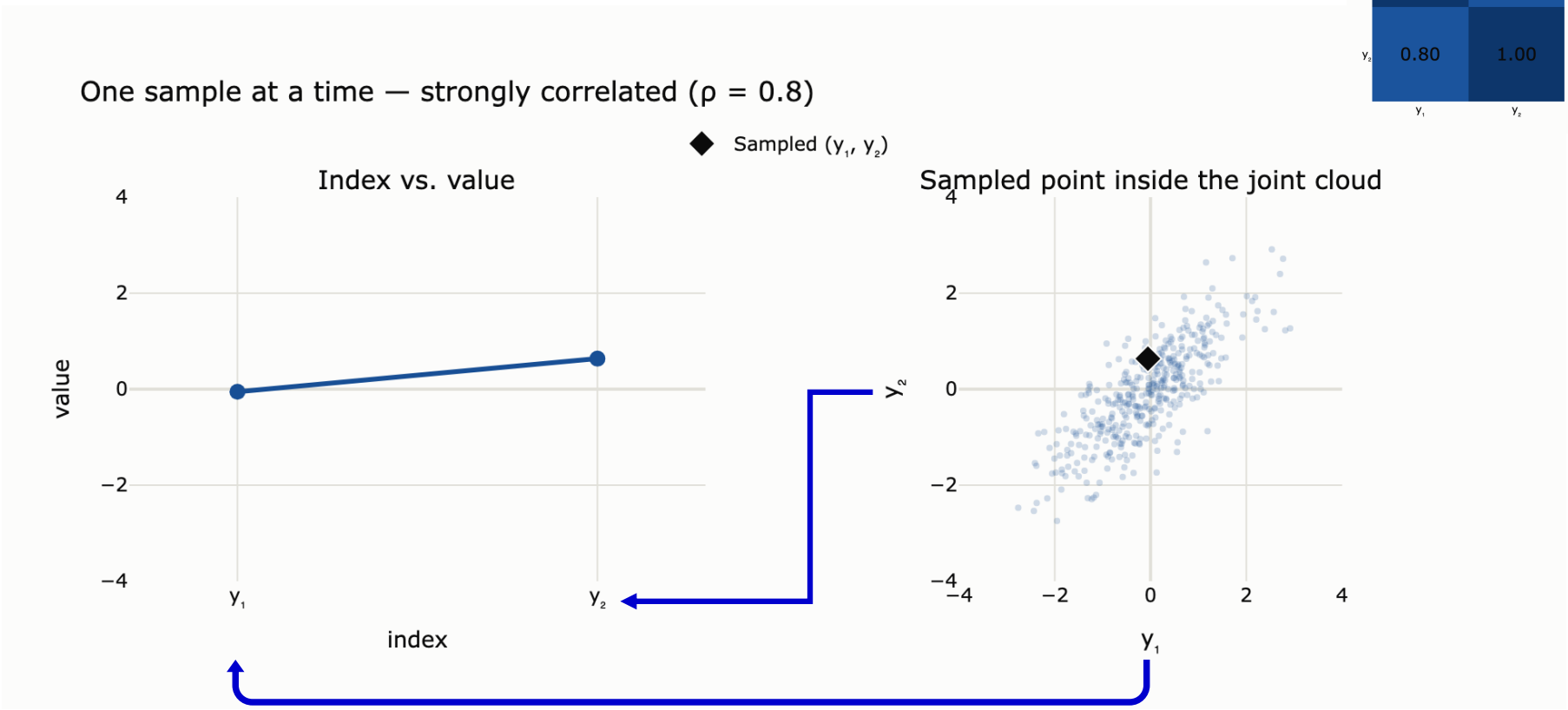


Marginal vs Conditional Distribution

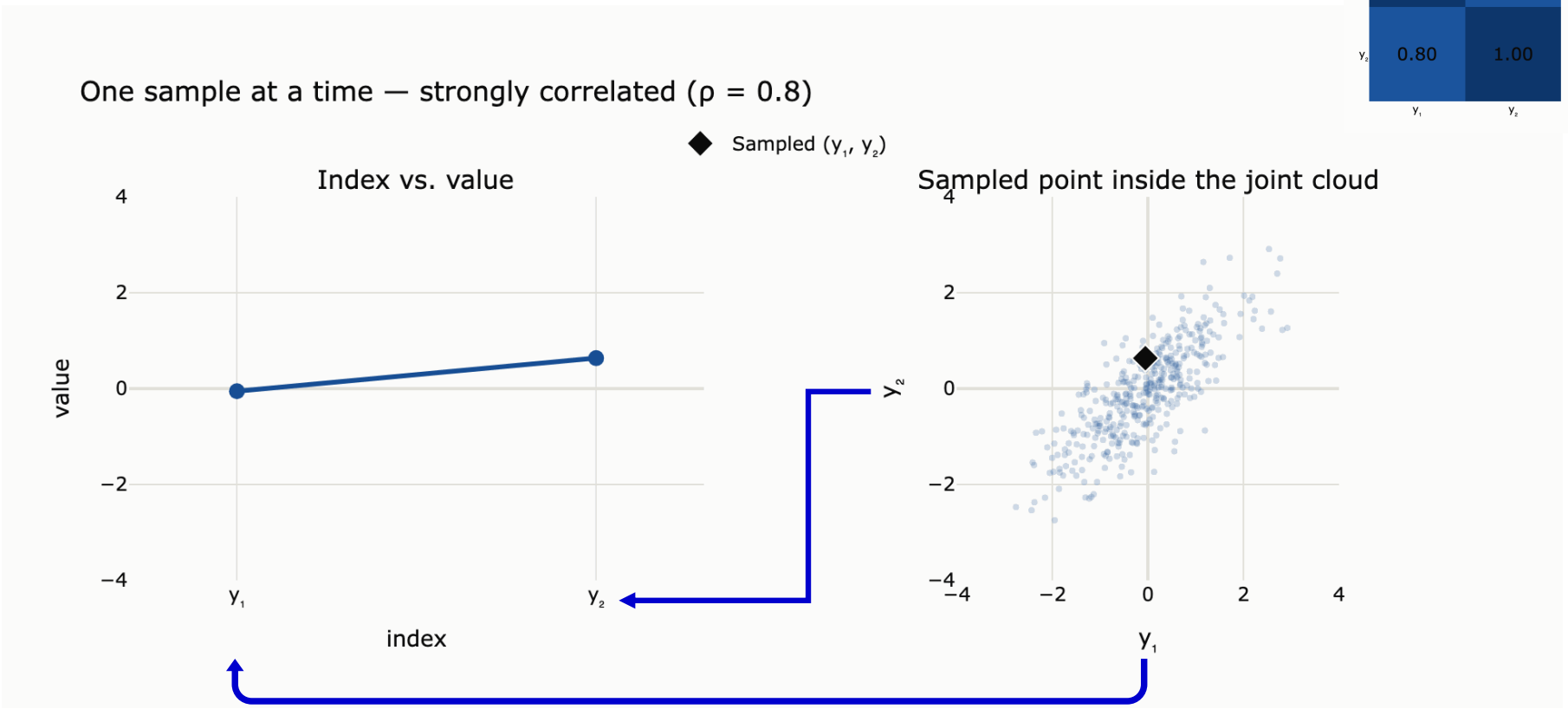


Multivariate Gaussian

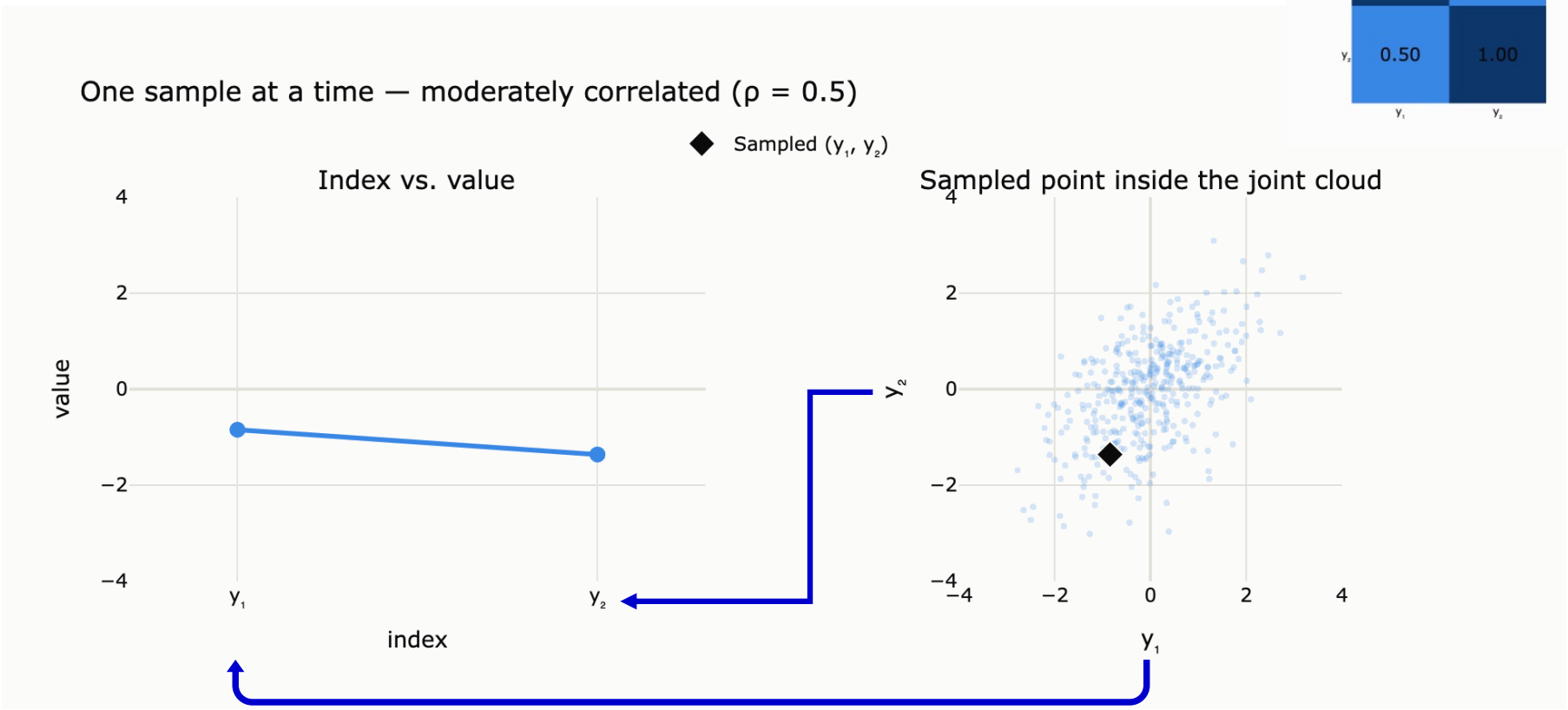




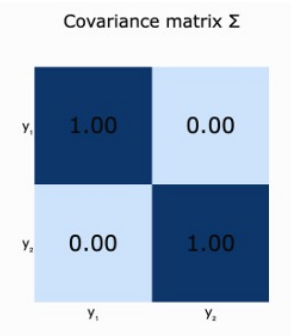
Multivariate Gaussian

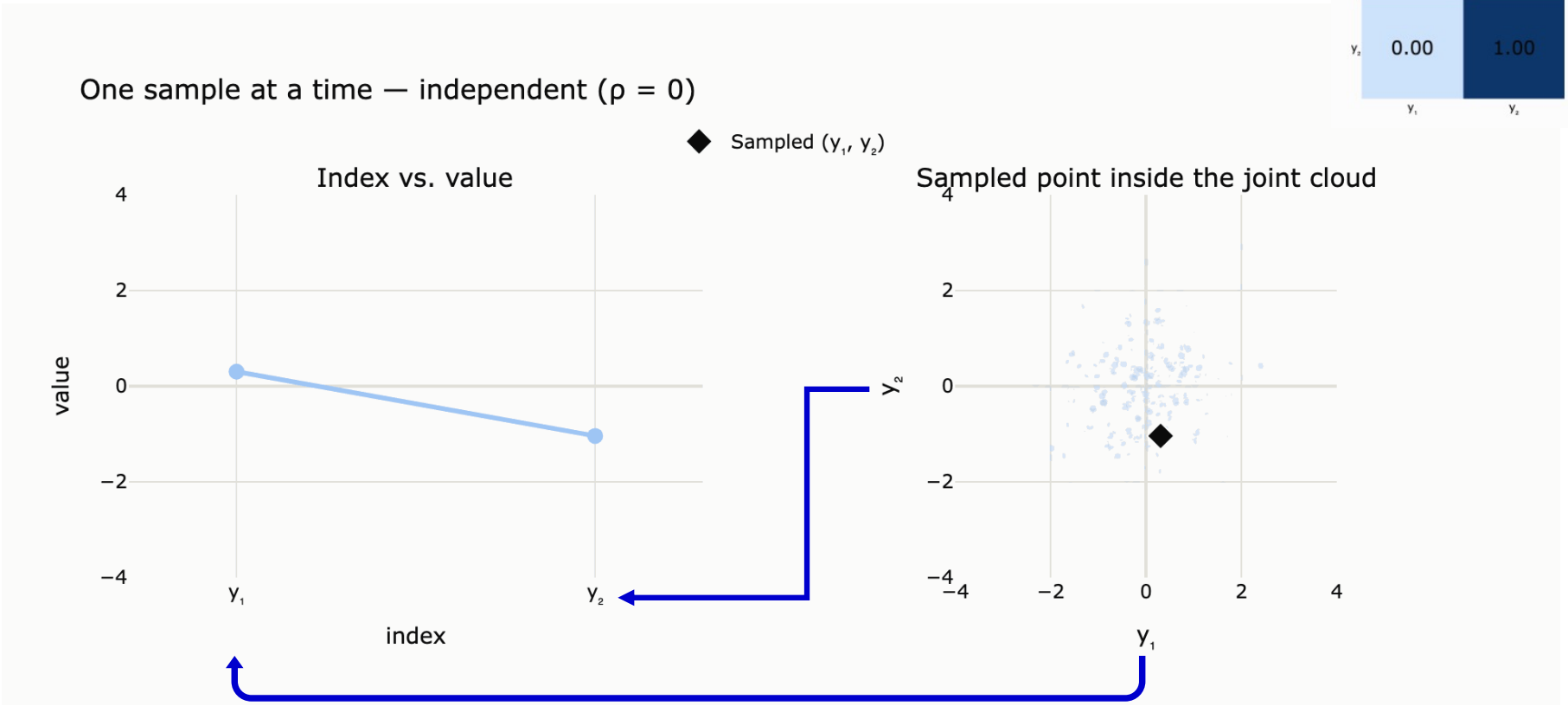


Multivariate Gaussian

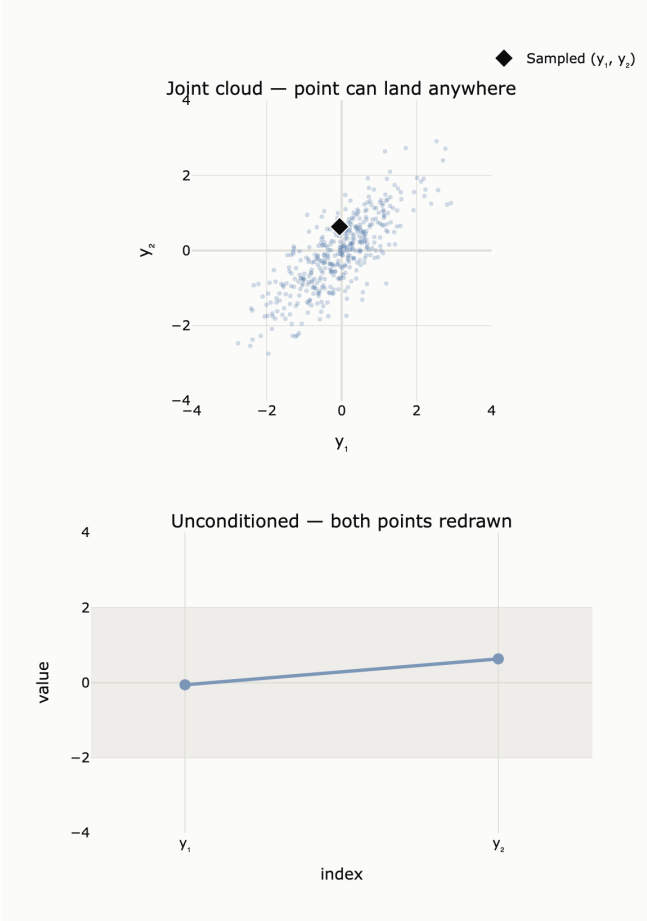


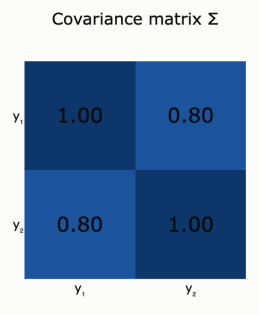
Multivariate Gaussian



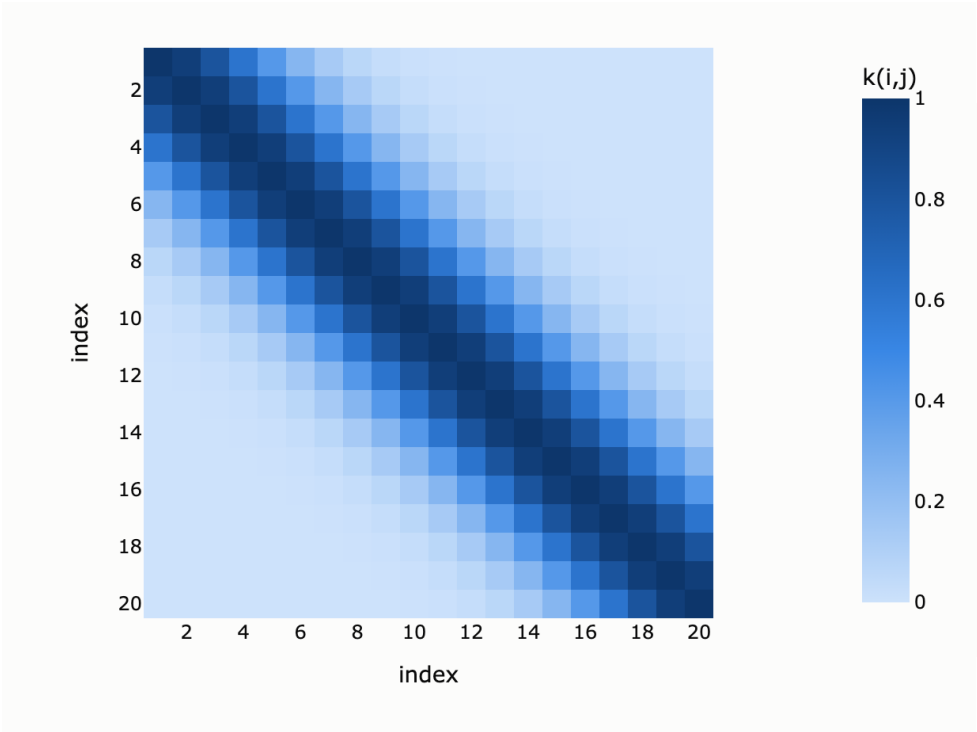


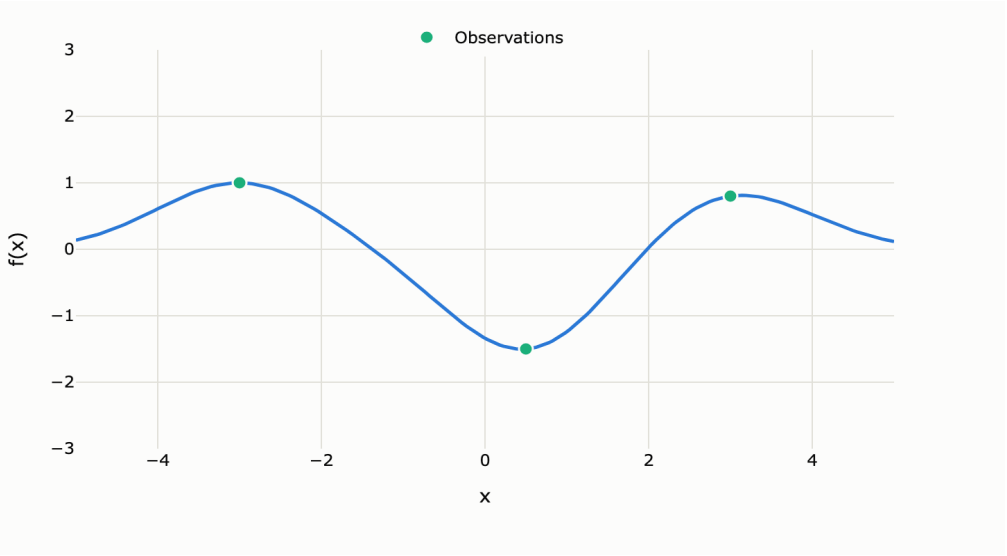
Multivariate Gaussian



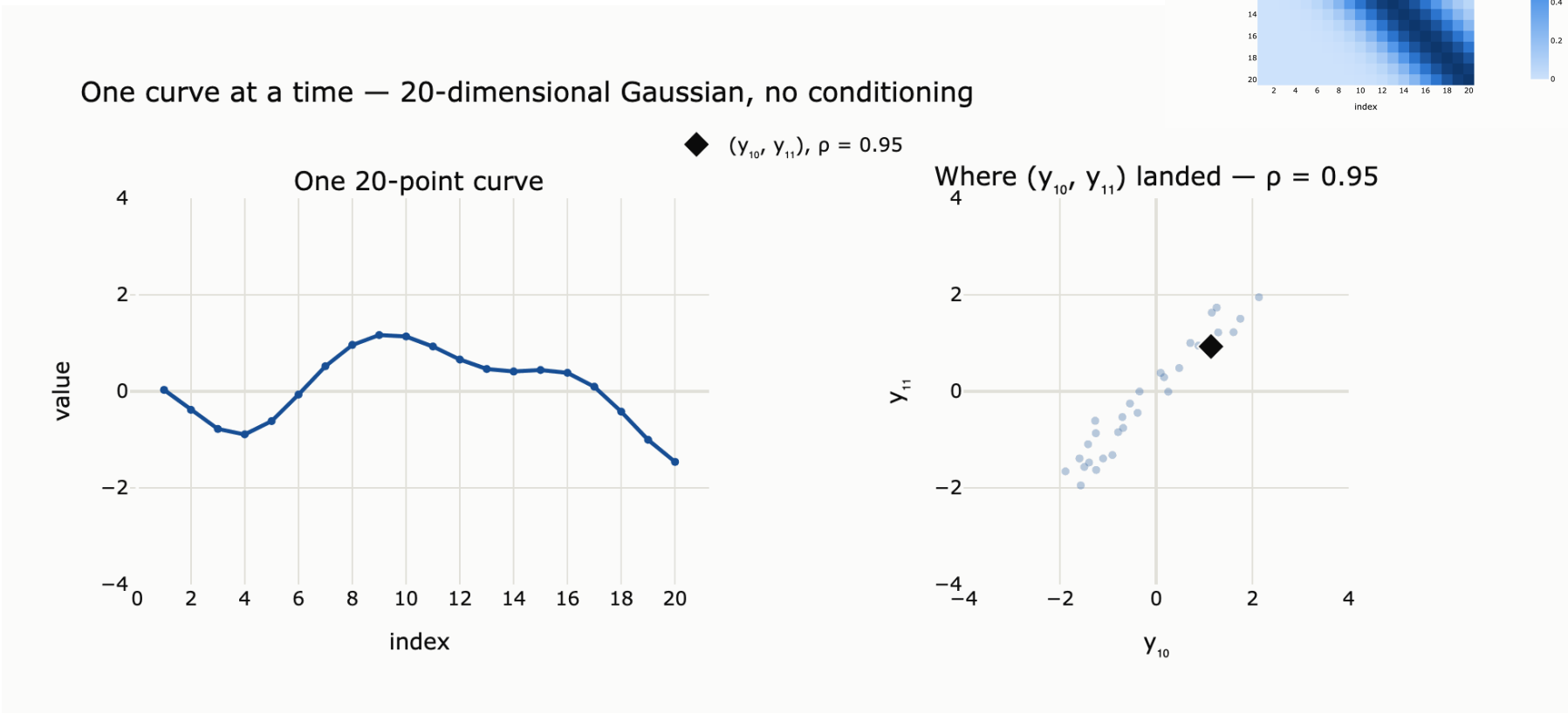


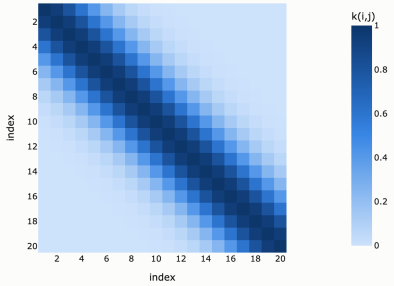
Higher Dimensional Gaussian



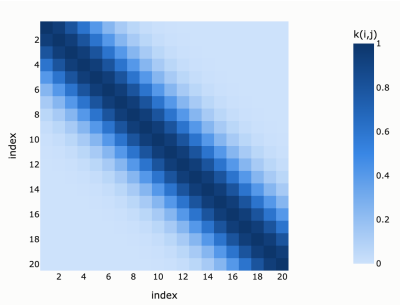


Higher Dimensional Gaussian

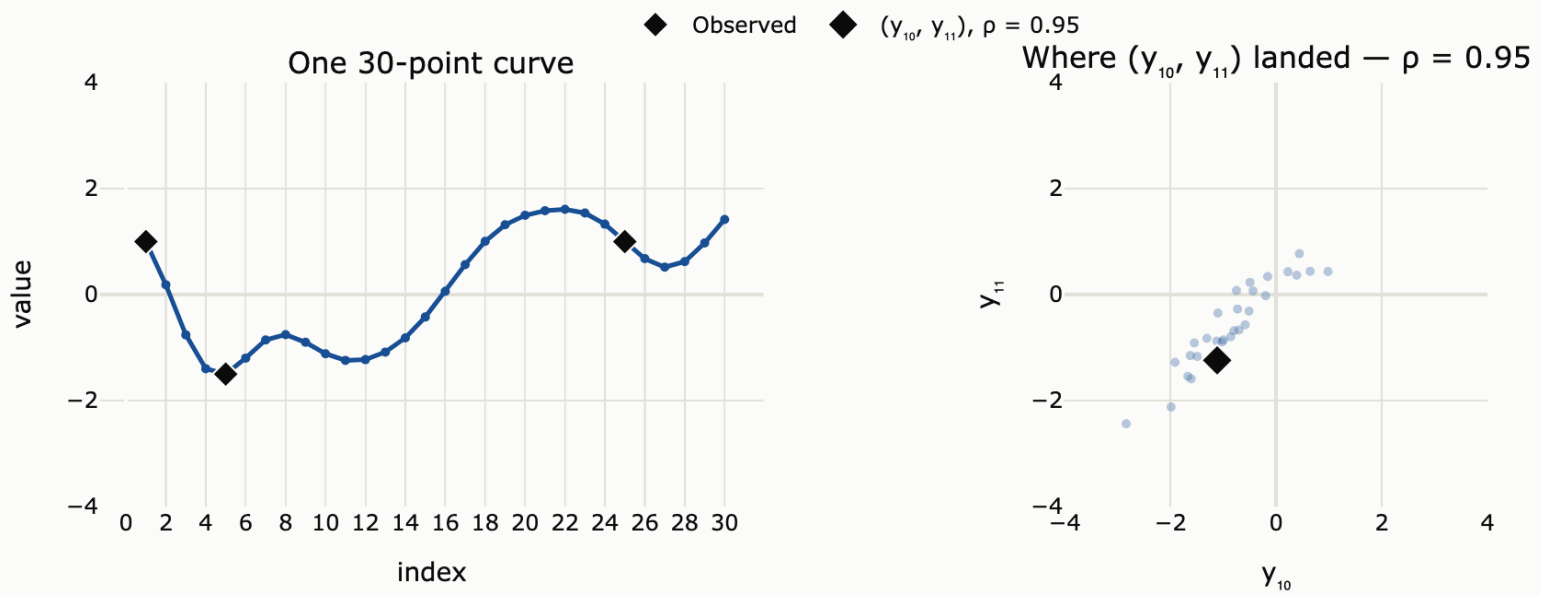




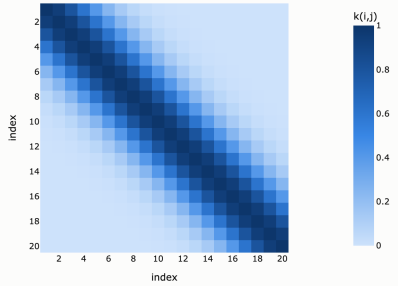
Higher Dimensional Gaussian



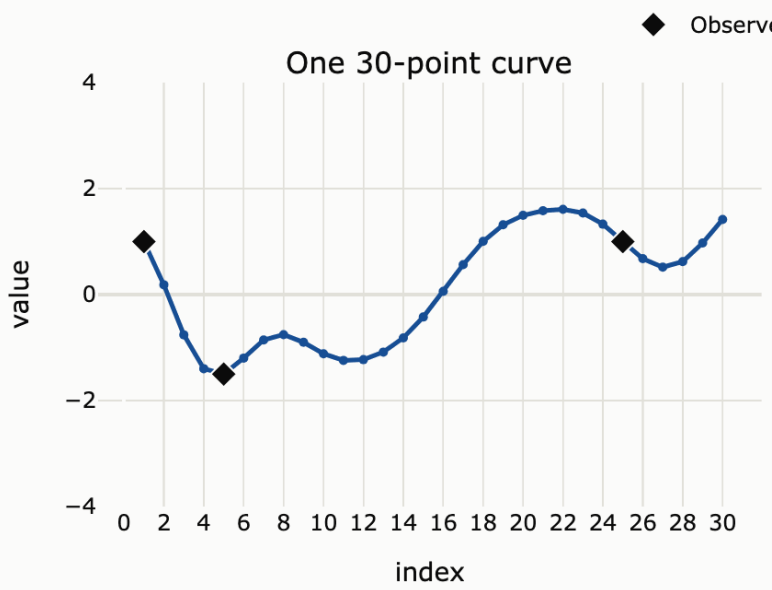
One curve at a time — conditioned on $y_1 = 1$, $y_5 = -1.5$, $y_{25} = 1$

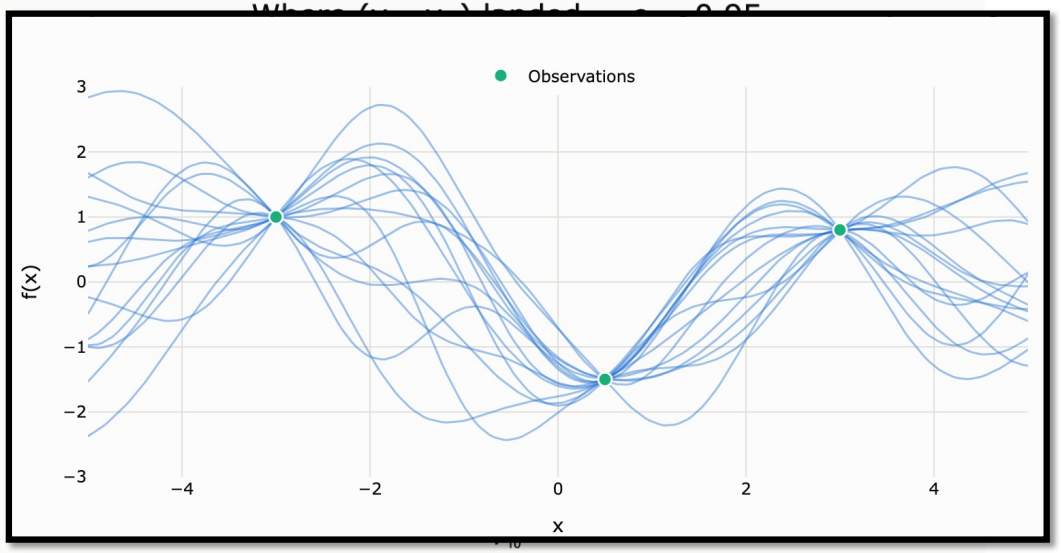


Higher Dimensional Gaussian

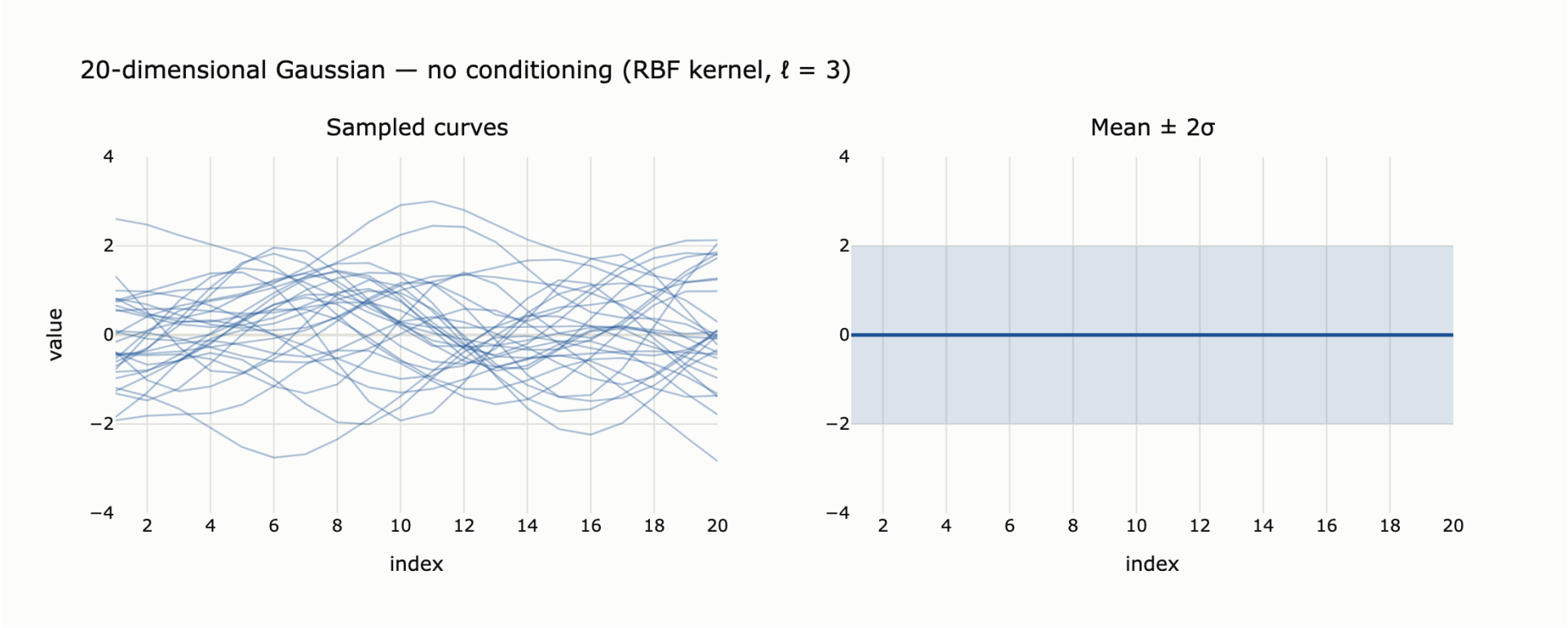


One curve at a time — conditioned on $y_1 = 1$, $y_5 = -1.5$, $y_{25} = 1$

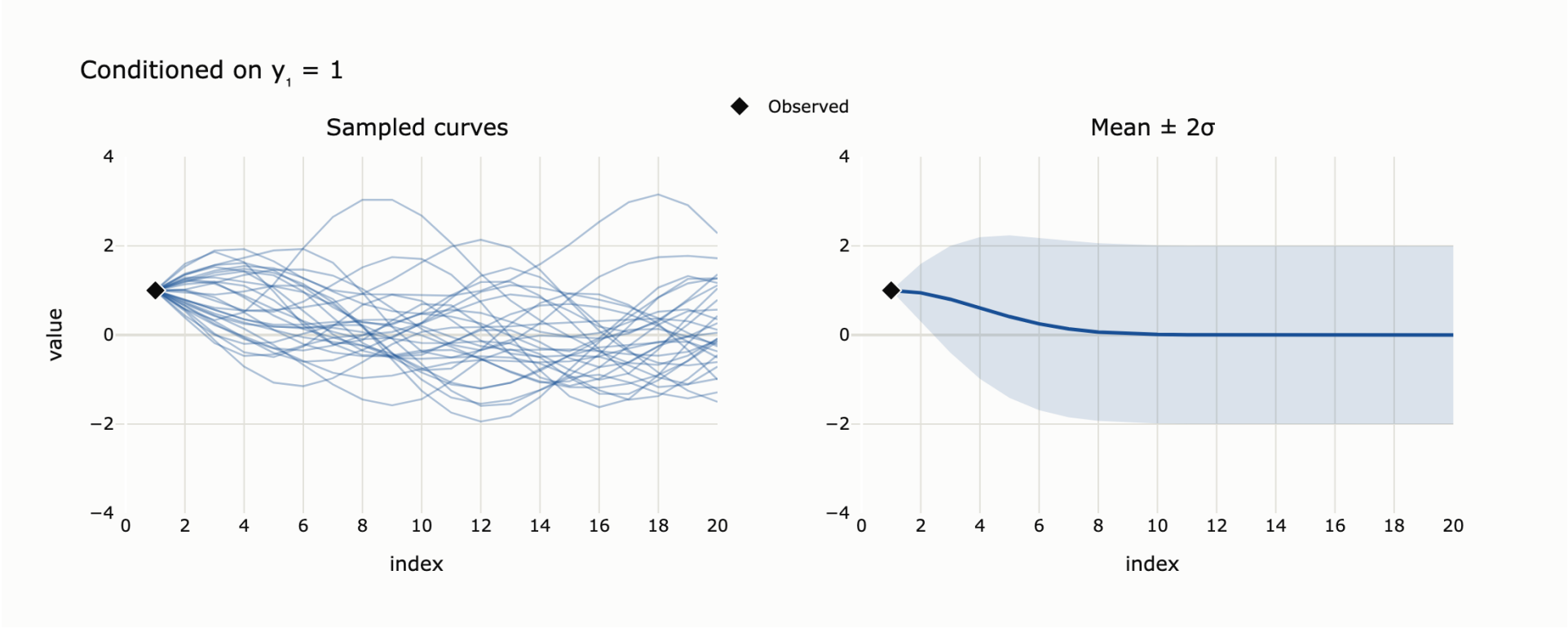




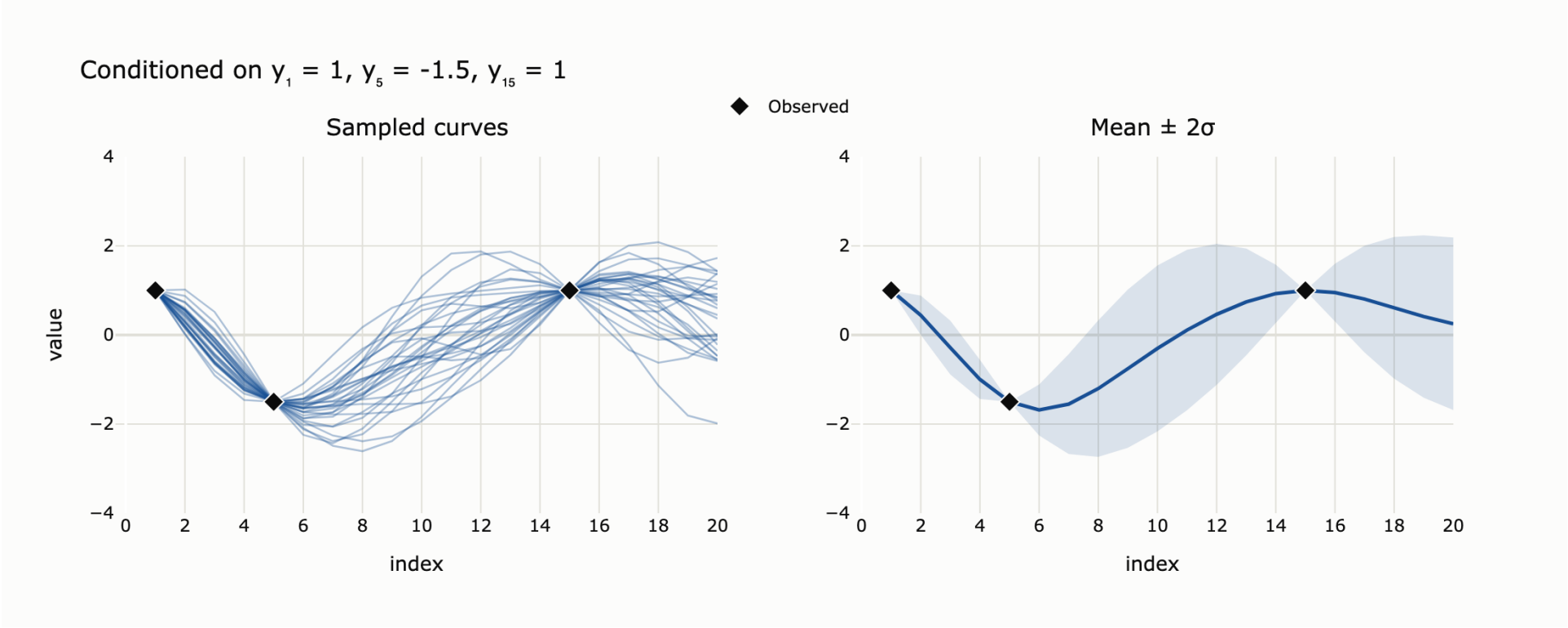
From Discreet to Continuous



From Discreet to Continuous

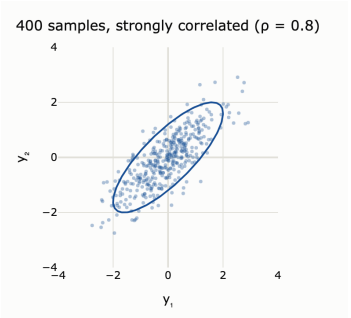


From Discreet to Continuous



From Discreet to Continuous

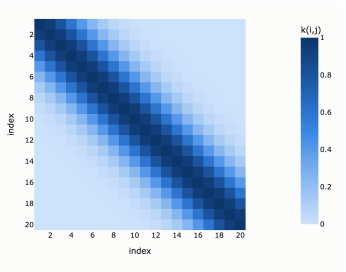
$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}(\det[\mathbf{\Sigma}])^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^T\mathbf{\Sigma}^{-1}(\mathbf{x}-\mu)\right]$$



$$\boldsymbol{\mu} = \begin{bmatrix}\mu_x \\ \mu_y\end{bmatrix}$$

We call them Dimensionless Models

$$\Sigma = \begin{bmatrix}\sigma_x^2 & \sigma_{xy}^2 \\ \sigma_{xy}^2 & \sigma_y^2\end{bmatrix}$$



Make it infinite Dimension

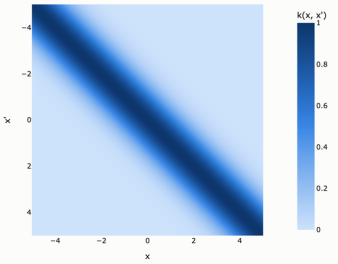
Mean Function

$$\mathrm{m}(\mathrm{x}) = 0$$

if data is standardized.

Covariance or Kernel function

$$k(x,x') = \sigma^2 \exp\left(-\frac{(x-x')^2}{2\ell^2}\right)$$



Gaussian Process

$$\mathrm{f}(\boldsymbol{x}) \sim GP(\mathrm{m}(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}'))$$

Mean Function

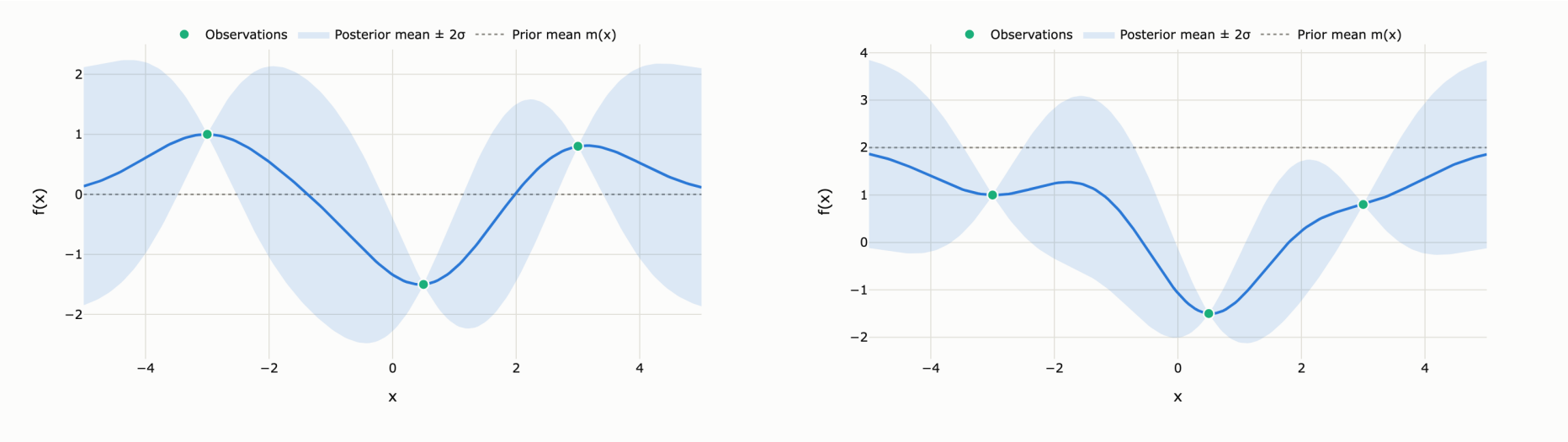
$$\mathrm{m}(\mathrm{x}) = 0$$

if data is standardized.

Covariance or Kernel function

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

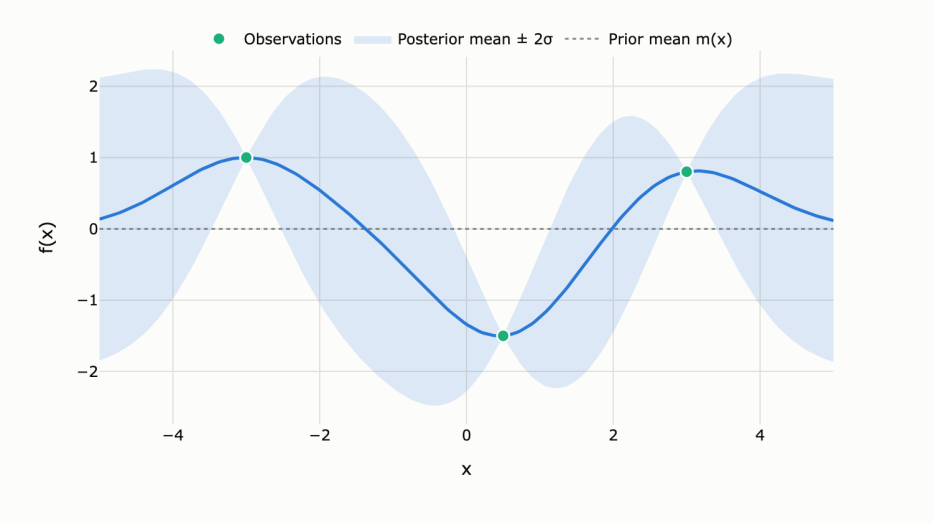
Mean Function



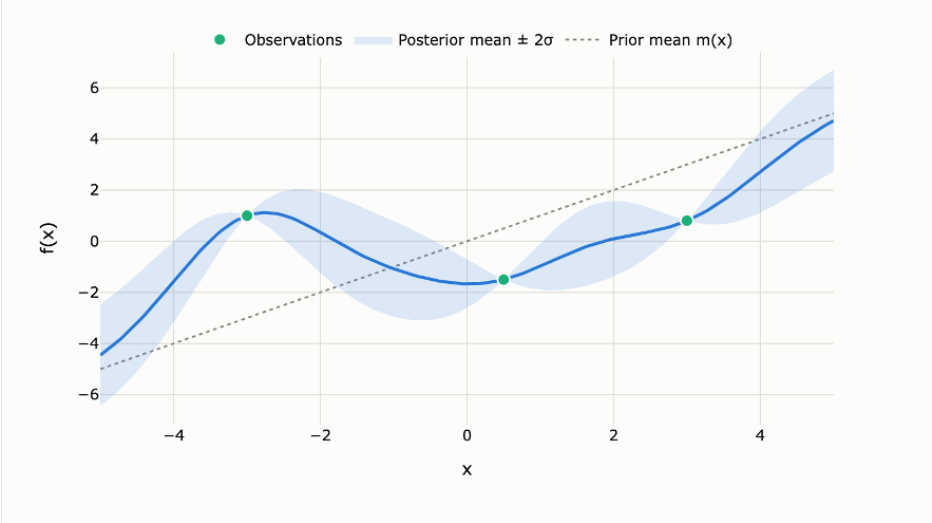
$$m(x) = 0\,, \quad \mu = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$m(x) = 2\,, \quad \mu = \begin{bmatrix} 2 \\ \vdots \\ 2 \end{bmatrix}$$

Mean Function

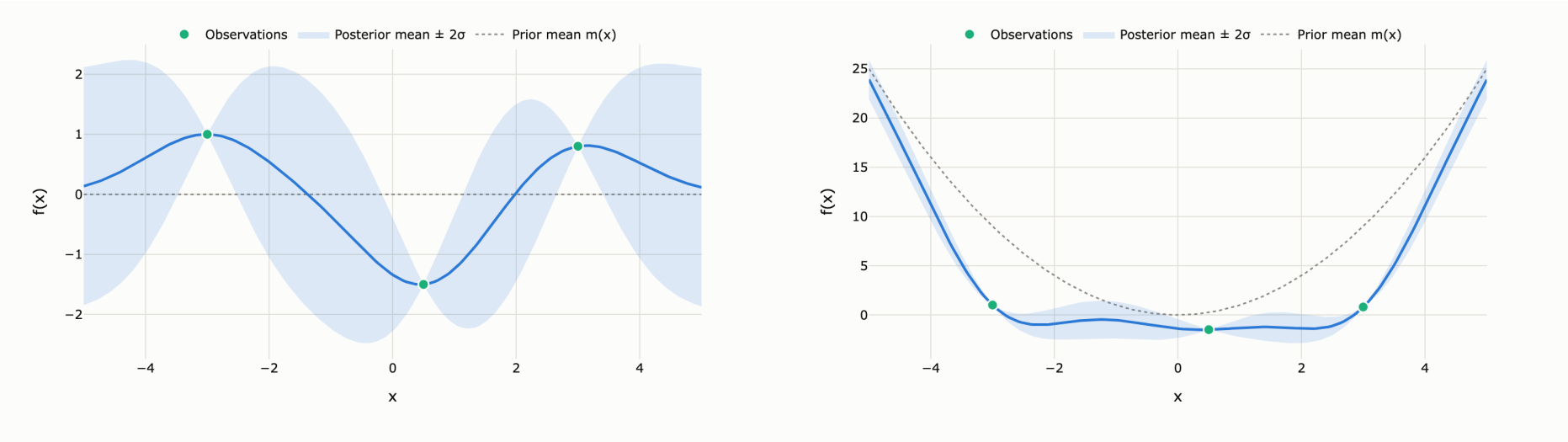


$m(x) = 0$



$m(x) = \mathrm{x}$

Mean Function



$m(x) = 0$

unless you have a good reason, set $m(x) = 0$

$m(x) = \mathrm{x}^2$

Gaussian Process

$$\mathrm{f}(\boldsymbol{x}) \sim GP(\mathrm{m}(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}'))$$

Mean Function

$$\mathrm{m}(\mathrm{x}) = 0$$

if data is standardized.

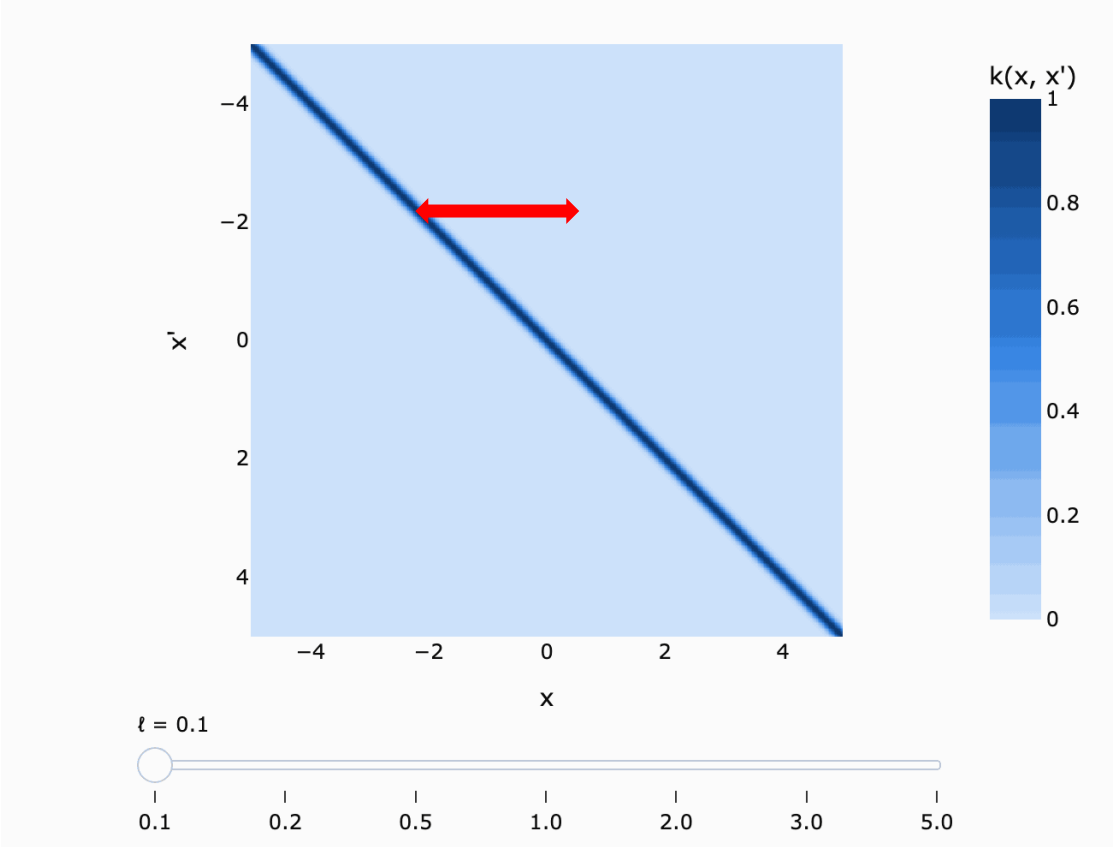
Covariance or Kernel function

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

RBF Kernel Function

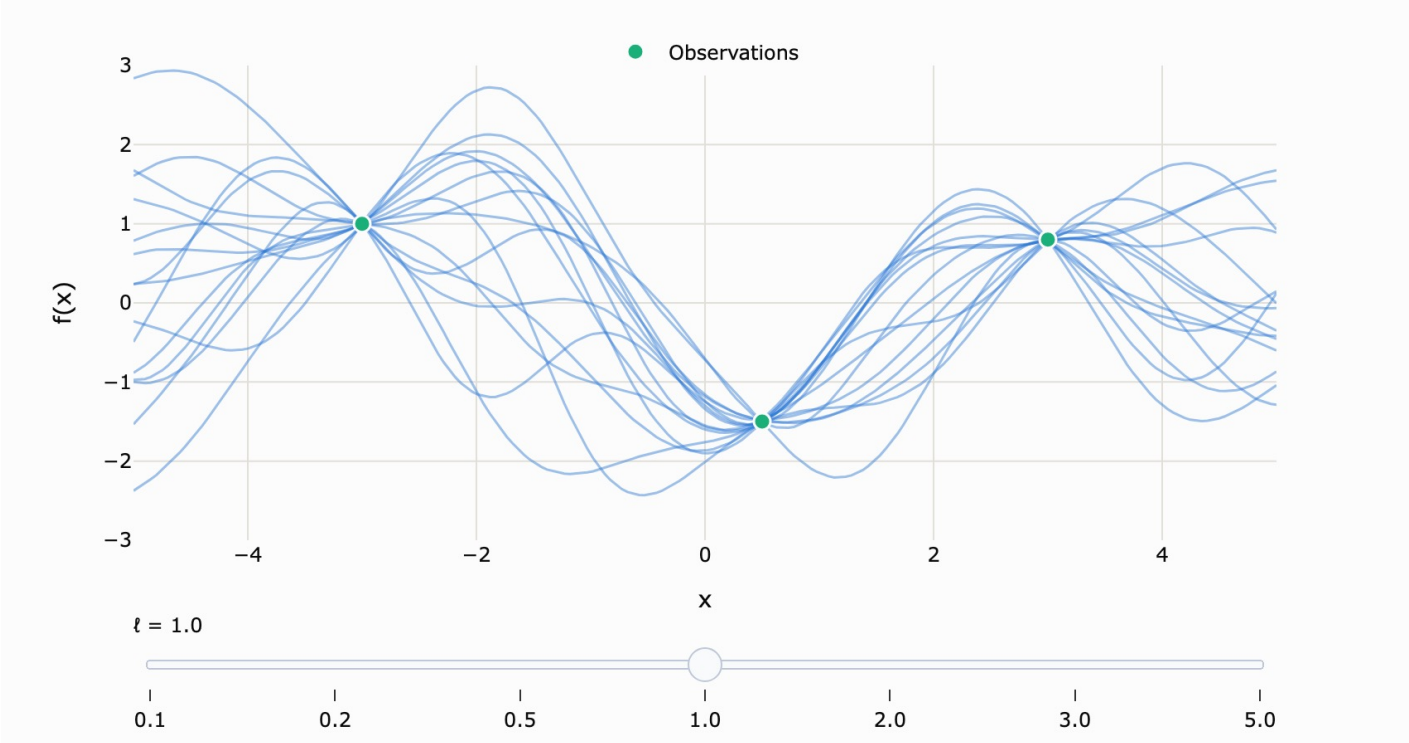
Length Scales

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



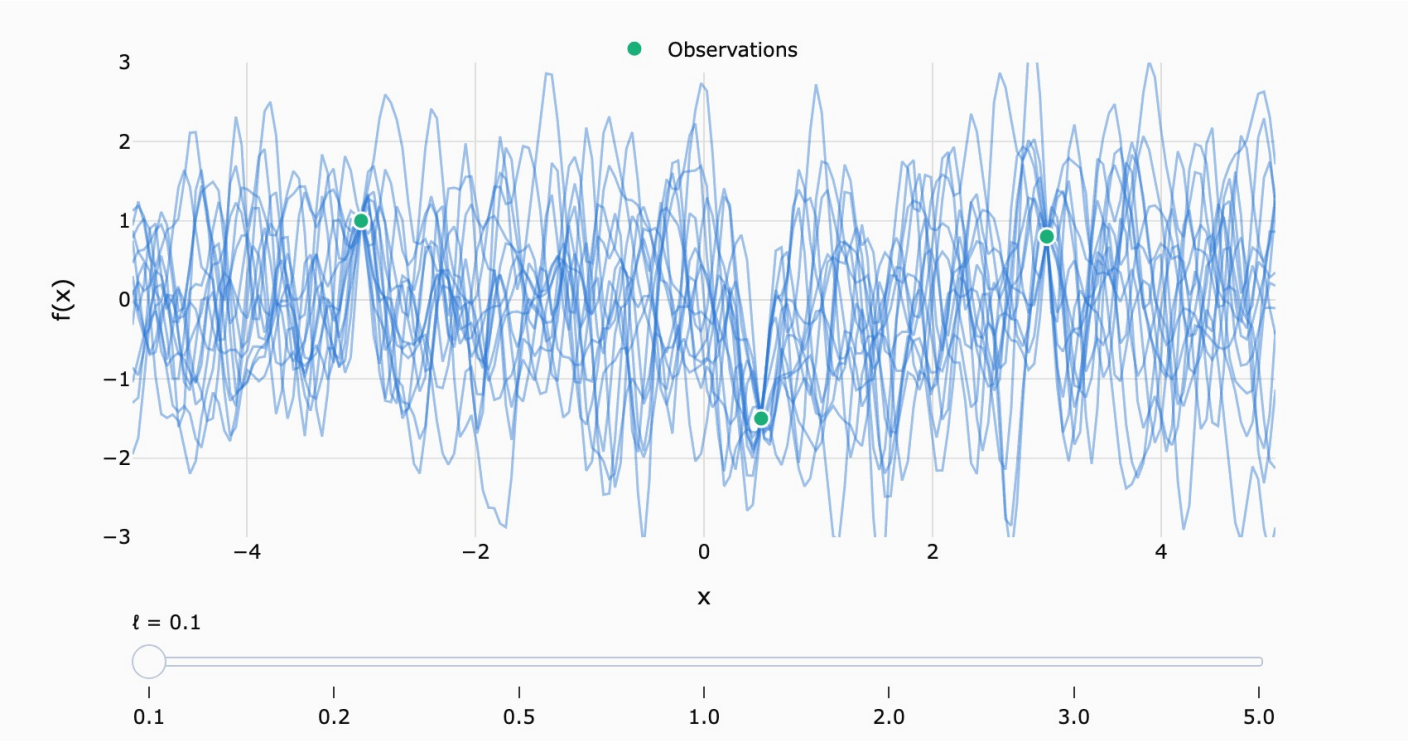
RBF Kernel Function

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



RBF Kernel Function

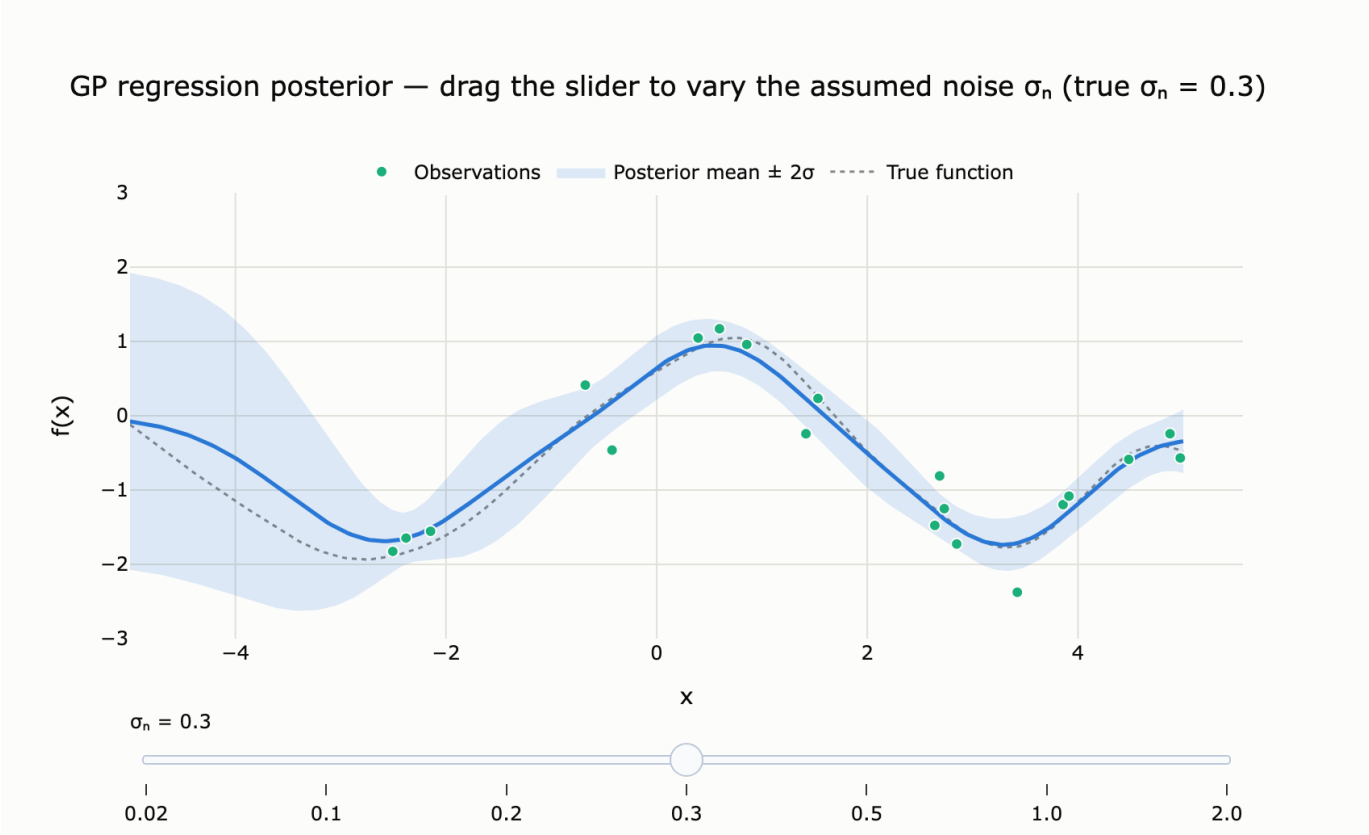
$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



RBF Kernel Function

Noise Parameter Sigma

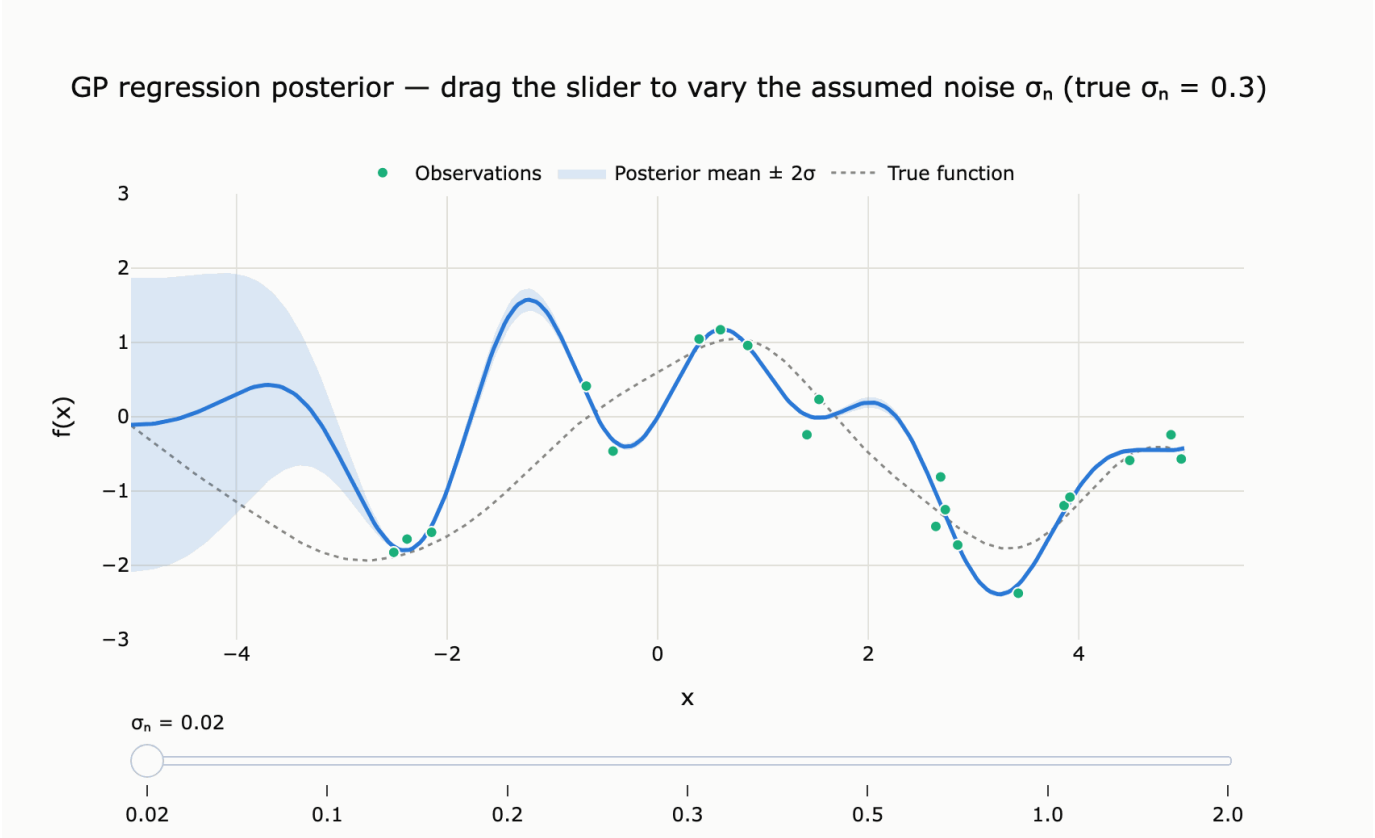
$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



RBF Kernel Function

Noise Parameter Sigma

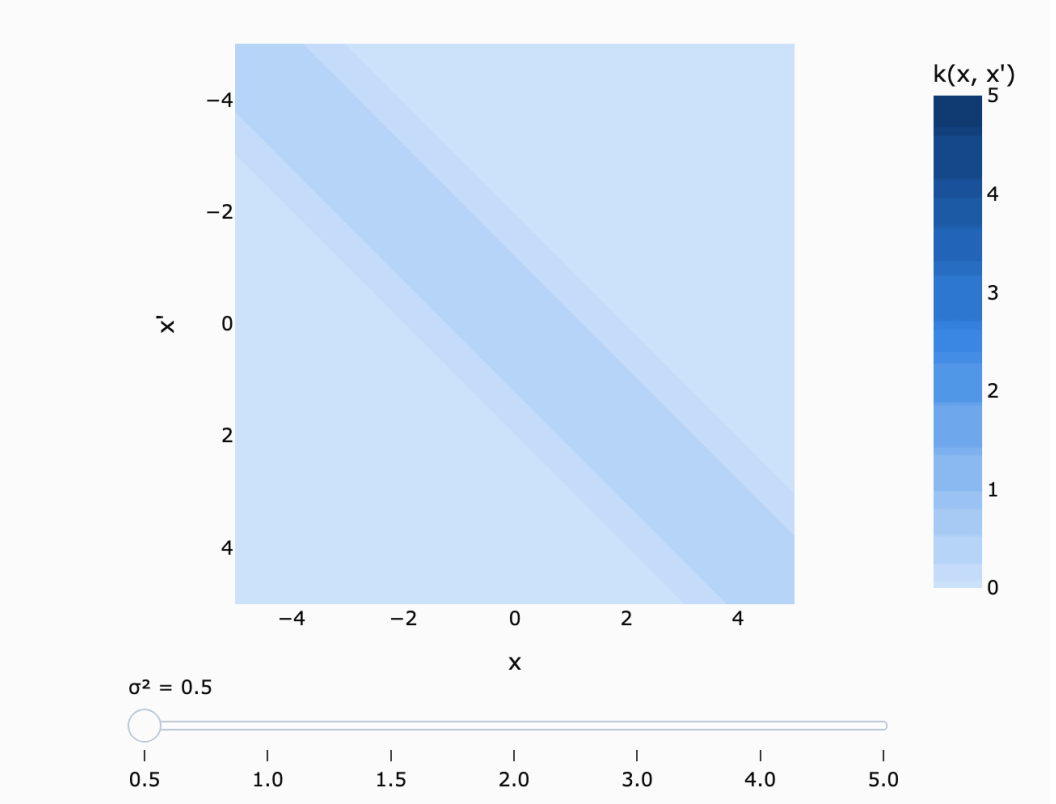
$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



RBF Kernel Function

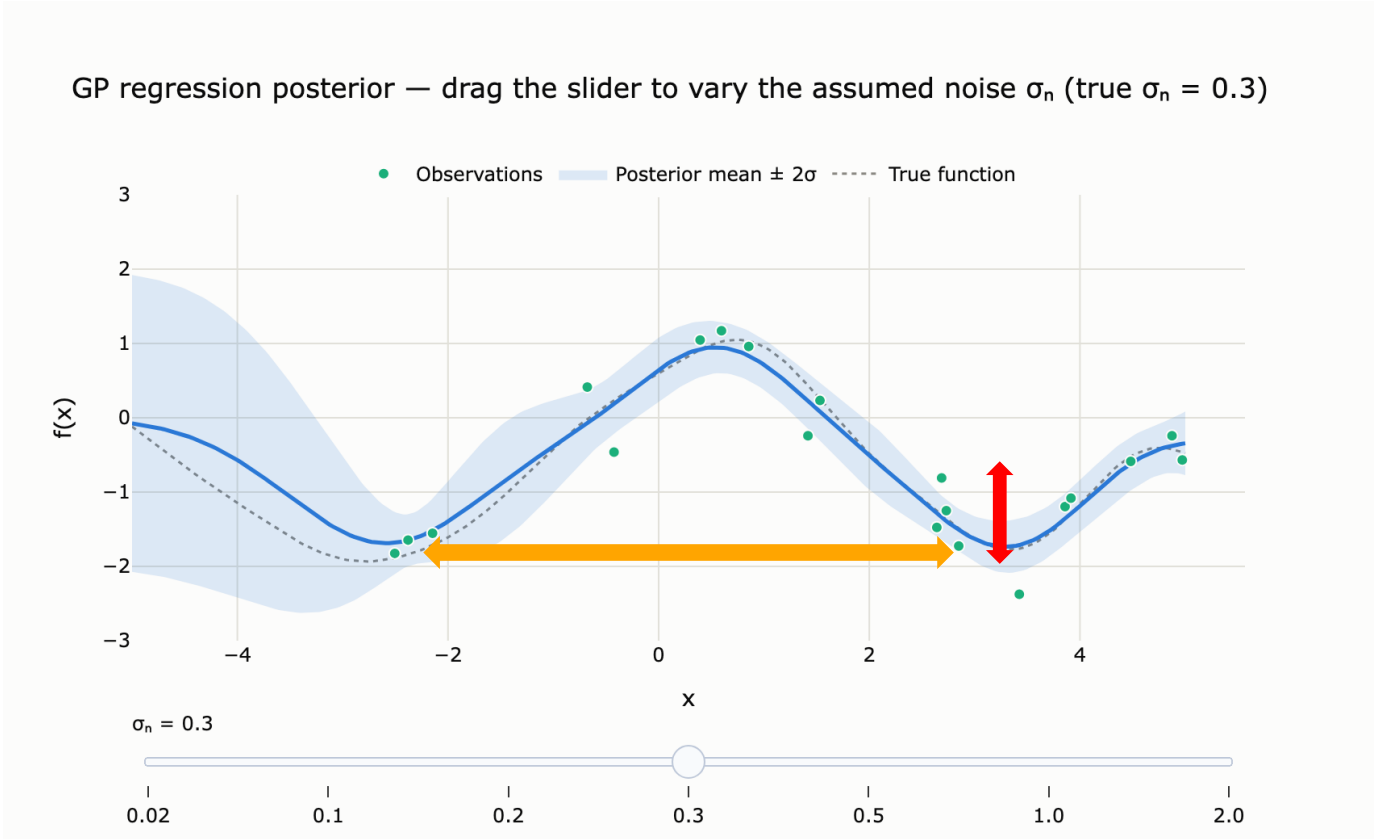
Noise Parameter Sigma

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

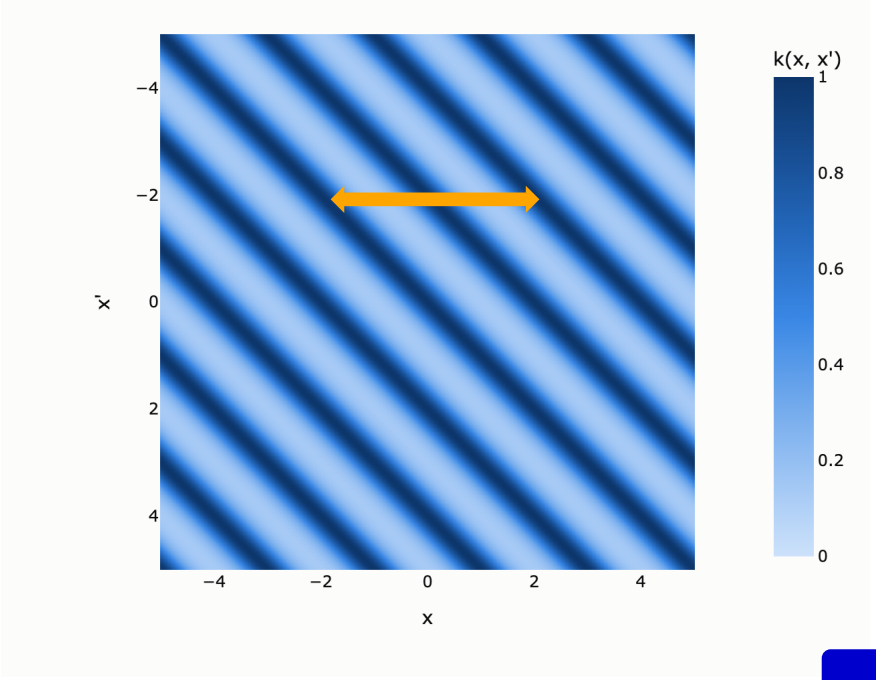


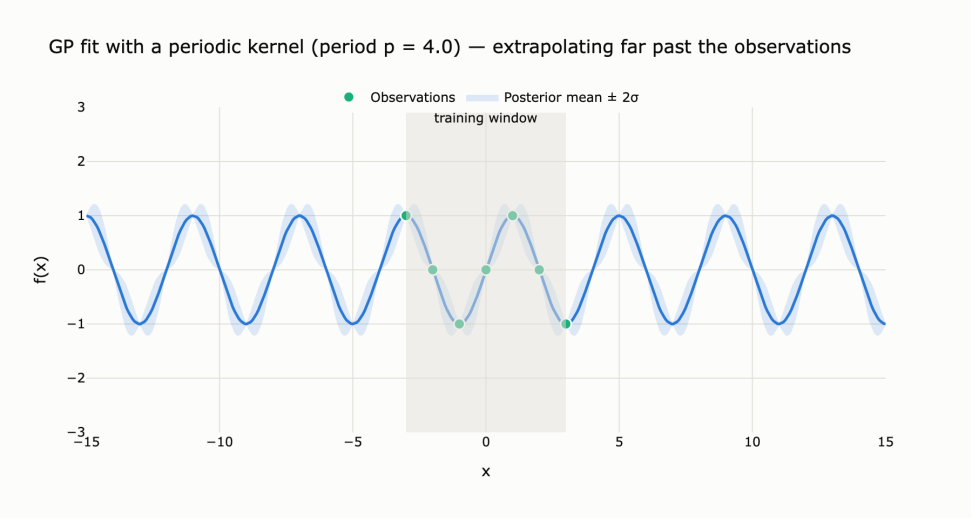
Hyperparameter Summery

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$



Other Kernels
Periodic Kernel





Period

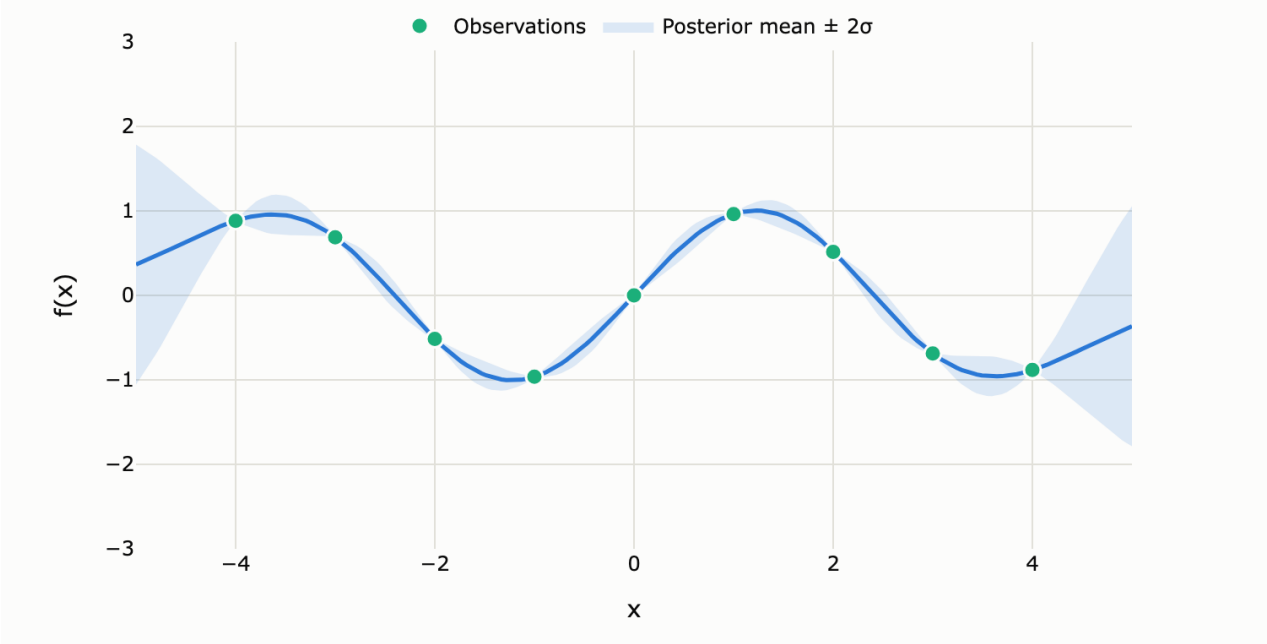
$$k(x, x') = \sigma^2 \exp\left(-\frac{2\sin^2(\pi|x - x'|/p)}{\ell^2}\right)$$

Recap

Mean Function

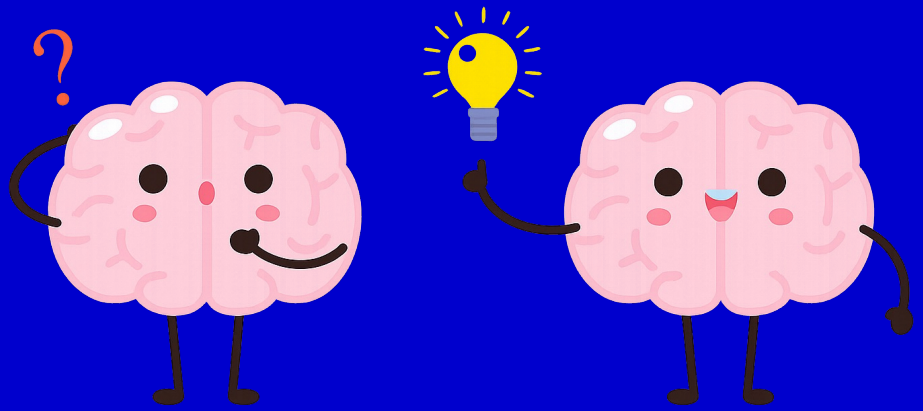
Covariance or Kernel function

$$\mathrm{f}(\boldsymbol{x}) \sim GP(\mathrm{m}(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}'))$$

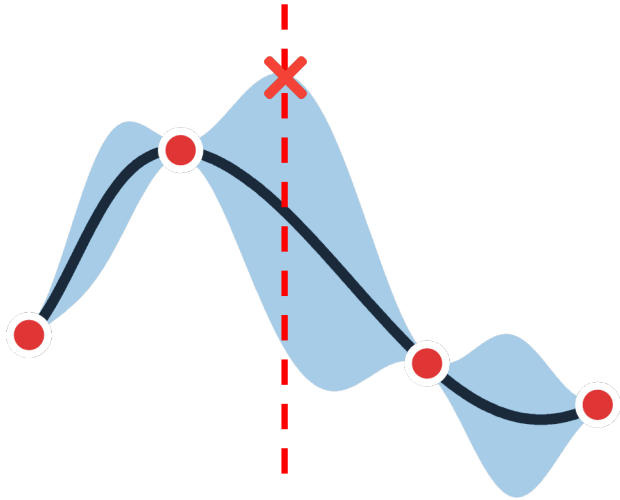


IMPERIAL

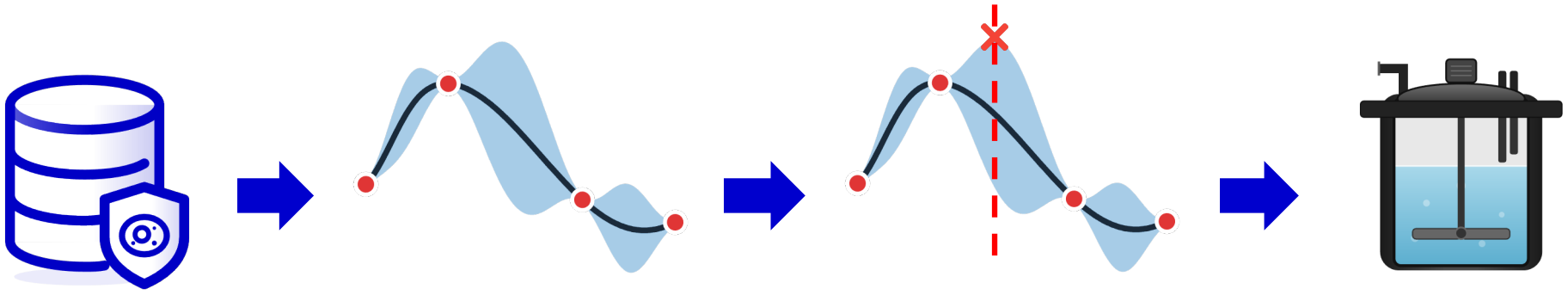
Questions



Bayesian Optimization



Bayesian Optimization



Data

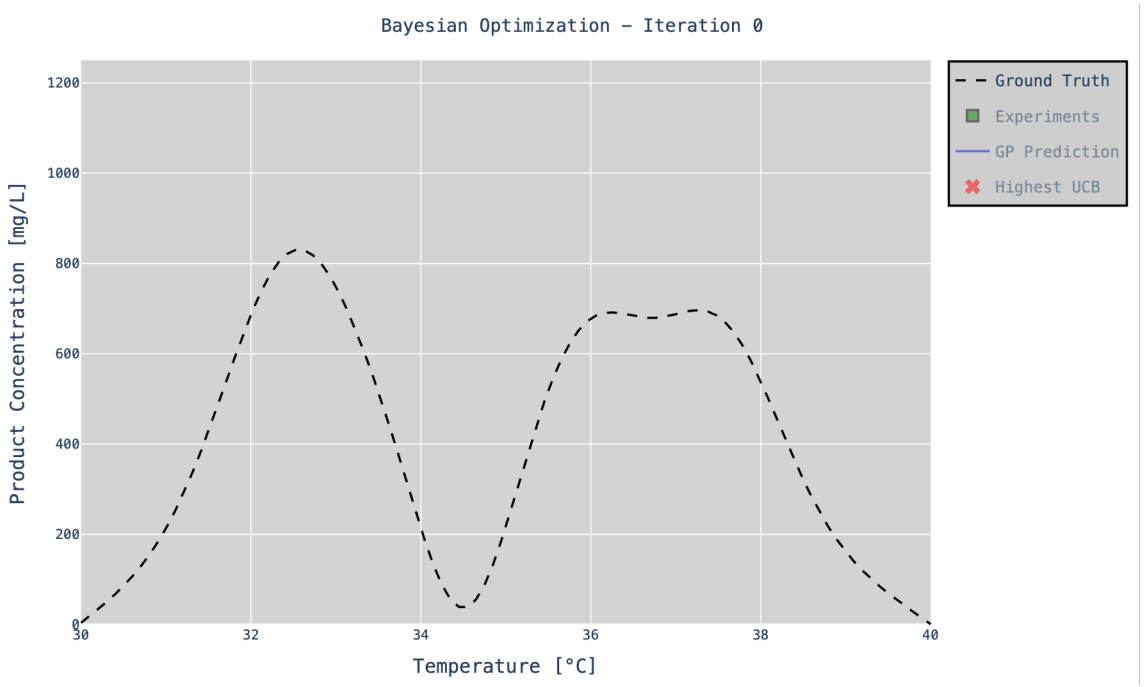
Train GP

Maximize
Acquisition Function

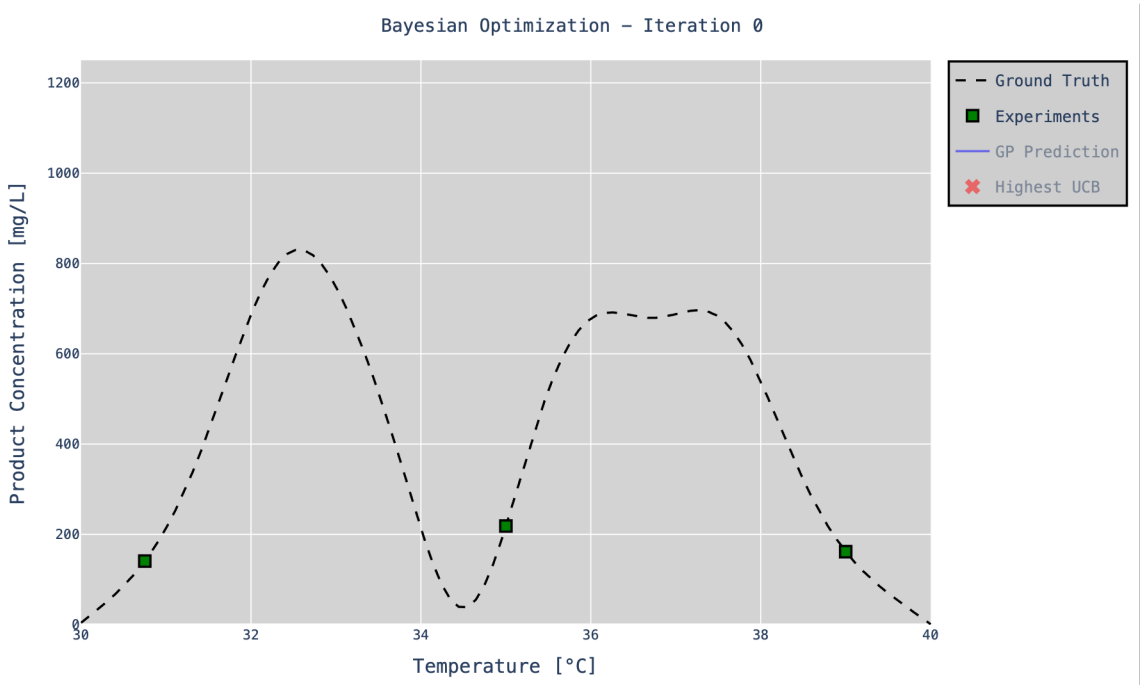
Run Experiments

Active Learning

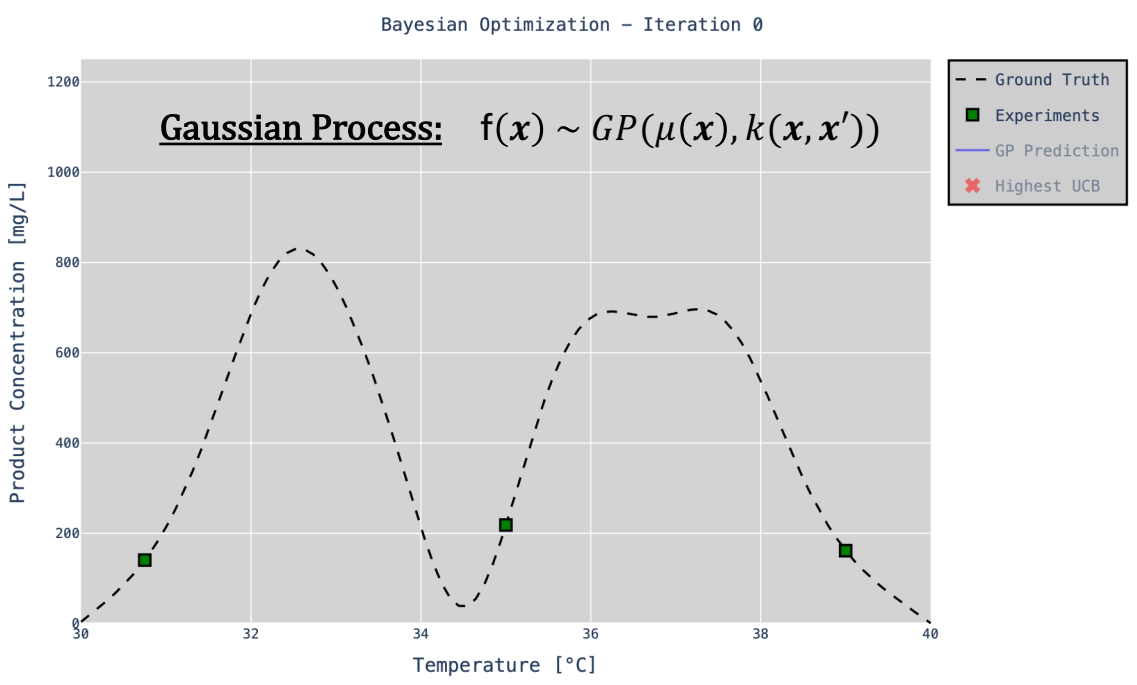
Bayesian Optimization



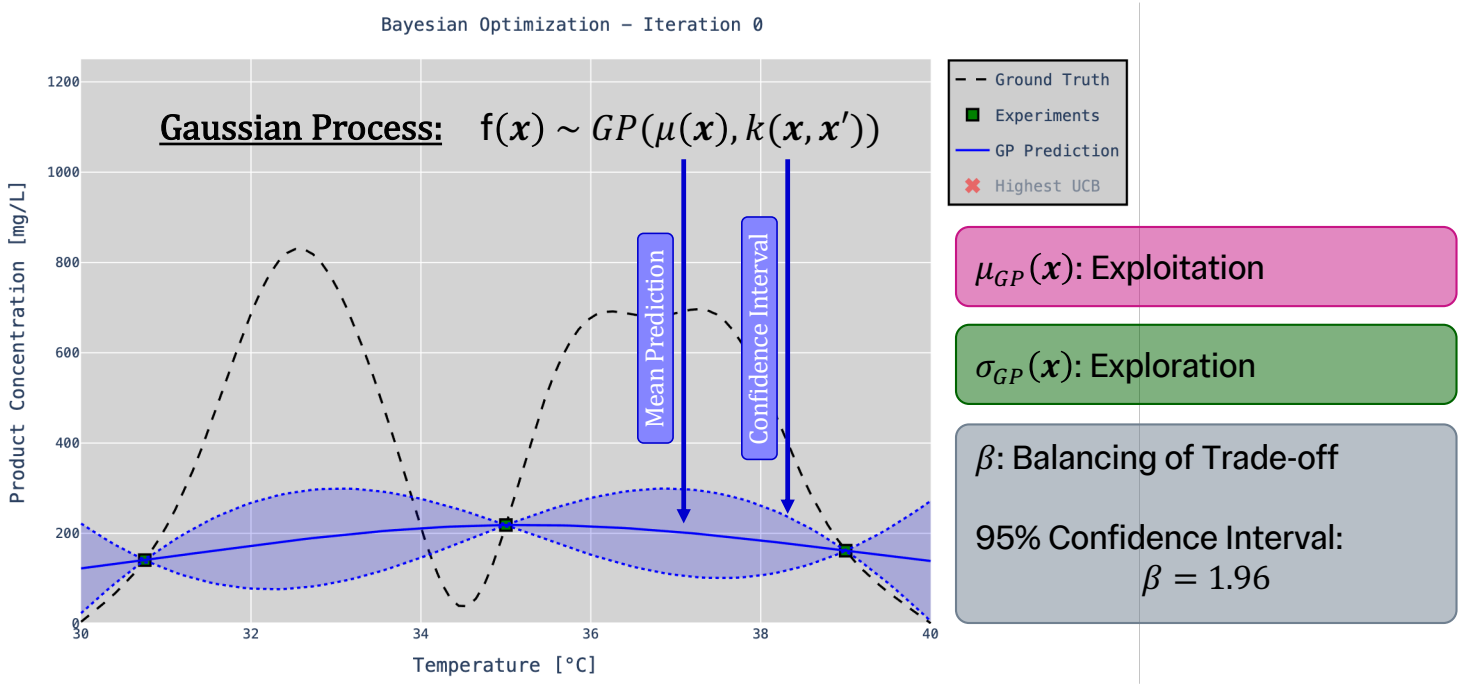
Bayesian Optimization



Bayesian Optimization



Bayesian Optimization



$\mu_{GP}(x)$: Exploitation

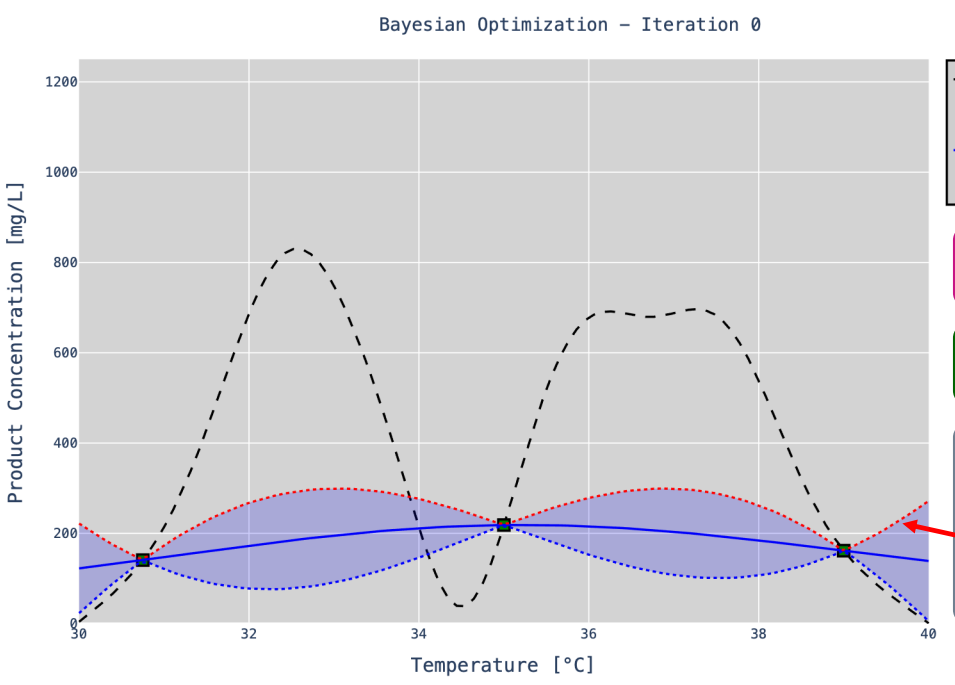
$\sigma_{GP}(x)$: Exploration

β: Balancing of Trade-off

95% Confidence Interval: $\beta = 1.96$

Acquisition Function: $UCB(x) = \mu_{GP}(x) + \beta * \sigma_{GP}(x)$

Bayesian Optimization



$\mu_{GP}(\boldsymbol{x})$: Exploitation

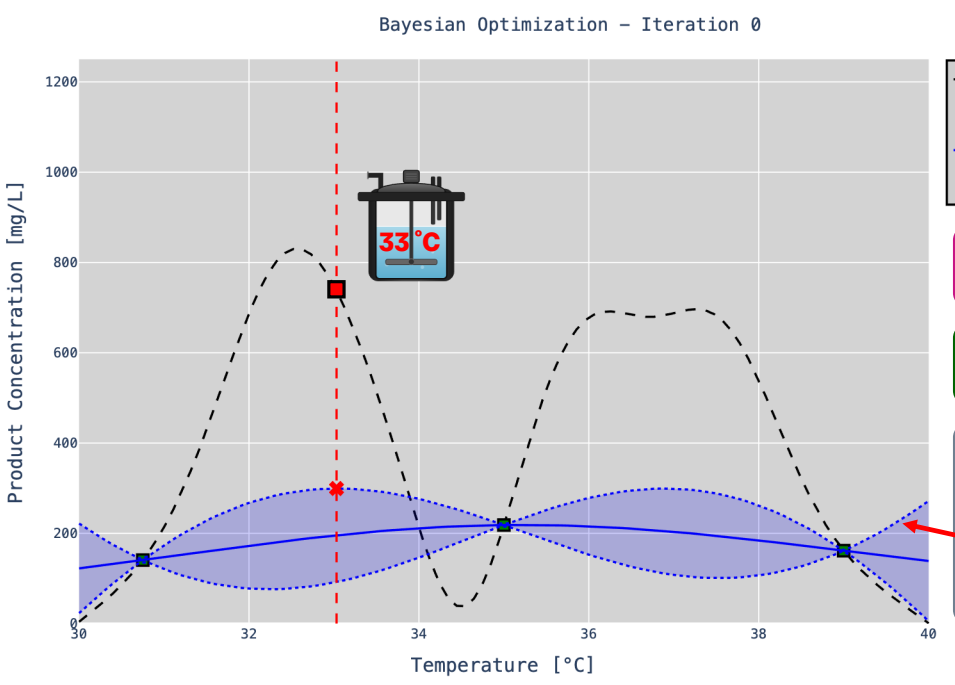
$\sigma_{GP}(\boldsymbol{x})$: Exploration

β: Balancing of Trade-off

95% Confidence Interval: $\beta = 1.96$

Acquisition Function: $UCB(\boldsymbol{x}) = \mu_{GP}(\boldsymbol{x}) + \beta * \sigma_{GP}(\boldsymbol{x})$

Bayesian Optimization



$\mu_{GP}(\boldsymbol{x})$: Exploitation

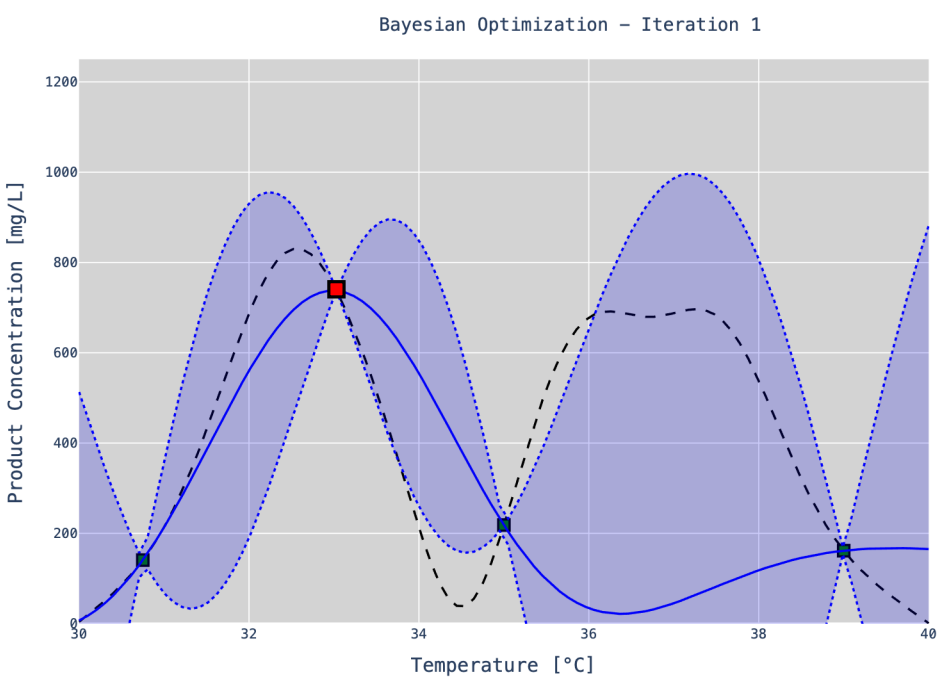
$\sigma_{GP}(\boldsymbol{x})$: Exploration

β: Balancing of Trade-off

95% Confidence Interval: $\beta = 1.96$

Acquisition Function: $UCB(\boldsymbol{x}) = \mu_{GP}(\boldsymbol{x}) + \beta * \sigma_{GP}(\boldsymbol{x})$

Bayesian Optimization



$\mu_{GP}(\boldsymbol{x})$: Exploitation

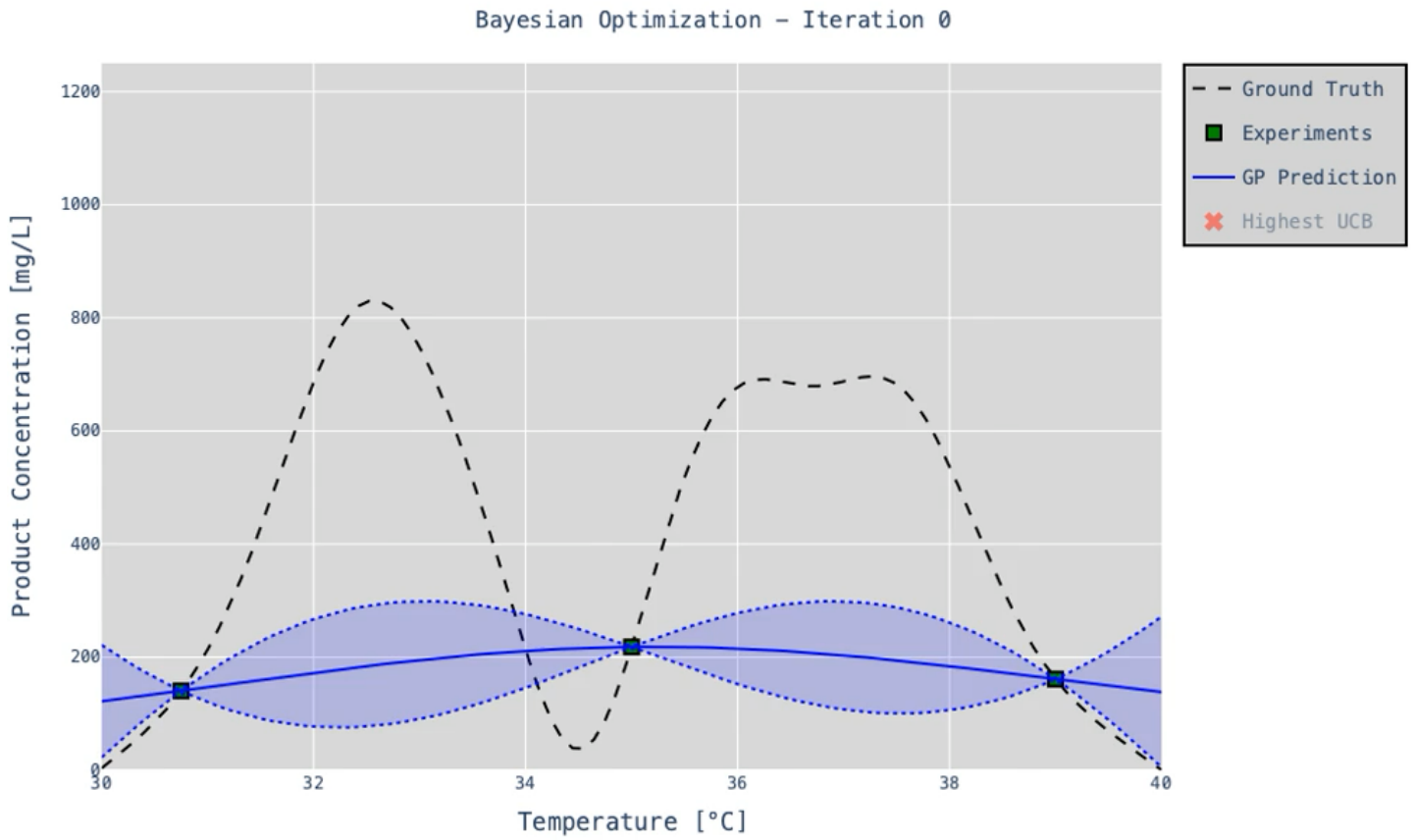
$\sigma_{GP}(\boldsymbol{x})$: Exploration

β: Balancing of Trade-off

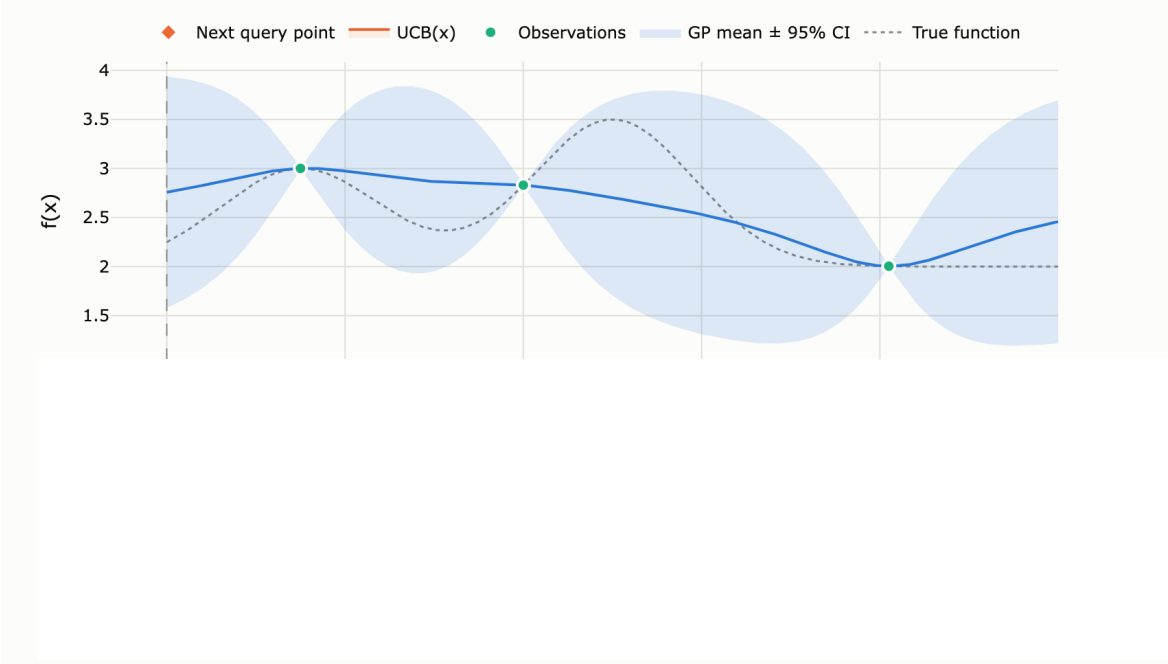
95% Confidence Interval:
$\beta = 1.96$

Acquisition Function: $UCB(\boldsymbol{x}) = \mu_{GP}(\boldsymbol{x}) + \beta * \sigma_{GP}(\boldsymbol{x})$

Bayesian Optimization

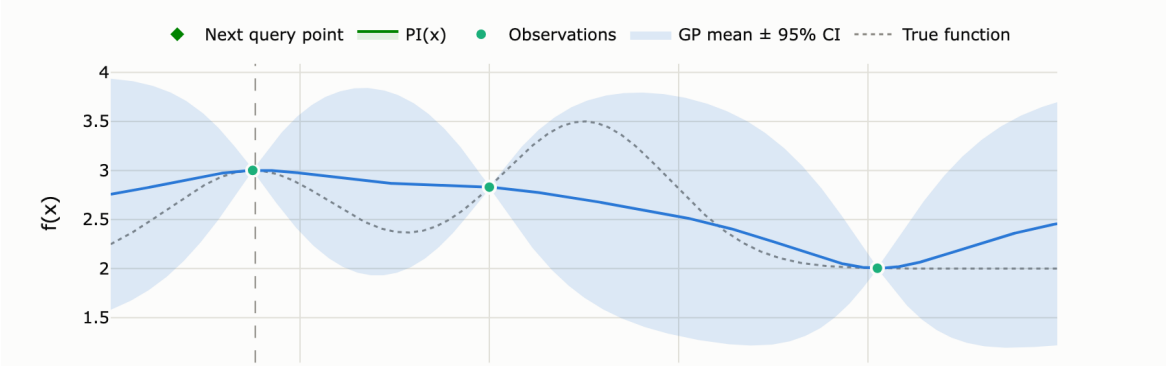


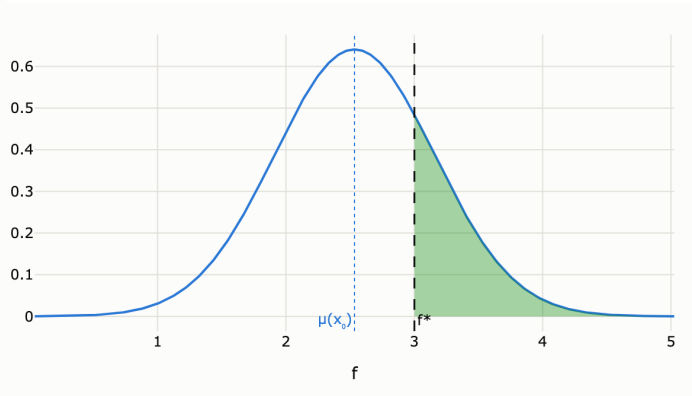
Acquisition Function
Upper Confidence Bound



$$\text{UCB} = \mu(\mathbf{x}) + \beta\sigma(\mathbf{x})$$

Acquisition Function
Probability of Improvement

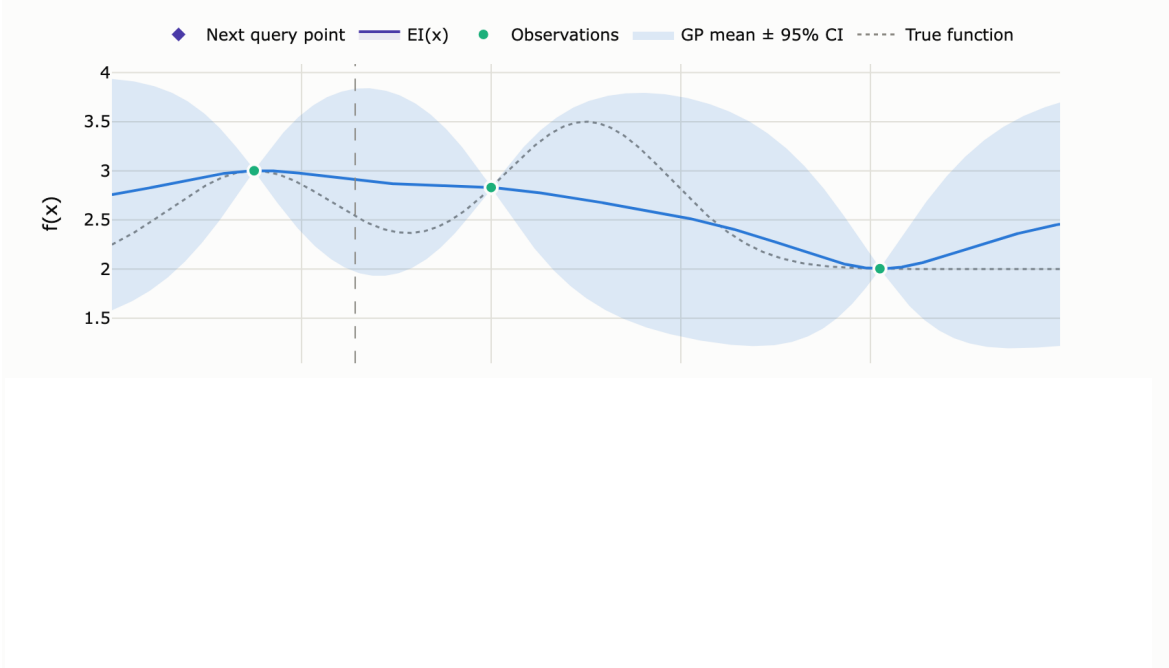


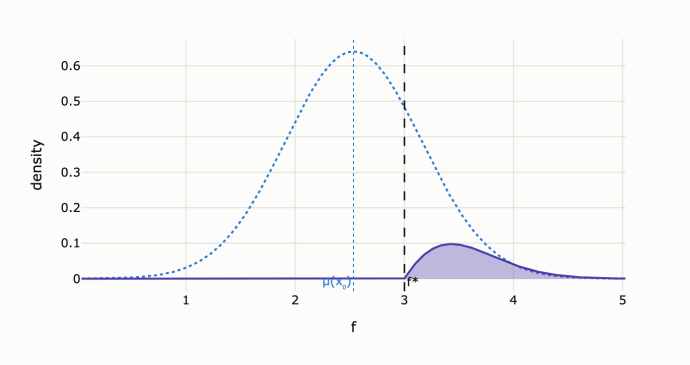


$$\mathrm{PI}(\mathbf{x}) = \Phi(\mathbf{z}_0) = \Phi\left(\frac{f(\mathbf{x}^*) - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right)$$

Acquisition Function

Expected Improvement

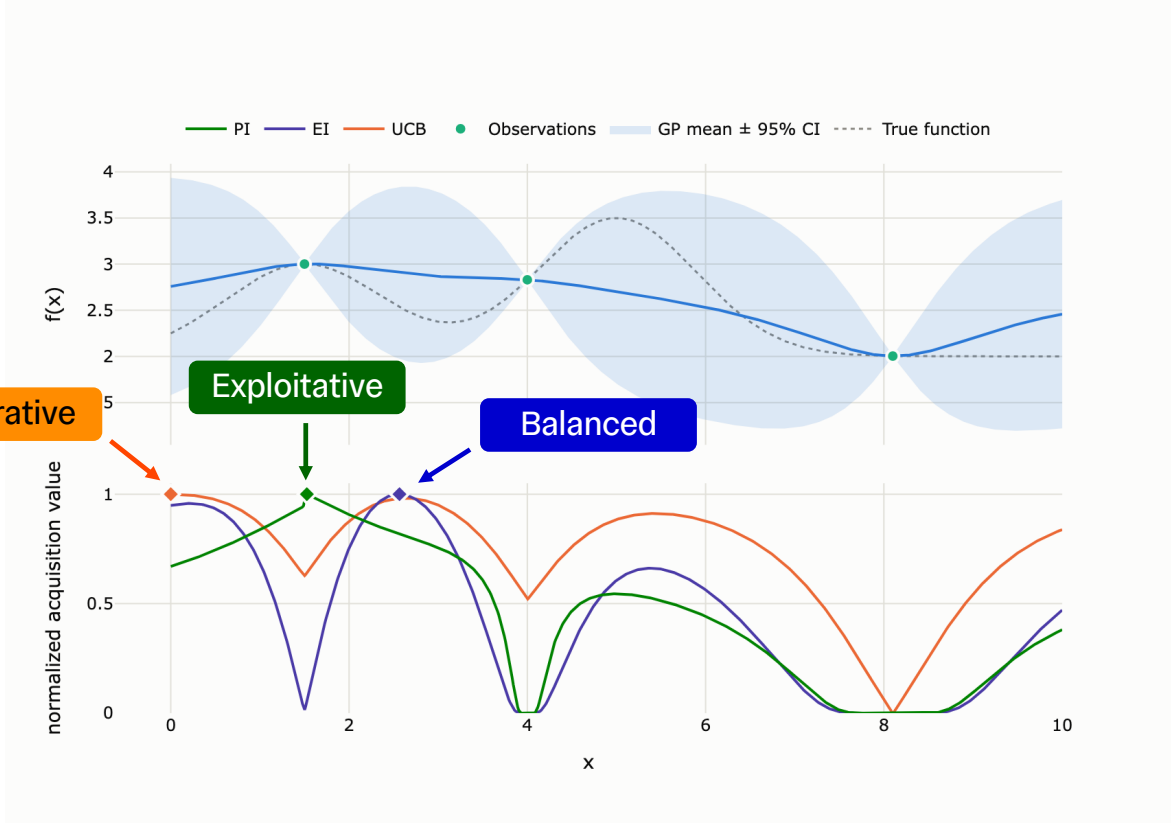




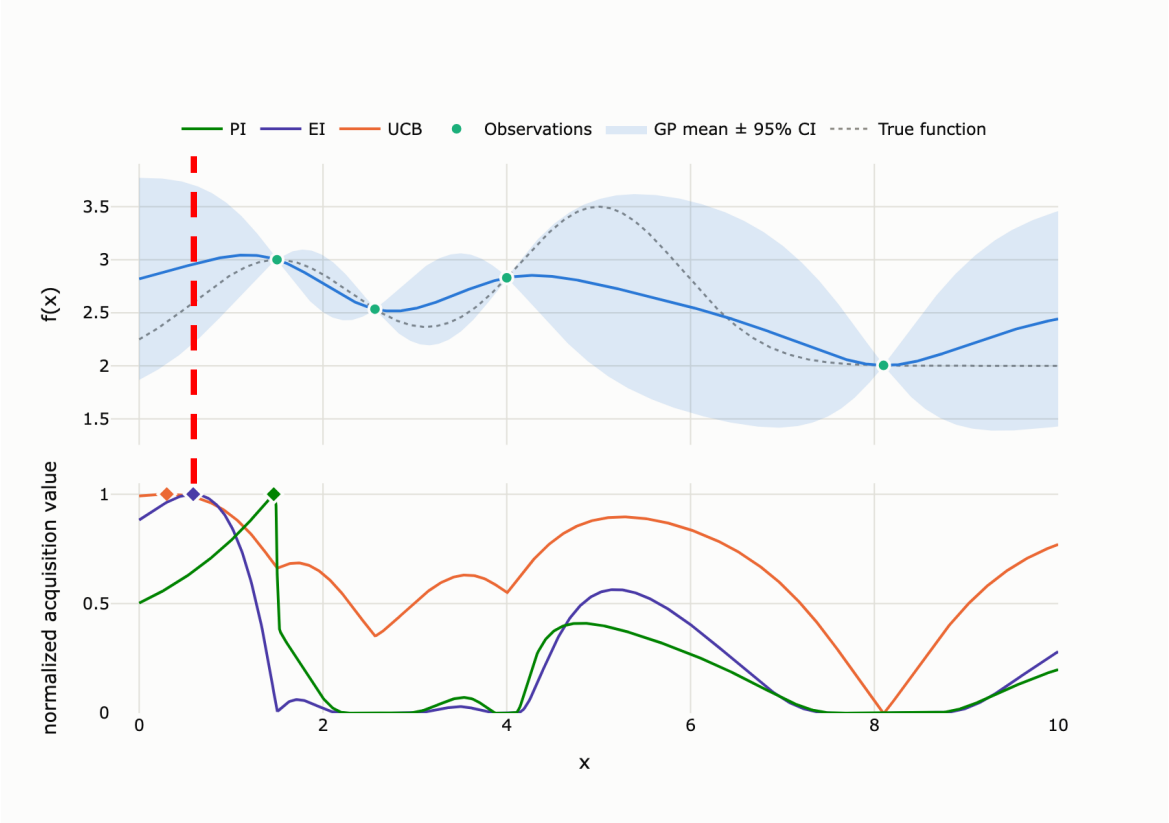
$$\mathrm{EI}(x) = (f(\mathbf{x}^*) - \mu(\mathbf{x}))\Phi\left(\frac{f(\mathbf{x}^*) - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right) + \sigma(\mathbf{x})\phi\left(\frac{f(\mathbf{x}^*) - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right)$$

Acquisition Function

What's the difference?

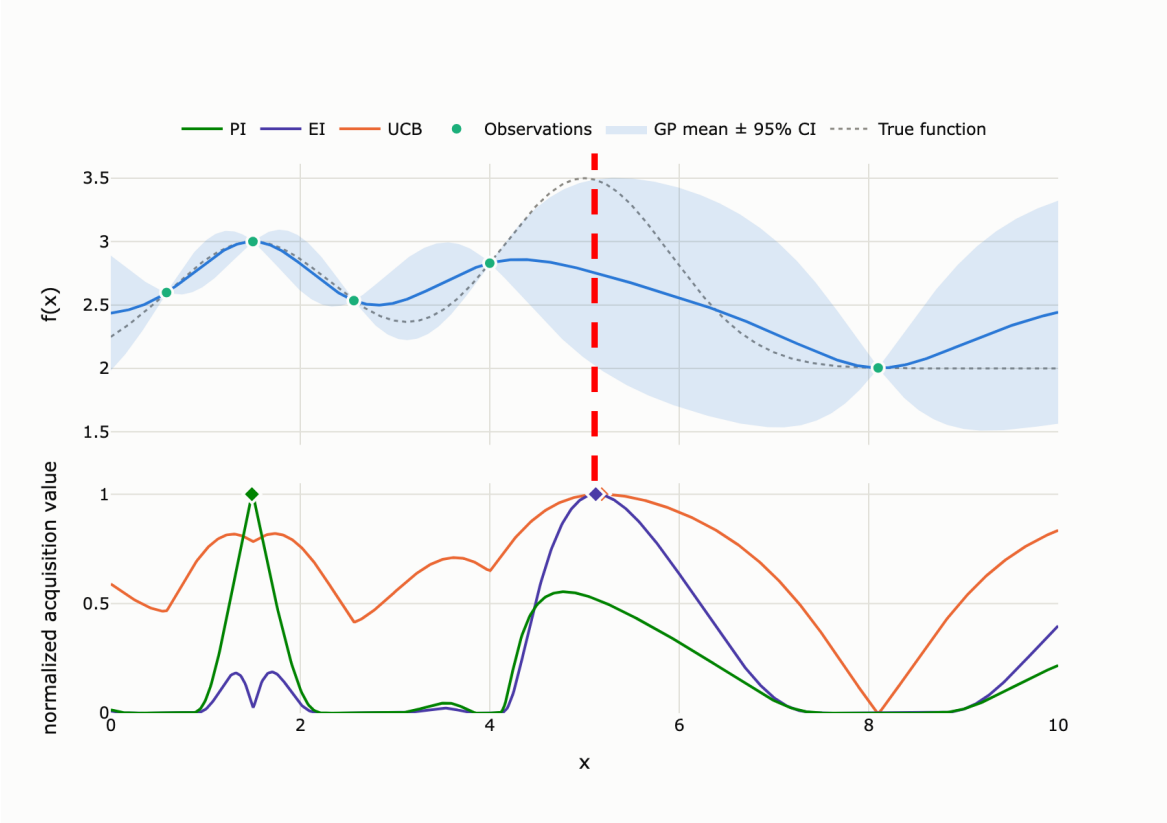


Acquisition Function
What's the difference?

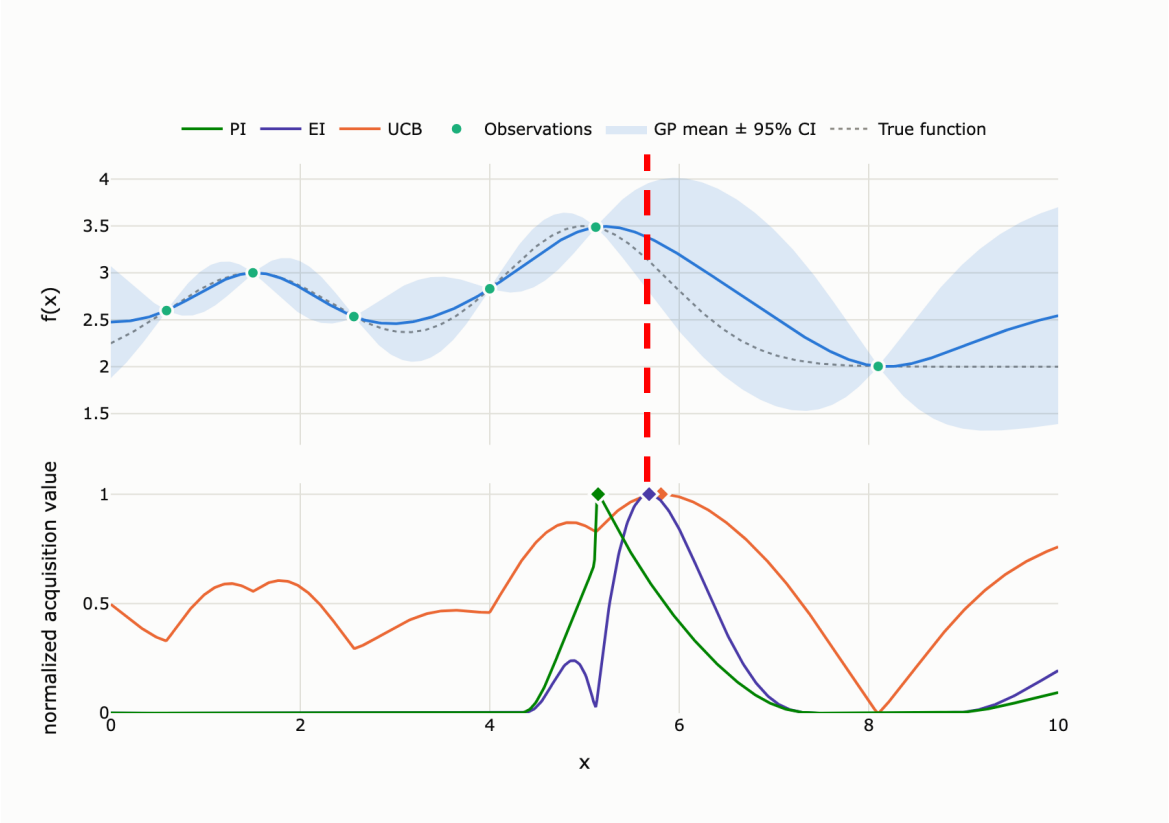


Acquisition Function

What's the difference?

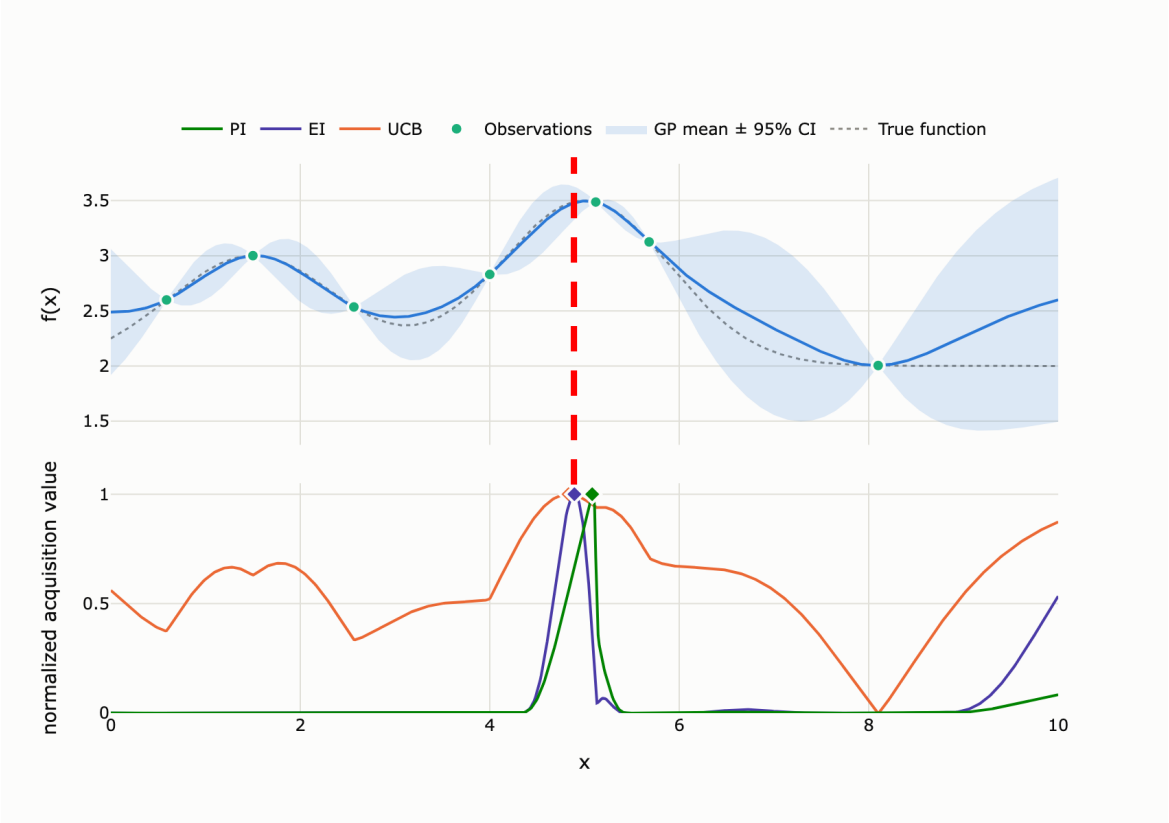


Acquisition Function
What's the difference?

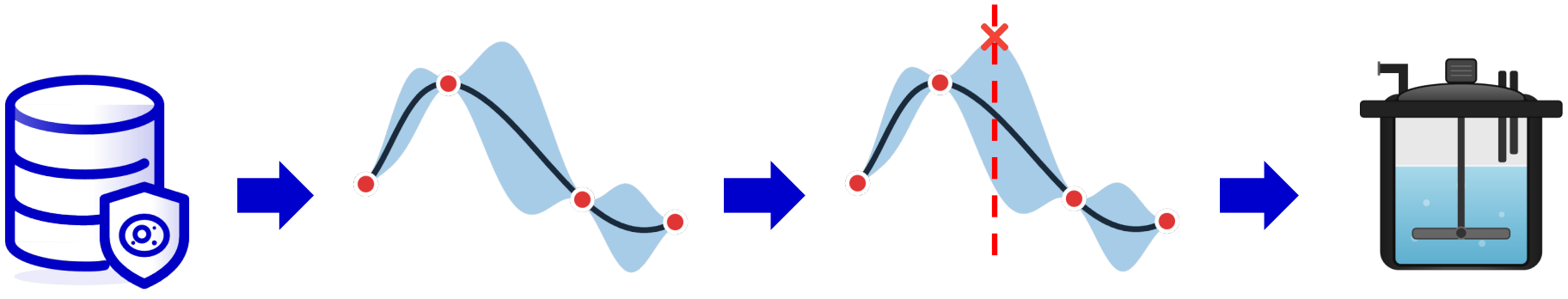


Acquisition Function

What's the difference?



Bayesian Optimization



Data

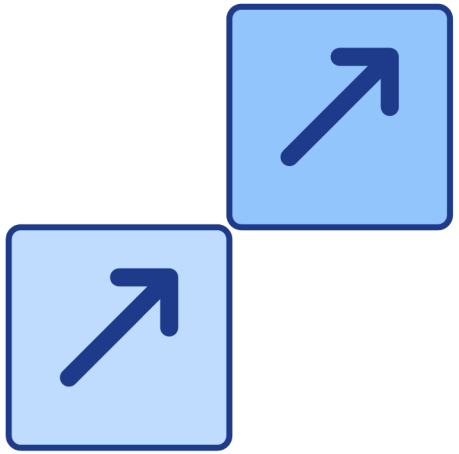
Train GP

Maximize
Acquisition Function

Run Experiments

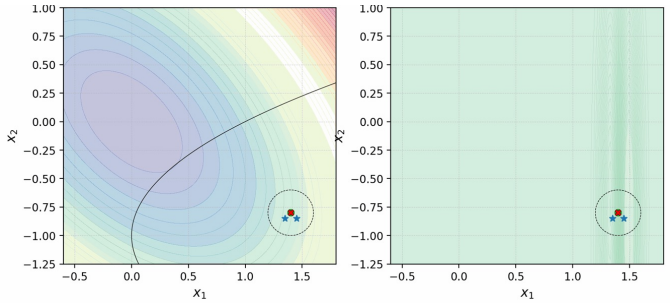
Active Learning

What else is there...

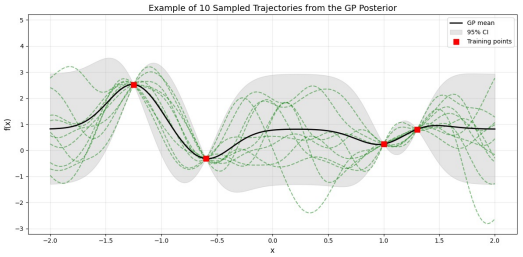


Safe, Batch & Multi-Fidelity Bayesian Optimization

Safe Bayesian Optimization

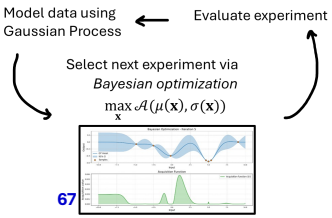


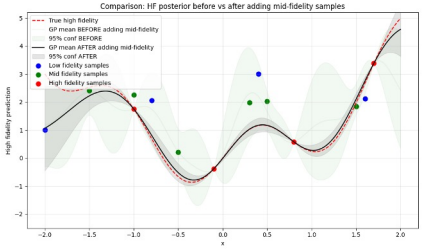
Batch Bayesian Optimization





Multi-Fidelity Bayesian Optimization

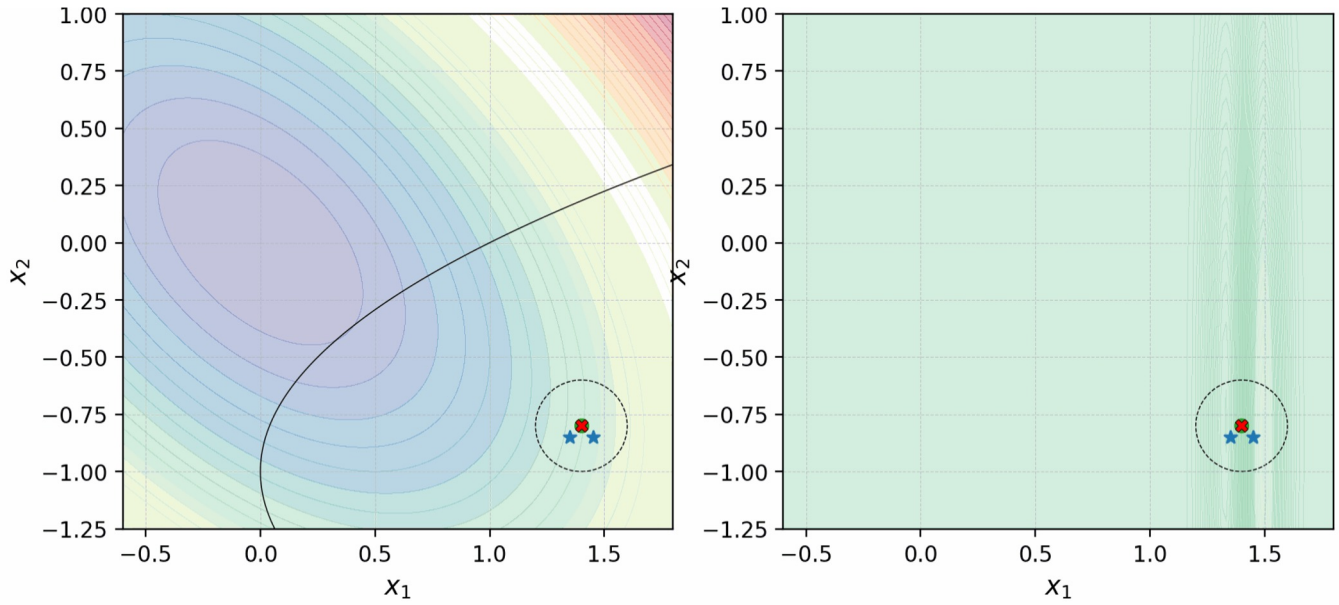




Safe Bayesian Optimization

By construction, Bayesian optimization is an inherently **exploratory algorithm**.

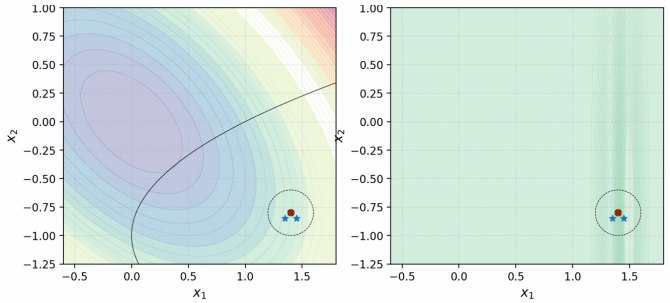
Safe Bayesian Optimization aims to **maximize an objective function** while ensuring, **with high confidence**, that all points satisfy **safety constraints**.



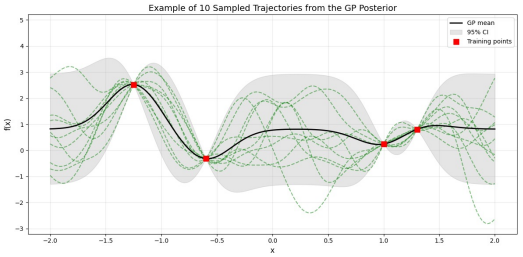
Real-time optimization meets Bayesian optimization and derivative-free optimization, A. del Rio Chanona, et al., 2021

Safe, Batch & Multi-Fidelity Bayesian Optimization

Safe Bayesian Optimization

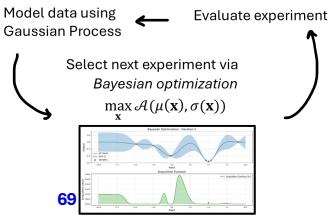


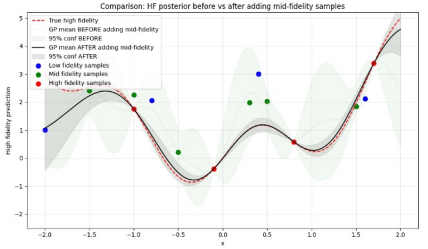
Batch Bayesian Optimization





Multi-Fidelity Bayesian Optimization





Batch Bayesian Optimization

Can we adapt Bayesian optimization to be used with **High Throughput Experimentation** (HTE)?

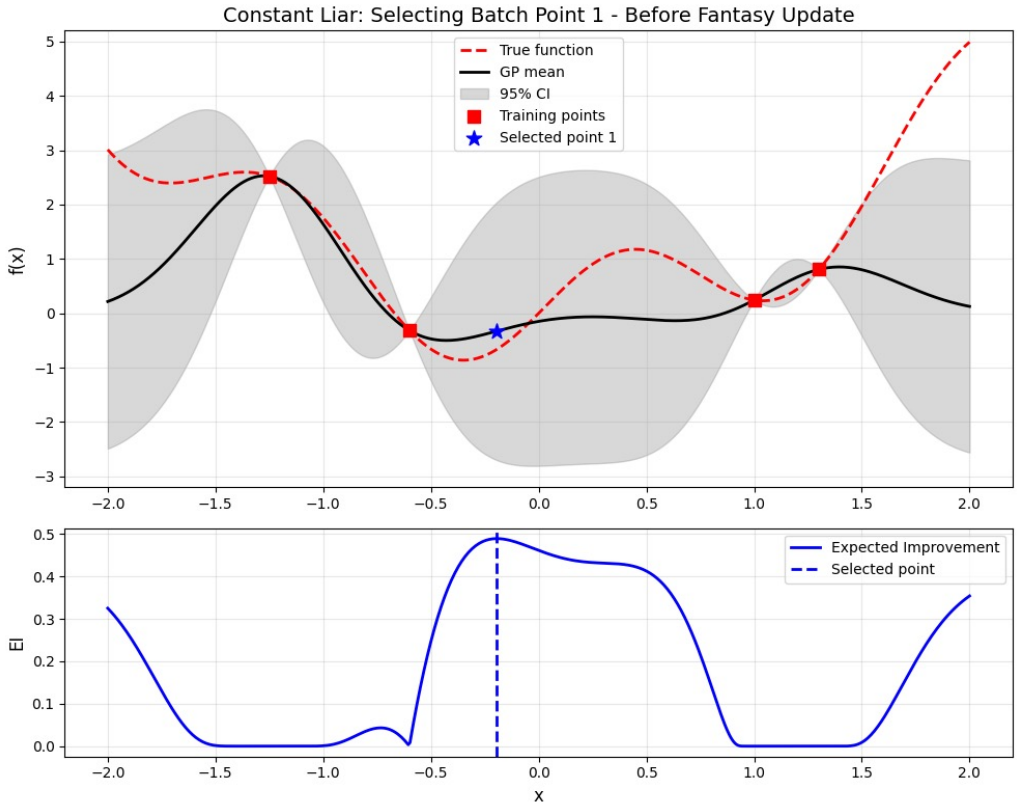
Batch Bayesian Optimization:

Multiple evaluations to be performed in parallel.



Batch Bayesian Optimization – Kriging believer

Instead of a fixed constant, you “**believe**” the GP prediction: i.e., you use the predicted mean of the GP.

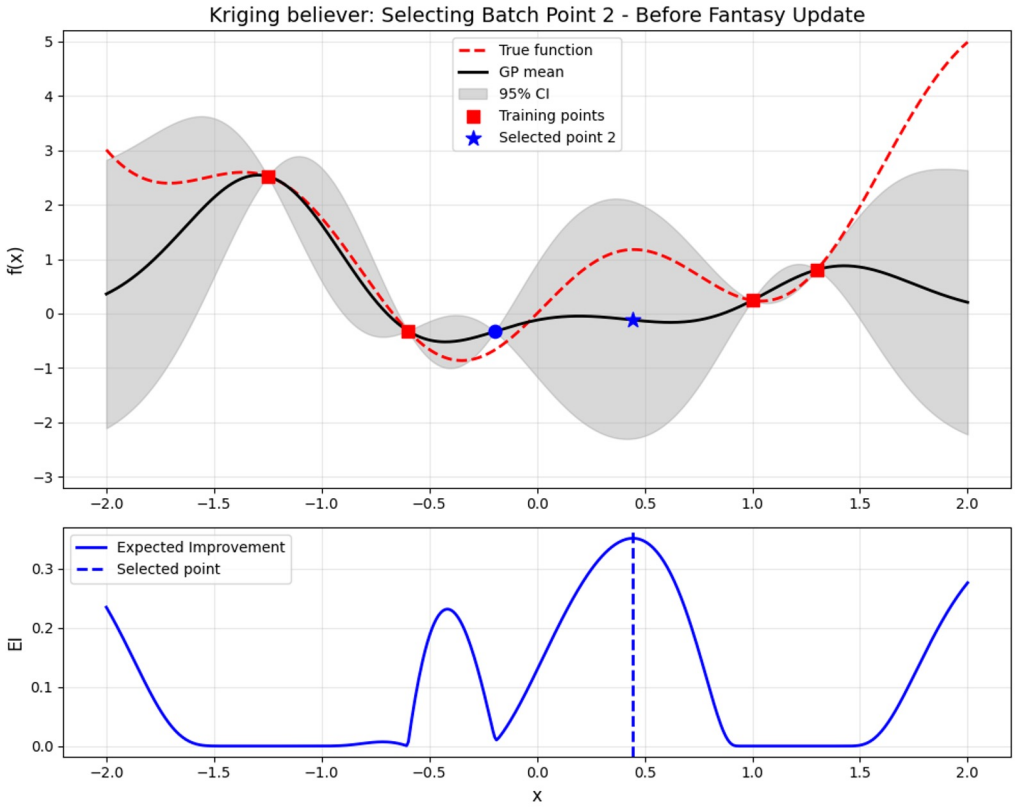


Choose point with **best Acquisition function value**, and update Gaussian process with **Kriging believer**.

Assumes mean predicted value by the Gaussian process.

Batch Bayesian Optimization – Kriging believer

Instead of a fixed constant, you “**believe**” the GP prediction: i.e., you use the predicted mean of the GP.

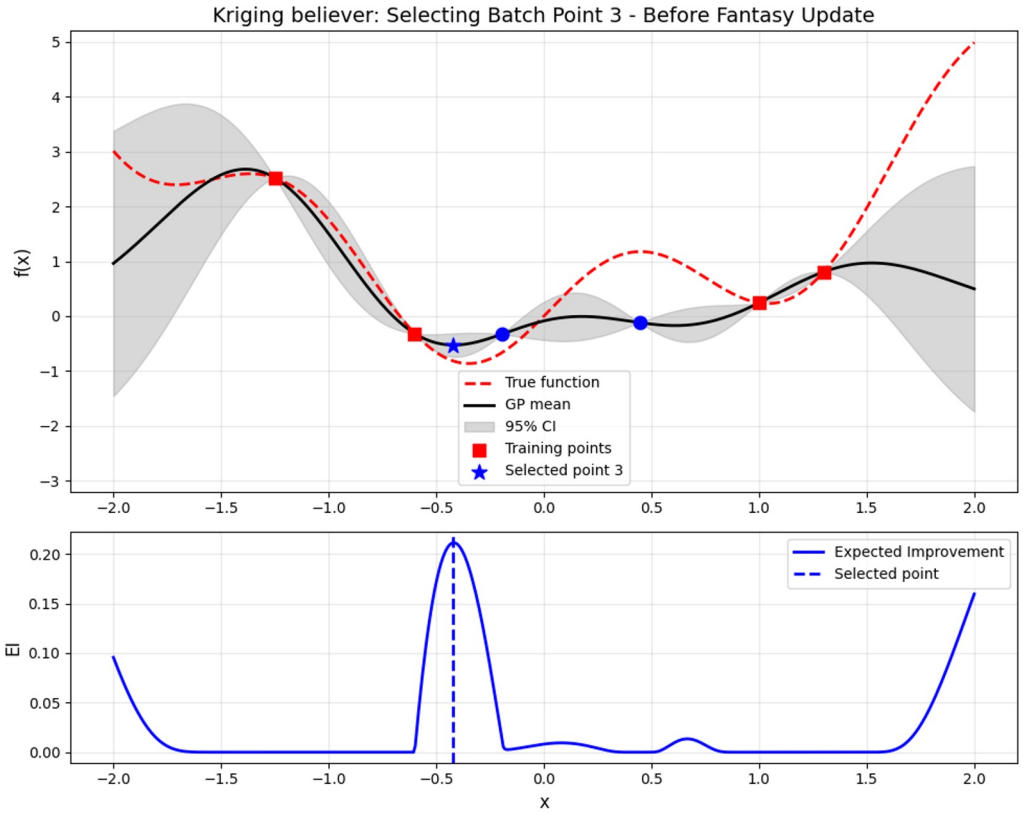


Choose point with **best Acquisition function value**, and update Gaussian process with **Kriging believer**.

Assumes mean predicted value by the Gaussian process.

Batch Bayesian Optimization – Kriging believer

Instead of a fixed constant, you “**believe**” the GP prediction: i.e., you use the predicted mean of the GP.

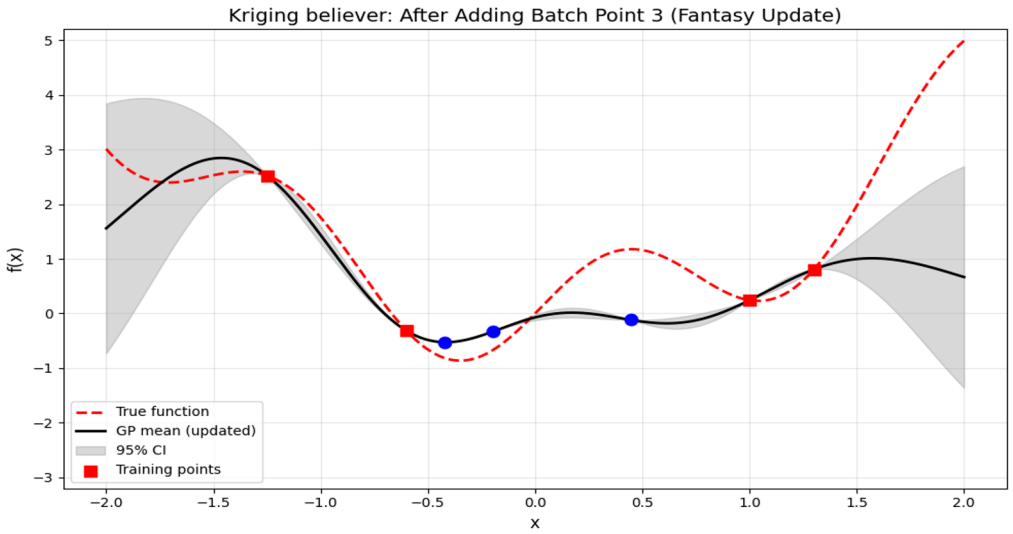


Choose point with **best Acquisition function value**, and update Gaussian process with **Kriging believer**.

Assumes mean predicted value by the Gaussian process.

Batch Bayesian Optimization – Kriging believer

Instead of a fixed constant, you “**believe**” the GP prediction: i.e., you use the predicted mean of the GP.

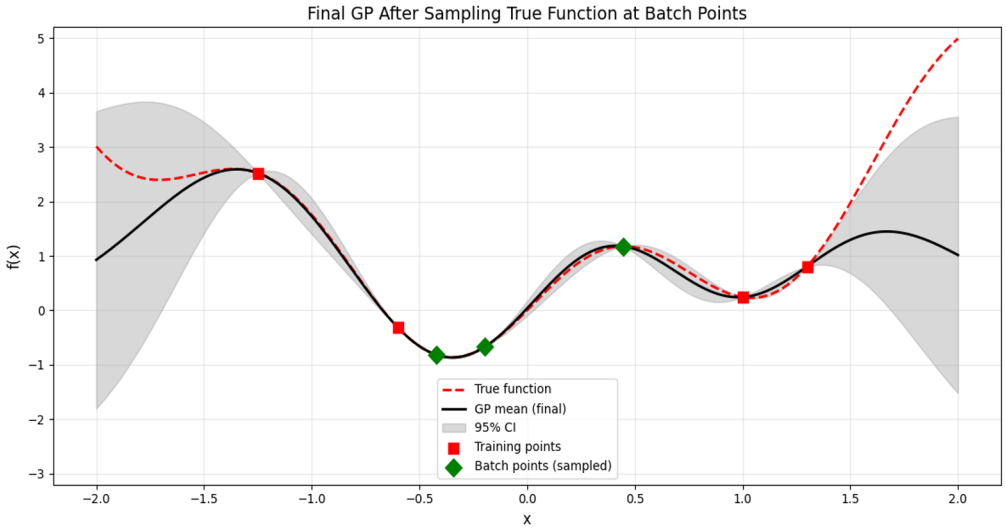


Choose point with **best Acquisition function value**, and update Gaussian process with **Kriging believer**.

Assumes mean predicted value by the Gaussian process.

Batch Bayesian Optimization – Kriging believer

Instead of a fixed constant, you "**believe**" the GP prediction: i.e., you use the predicted mean of the GP.



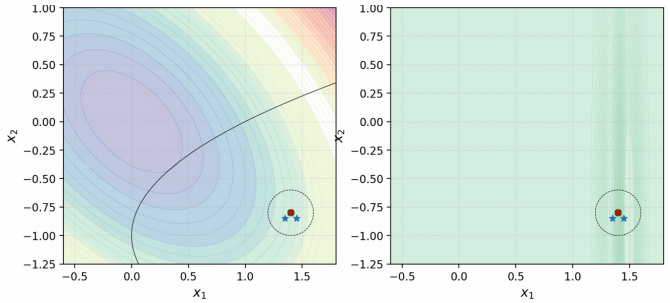
Choose point with **best Acquisition function value**, and update Gaussian process with **Kriging believer**.

Assumes mean predicted value by the Gaussian process.

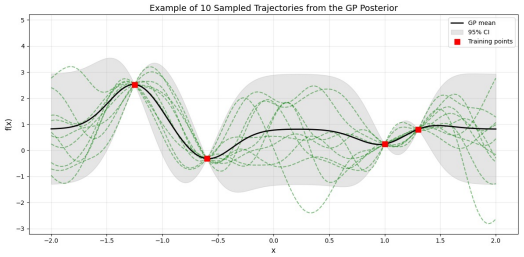
The batch is sampled from the **true** function.

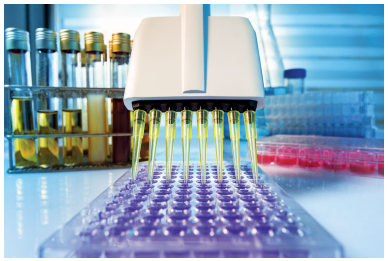
Safe, Batch & Multi-Fidelity Bayesian Optimization

Safe Bayesian Optimization

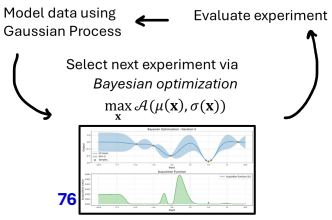


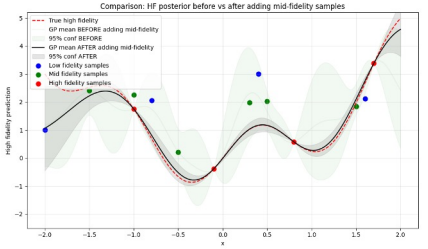
Batch Bayesian Optimization





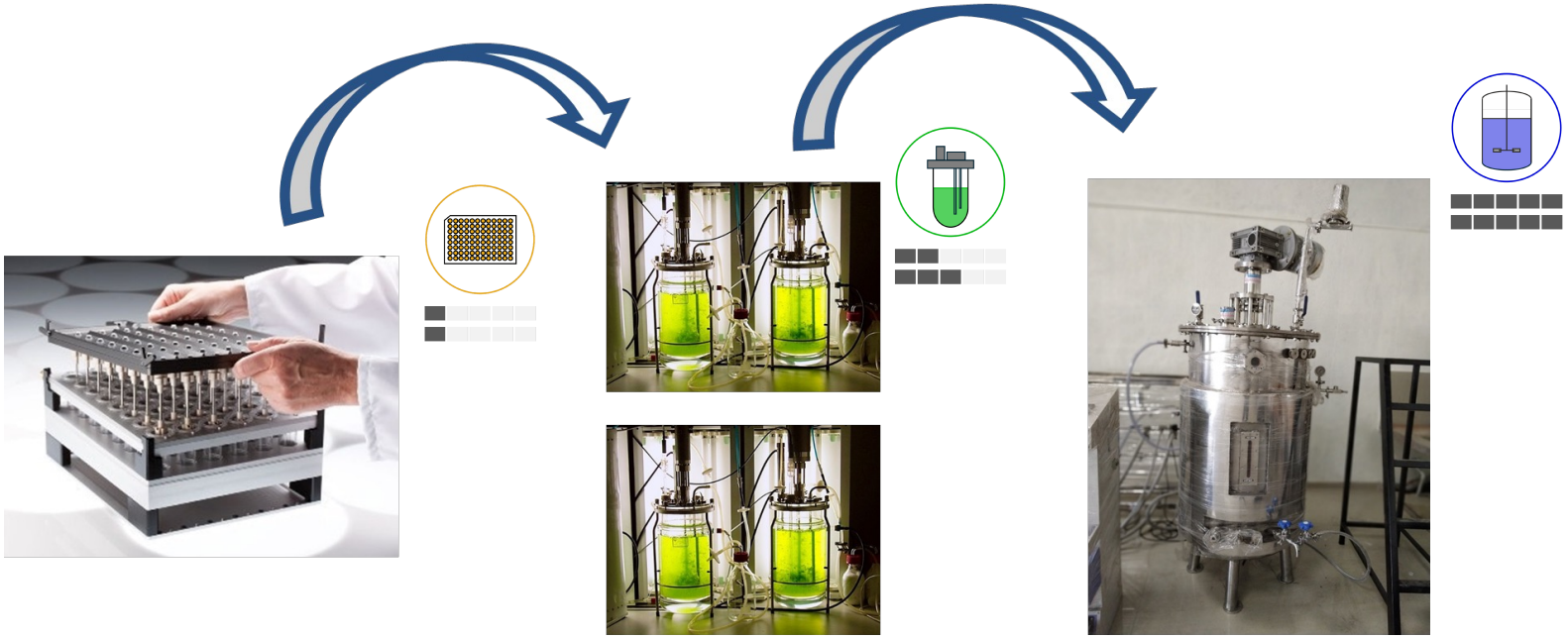
Multi-Fidelity Bayesian Optimization





Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?



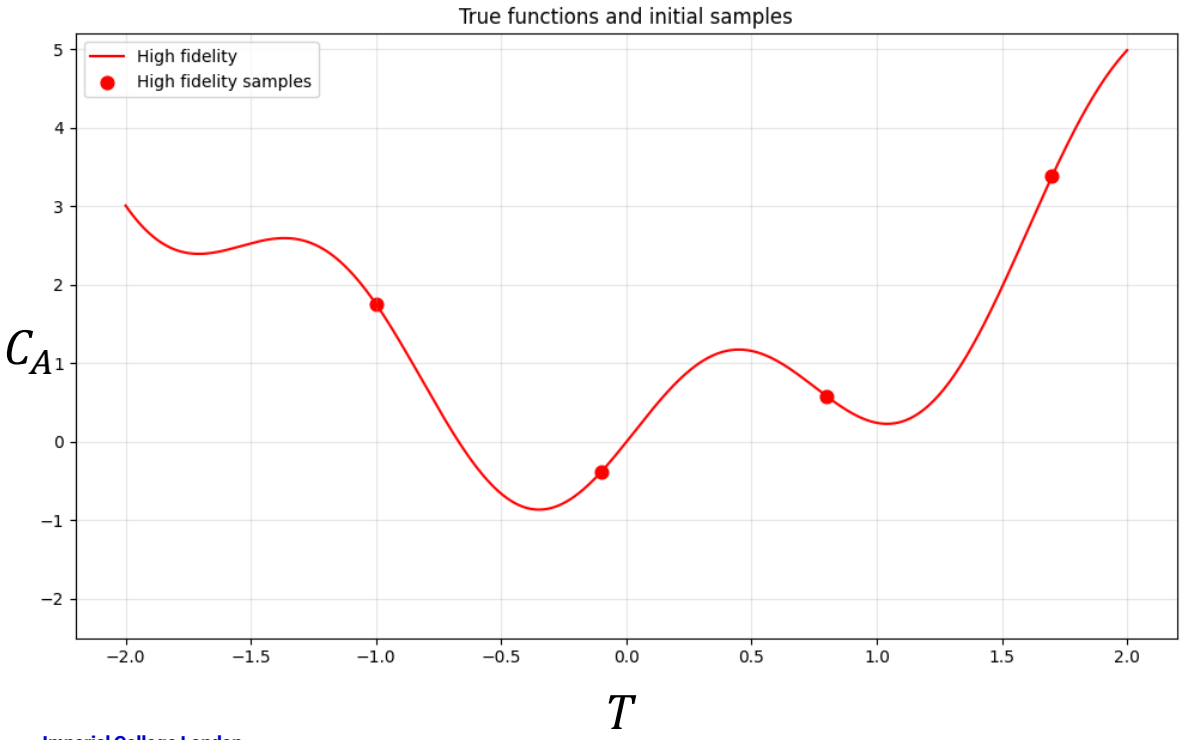
$
low information

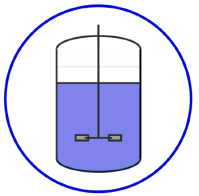
$$
mid information

$$$
high information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

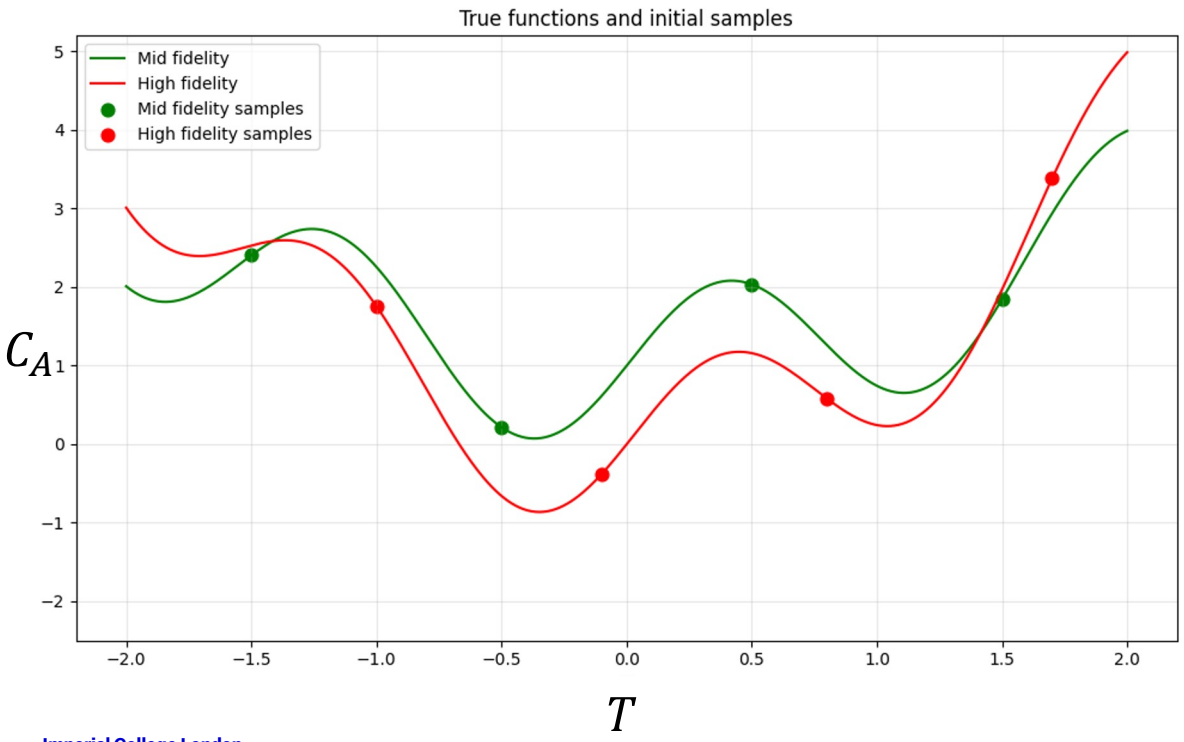
cost

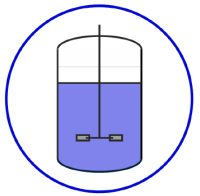
information



Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information



Amber scale:

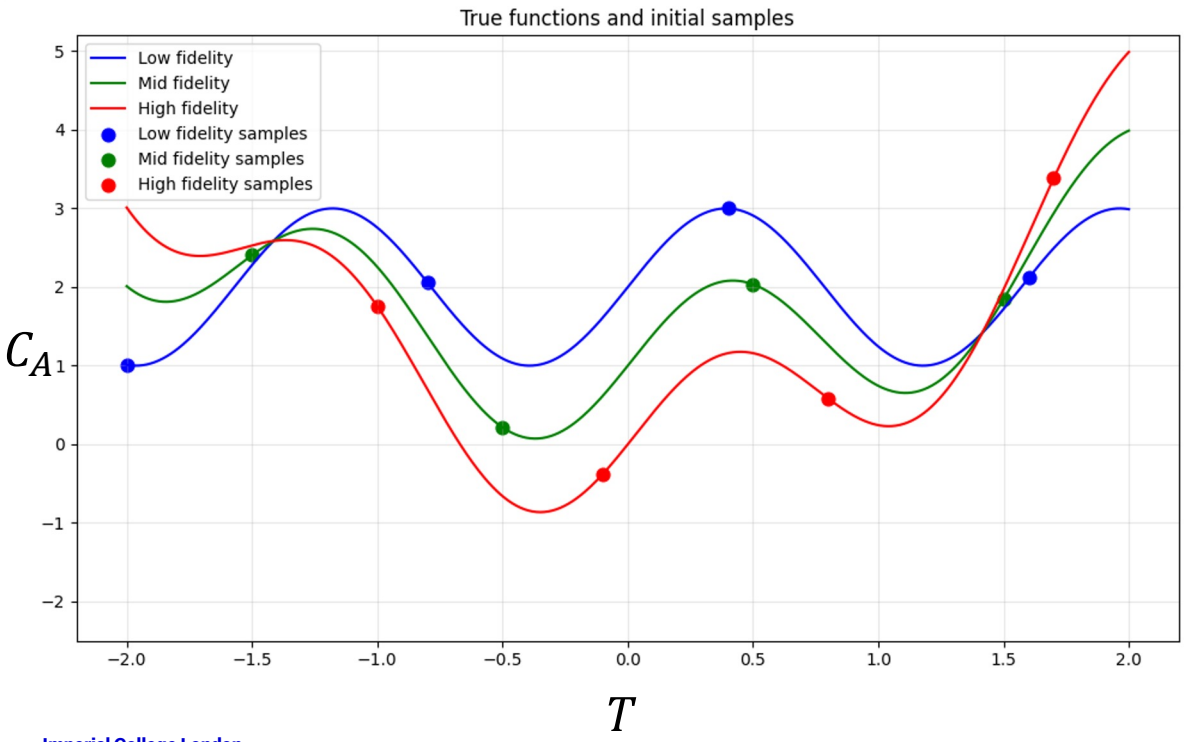
Mid fidelity

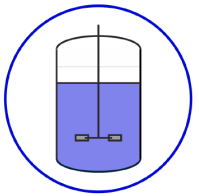
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information

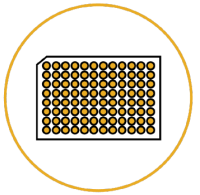


Amber scale:

Mid fidelity

cost

information



Micro titer scale:

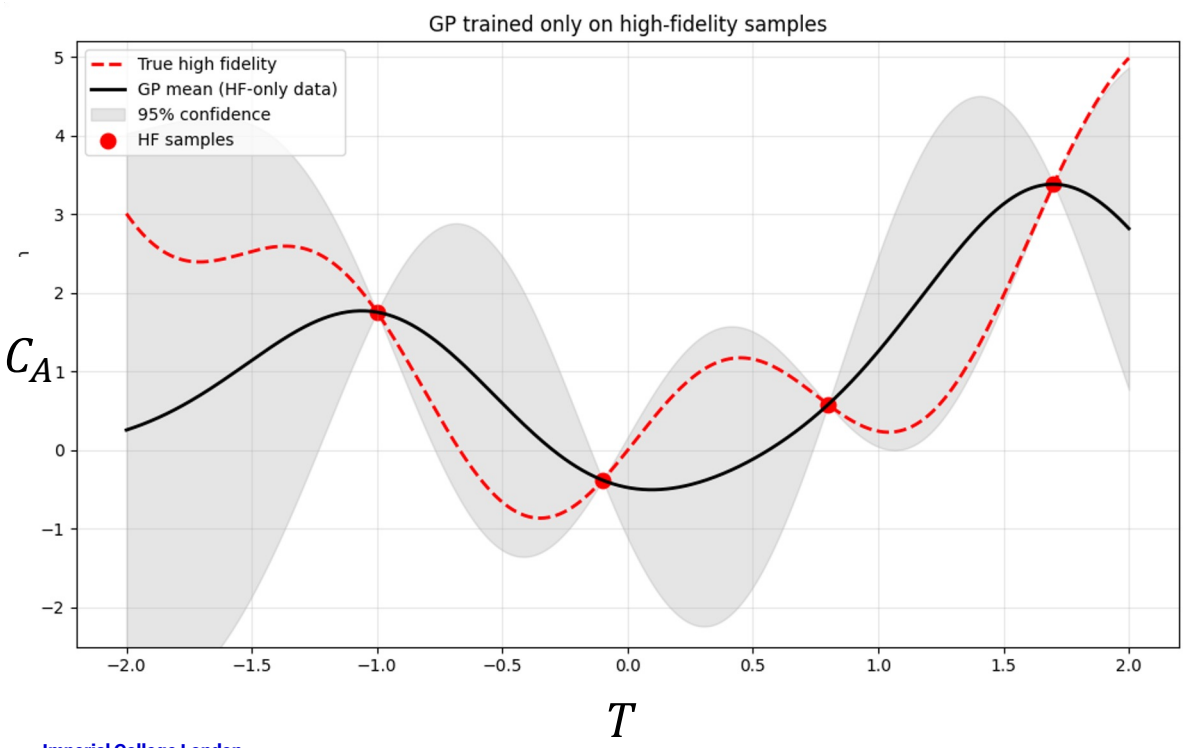
Low fidelity

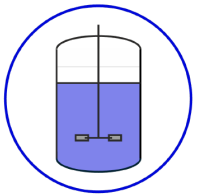
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

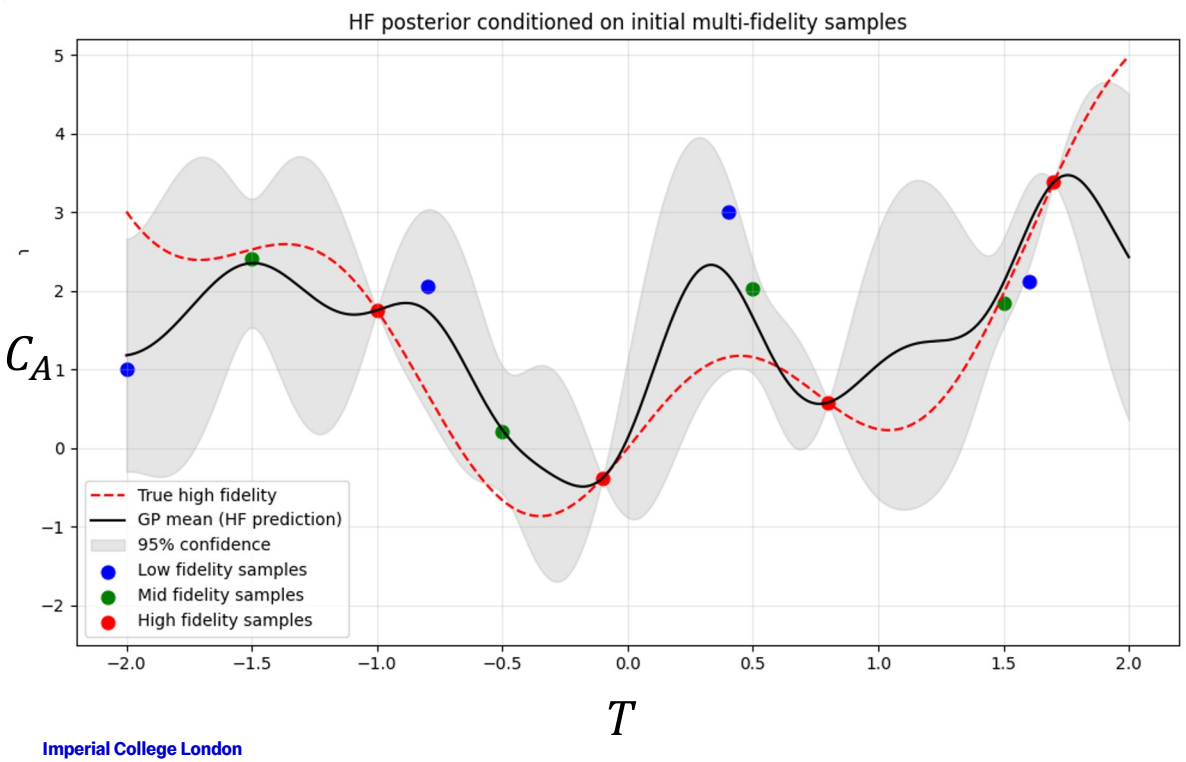
cost

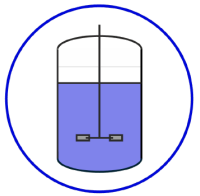
information



Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information

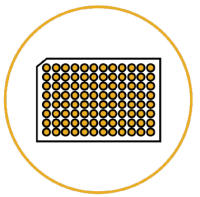


Amber scale:

Mid fidelity

cost

information



Micro titer scale:

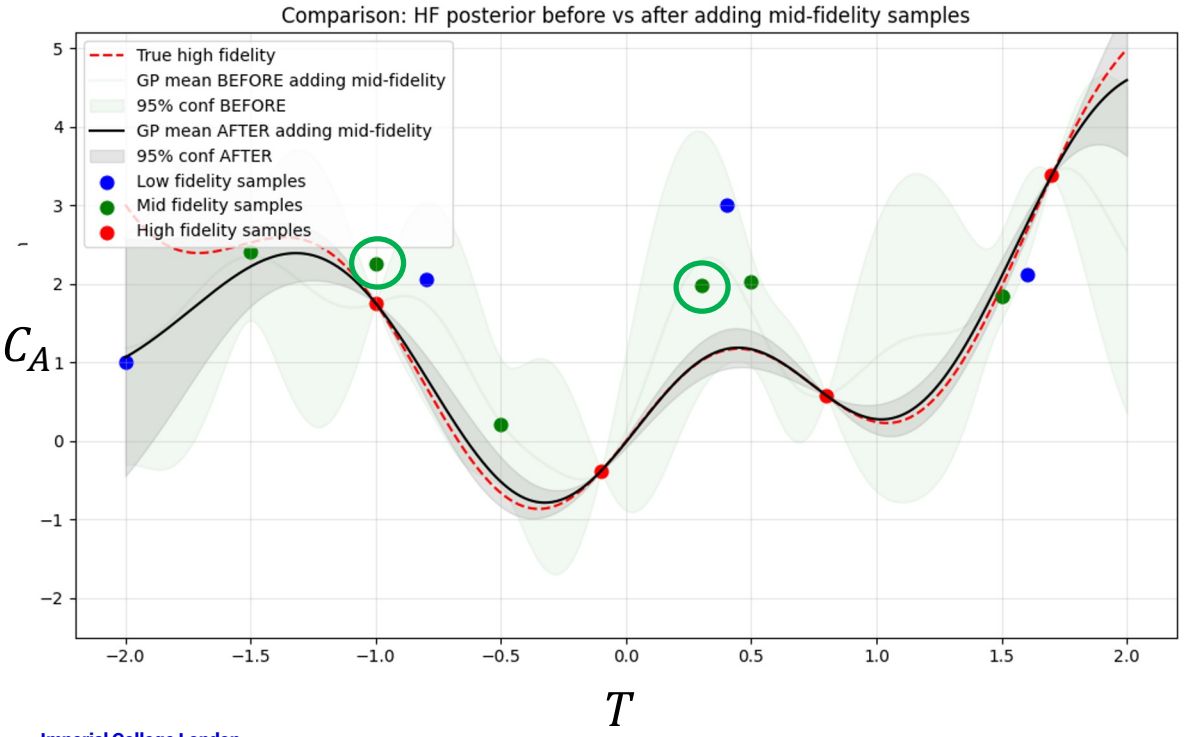
Low fidelity

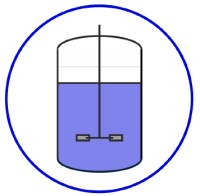
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information

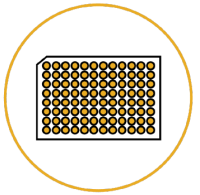


Amber scale:

Mid fidelity

cost

information



Micro titer scale:

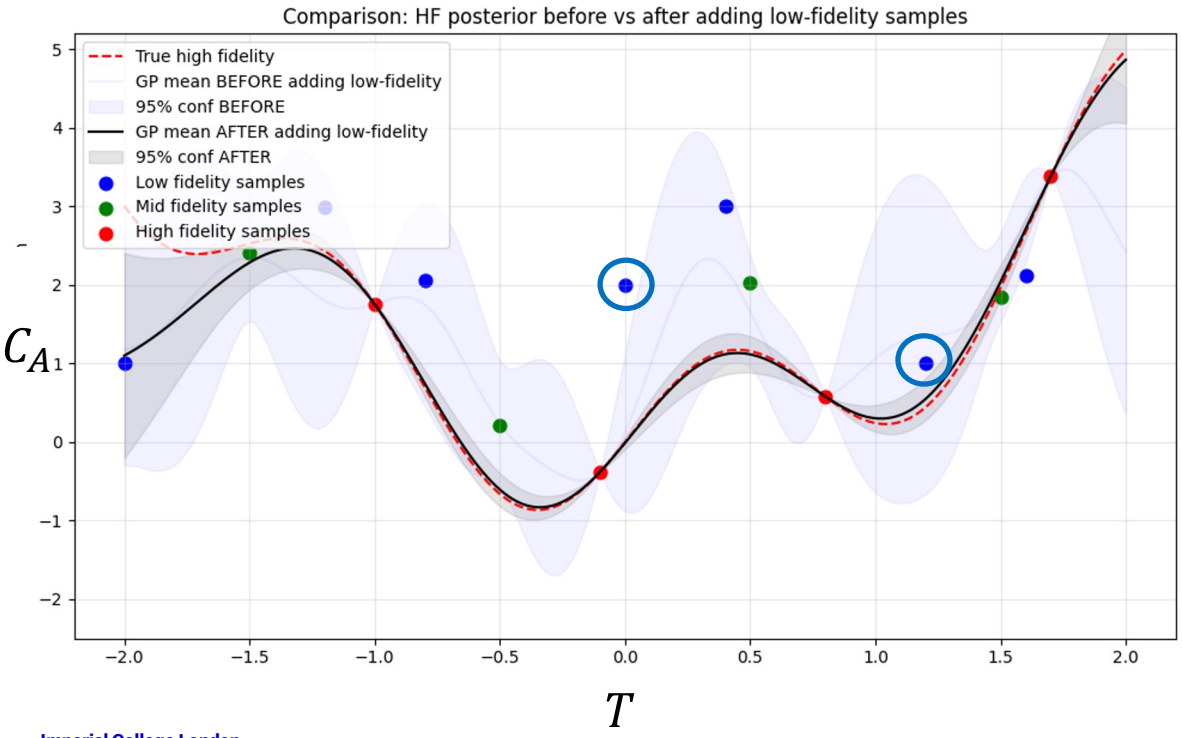
Low fidelity

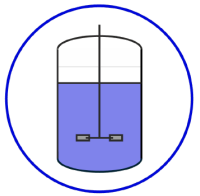
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information

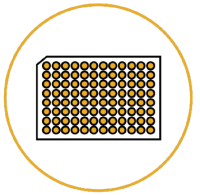


Amber scale:

Mid fidelity

cost

information



Micro titer scale:

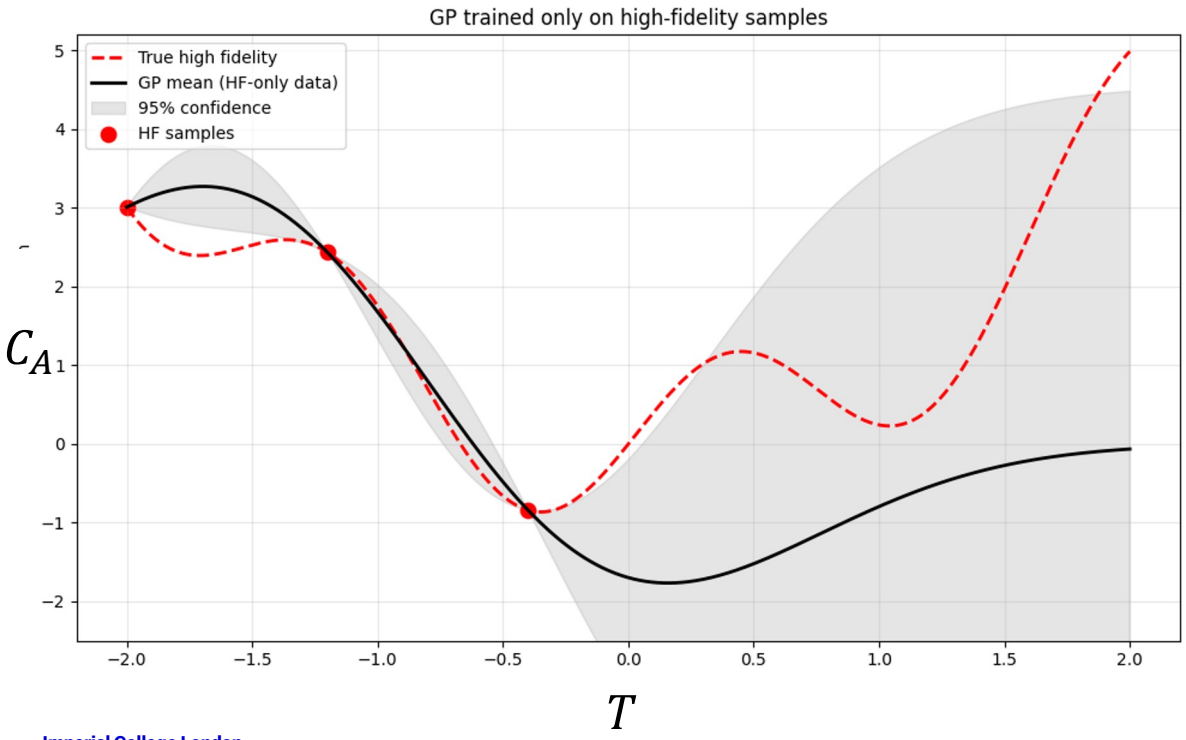
Low fidelity

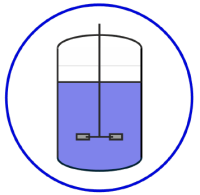
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

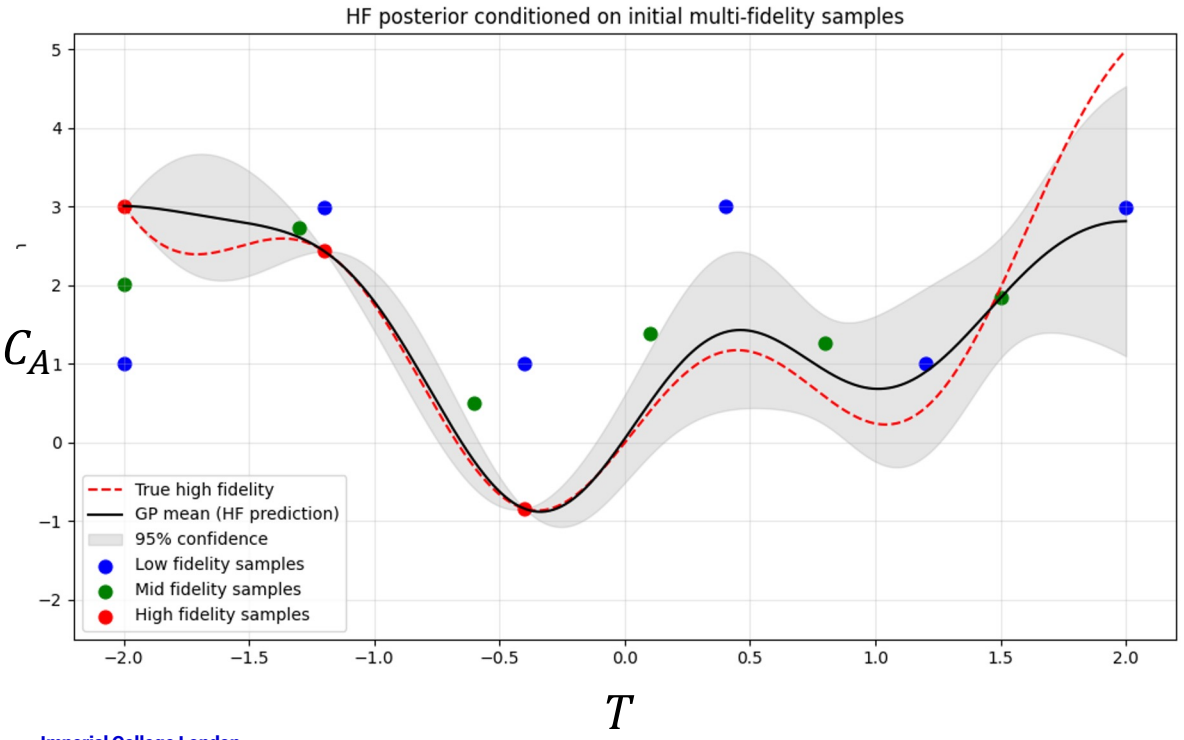
cost

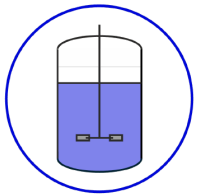
information



Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?





Pilot scale:

High fidelity

cost

information

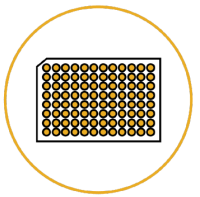


Amber scale:

Mid fidelity

cost

information



Micro titer scale:

Low fidelity

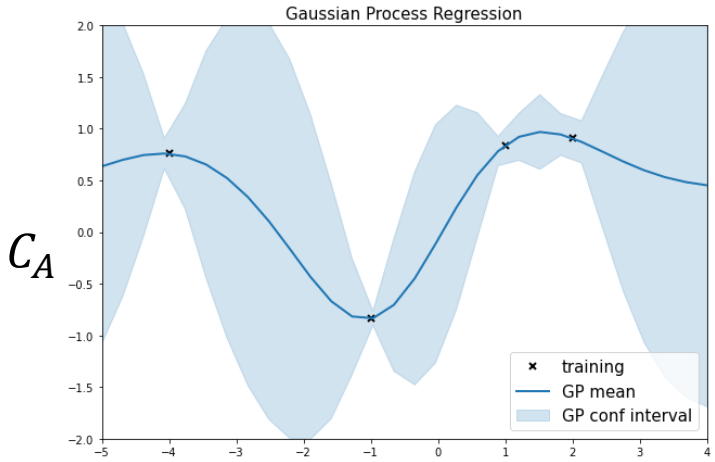
cost

information

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



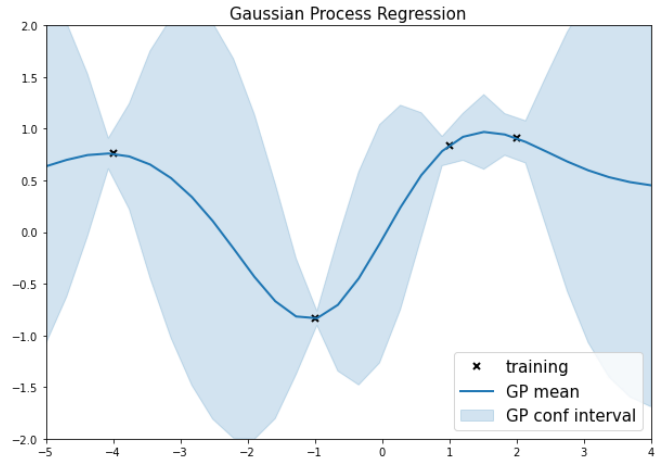
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



C_A

$$\underbrace{\mathcal{A}_{\text{ext}}(\mathbf{x}, \mathbf{z})}_{\text{Next scale \& } T} = \frac{\mu_{\hat{f}}(\mathbf{x}, \mathbf{z}^*) - \beta\sigma_{\hat{f}}(\mathbf{x}, \mathbf{z}^*)}{c(\mathbf{z})} k\big((\mathbf{x}, \mathbf{z}), (\mathbf{x}, \mathbf{z}^*)\big)$$

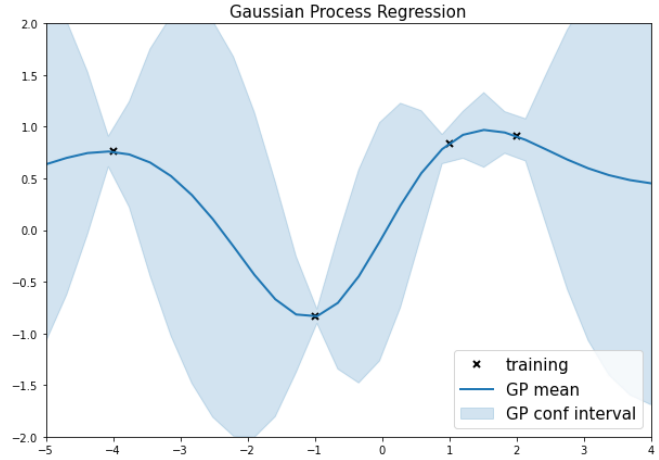
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



C_A

Acquisition function

$$\underbrace{\mathcal{A}_{\text{ext}}(\mathbf{x}, \mathbf{z})}_{\text{Next scale \& } T} = \frac{\overbrace{\mu_{\hat{f}}(\mathbf{x}, \mathbf{z}^*) - \beta\sigma_{\hat{f}}(\mathbf{x}, \mathbf{z}^*)}}{c(\mathbf{z})} k\big((\mathbf{x}, \mathbf{z}), (\mathbf{x}, \mathbf{z}^*)\big)$$

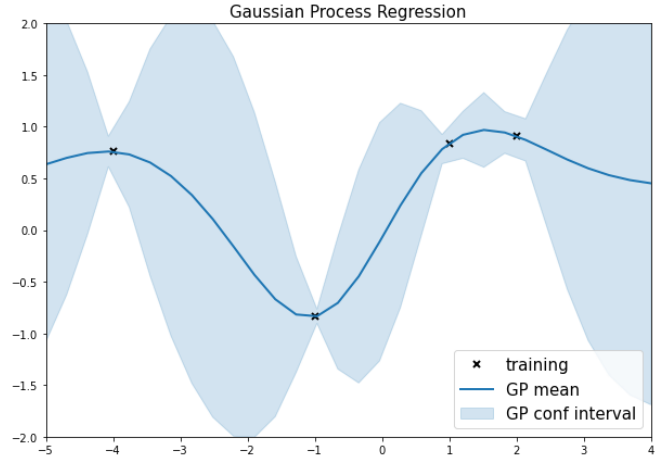
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



C_A

Acquisition function

$$\underbrace{\mathcal{A}_{\text{ext}}(\mathbf{x},\mathbf{z})}_{\text{Next scale \& } T} = \frac{\overbrace{\mu_{\hat{f}}(\mathbf{x},\mathbf{z}^*) - \beta\sigma_{\hat{f}}(\mathbf{x},\mathbf{z}^*)}}{\underbrace{c(\mathbf{z})}_{\text{cost}}} k\big((\mathbf{x},\mathbf{z}),(\mathbf{x},\mathbf{z}^*)\big)$$

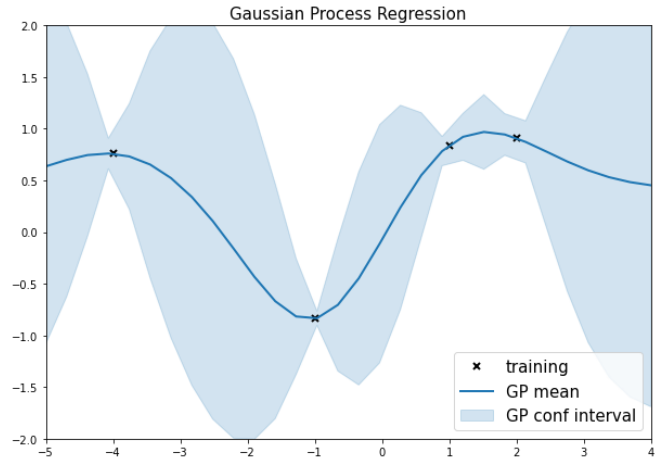
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



C_A

Acquisition function

$$\underbrace{\mathcal{A}_{\text{ext}}(\mathbf{x},\mathbf{z})}_{\text{Next scale \& } T} = \frac{\overbrace{\mu_{\hat{f}}(\mathbf{x},\mathbf{z}^*) - \beta\sigma_{\hat{f}}(\mathbf{x},\mathbf{z}^*)}}{\underbrace{c(\mathbf{z})}_{\text{cost}}} \underbrace{k\big((\mathbf{x},\mathbf{z}),(\mathbf{x},\mathbf{z}^*)\big)}_{\text{loss of information}}$$

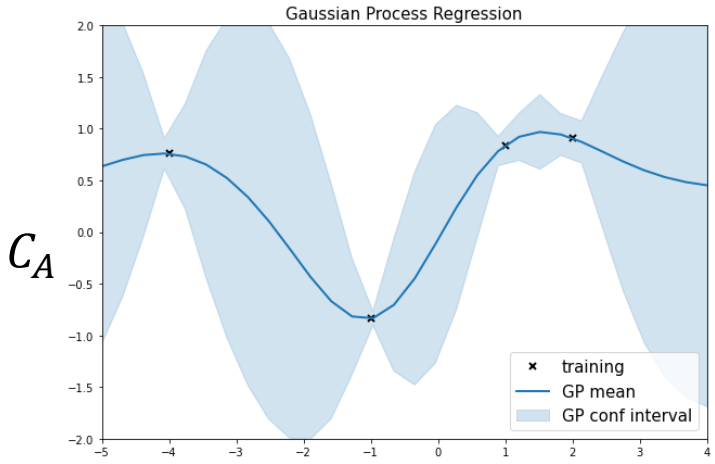
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

Multi-Fidelity Bayesian Optimization

Can we adapt Bayesian optimization to be used for different experimental scales?

How does multi-fidelity BO work?



C_A

Acquisition function

$$\underbrace{\mathcal{A}_{\text{ext}}(\mathbf{x}, \mathbf{z})}_{\text{Next scale \& } T} = \frac{\overbrace{\mu_{\hat{f}}(\mathbf{x}, \mathbf{z}^*) - \beta\sigma_{\hat{f}}(\mathbf{x}, \mathbf{z}^*)}^{}}{\underbrace{c(\mathbf{z})}_{\text{cost}}} \underbrace{k\big((\mathbf{x}, \mathbf{z}), (\mathbf{x}, \mathbf{z}^*)\big)}_{\text{loss of information}}$$

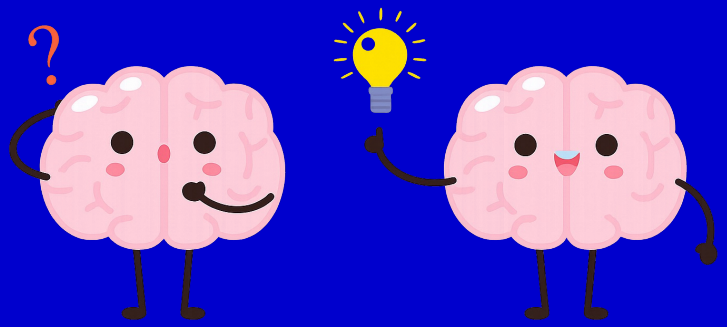
Decision variables → $\mathbf{x}, \mathbf{z}$ ← Fidelities are treated as inputs

T, scale

*For discrete fidelities (like here), you can also look into multi-task (multi-output) GPs.

IMPERIAL

Questions



Sargent Centre for Process Systems Engineering

AISL Autonomous Industrial Systems Lab

OPTIML PSE

Samuel Stricker

1.4.1 Gaussian Processes

GPs provide the probabilistic foundation for many BO workflows because they combine flexible non-parametric regression with explicit uncertainty estimates [21]. A GP is fully specified by a mean function $\mu(\mathbf{x})$ and a covariance (kernel) function $k(\mathbf{x}, \mathbf{x}')$:

$$f(\mathbf{x}) \sim \mathcal{GP}\big(\mu(\mathbf{x}),\, k(\mathbf{x}, \mathbf{x}')\big)\,. \tag{1.1}$$

The kernel encodes assumptions about the smoothness and structure of f: points close in input space are assigned high covariance, reflecting the belief that nearby inputs should produce similar outputs [21]. Given observations $\mathcal{D}_t$ with Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$, Bayesian conditioning yields a closed-form posterior predictive distribution at a new point $\mathbf{x}_*$:

$$\mu_d(\mathbf{x}_*) = \mathbf{k}_*^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}, \qquad \sigma_d^2(\mathbf{x}_*) = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}_*, \tag{1.2}$$

where $\mathbf{K} = k(\mathbf{X}, \mathbf{X})$ is the covariance matrix of the observed inputs and $\mathbf{k}_* = k(\mathbf{X}, \mathbf{x}_*)$ [21]. The posterior mean $\mu_d(\mathbf{x})$ acts as the current best estimate of process performance, while the posterior standard deviation $\sigma_d(\mathbf{x})$ quantifies remaining uncertainty. Kernel hyperparameters are typically selected by maximizing the marginal log-likelihood of the observations [21].